Politechnika
Śląska

PRACA MAGISTERSKA

Układ aktywnego tłumienia hałasu pod nadzorem jądra systemu Linux
z rozszerzeniami real-time

Daniel KOPIECKI
Nr albumu: 281324

Kierunek: Automatyka i Robotyka
Specjalność: Systemy Pomiarowe i Informacyjne

PROWADZĄCY PRACĘ
dr inż. Krzysztof Mazur
KATEDRA Pomiarów i Systemów Sterowania
Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2023

Tytuł pracy

Układ aktywnego tłumienia hałasu pod nadzorem jądra systemu Linux z rozszerzeniami real-time

Streszczenie

Praca opisuje proces projektowania i implementacji układu aktywnej redukcji hałasu na bazie wbudowanego systemu Linux z roszerzeniami czasu rzeczywistego w oparciu o samodzielnie skonstruowaną elektronikę komunikującą się z zainstalowanym na platformie obliczeniowej systemem operacyjnym i aplikacją działającą w trybie użytkownika. W treści szczegółowo wyjaśnione zostały zasady działania systemów aktywnej redukcji hałasu, przedstawiony został przebieg realizacji części sprzętowej i programowej układu wraz z weryfikacją ich działania. Na skonstruowanym układzie wykonany został zestaw badań sprawdzający wpływ konfiguracji systemu na uzyskany poziom tłumienia oraz przedstawione zostały obserwacje dotyczące możliwych usprawnień.

Słowa kluczowe

aktywne tłumienie hałasu, filtracja adaptacyjna, identyfikacja systemów, jądro Linux, system czasu rzeczywistego, płytka drukowana

Thesis title

Active noise reduction system under the supervision of the Linux kernel with real-time extensions

Abstract

The thesis describes the design and implementation process of active noise control system based on embedded Linux with real-time extensions using self-constructed electronic circuit communicating with operating system installed on chosen computing platform and user-space application. The content explains in detail the principles of active noise control systems, the course of hardware and software parts implementation was presented along with verificiation of their operation. On the constructed system a set of tests was performed to check the influence of system configuration on the obtained attenuation level. Finally, observations regarding possible improvements have been made and presented.

Key words

active noise control, adaptive filtering, Linux kernel, system identification, real-time operating system, printed circuit board

Spis treści

1 Wstęp **1**

1.1 Wprowadzenie . . . 1

1.2 Uogólniona koncepcja . . . 2

1.2.1 Obszar zastosowania . . . 2

1.2.2 Idea przewodnia . . . 3

1.3 Określenie problematyki . . . 4

1.3.1 Cel projektu . . . 5

1.4 Zwięzła charakterystyka rozdziałów . . . 6

2 Analiza tematu **7**

2.1 Struktury systemów . . . 7

2.2 Zasada działania . . . 9

2.2.1 Identyfikacja toru pierwotnego . . . 9

2.2.2 Algorytm LMS . . . 11

2.2.3 Wpływ toru wtórnego . . . 13

2.2.4 Algorytm FXLMS . . . 15

2.2.5 Usprawnienie procesu doboru kroku . . . 17

2.2.6 Metody identyfikacji toru wtórnego . . . 19

2.2.7 Neutralizacja sprzężenia — możliwe usprawnienie . . . 21

3 Założenia i proponowane rozwiązanie **23**

3.1 Wymagania projektowe . . . 23

3.1.1 Część sprzętowa . . . 24

3.1.2 Część programowa . . . 27

3.2 Architektura systemu . . . 28

3.3 Wykorzystane narzędzia . . . 30

4 Implementacja części sprzętowej **31**

4.1 Urządzenia zewnętrzne . . . 32

4.1.1 Mikrofony . . . 32

4.1.2 Zestaw głośnikowy . . . 33

4.1.3 Platforma obliczeniowa . . . 33
4.2 Zasilanie . . . 34
4.2.1 Sekcja wejściowa . . . 34
4.2.2 Symetryzacja szyn zasilających . . . 36
4.2.3 Zasilanie domeny cyfrowej . . . 38
4.2.4 Referencja domeny analogowo-cyfrowej . . . 39
4.2.5 Generacja składowej stałej . . . 39
4.3 Wejściowe tory analogowe . . . 40
4.3.1 Wejście zbalansowane . . . 40
4.3.2 Początkowa filtracja i saturacja . . . 41
4.3.3 Wzmocnienie różnicowe . . . 42
4.3.4 Filtracja antyaliasingowa . . . 43
4.3.5 Pozycjonowanie . . . 46
4.3.6 Filtr końcowy i saturacja . . . 47
4.3.7 Pełen schemat sekcji . . . 48
4.4 Wyjściowe tory analogowe . . . 48
4.4.1 Filtracja początkowa . . . 48
4.4.2 Wzmocnienie . . . 49
4.4.3 Filtracja rekonstrukcyjna . . . 50
4.4.4 Symetryzacja . . . 50
4.4.5 Filtracja końcowa . . . 51
4.4.6 Pełen schemat sekcji . . . 51
4.5 Sekcja cyfrowa . . . 52
4.5.1 Interfejs cyfrowy . . . 52
4.5.2 Interakcja z przetwornikami . . . 54
4.6 Dodatkowe kwestie sprzętowe . . . 57

5 Implementacja części programowej 61
5.1 Rozszerzenia czasu rzeczywistego . . . 61
5.1.1 Problemy jądra Linux w kontekście real-time . . . 61
5.1.2 Zestaw łatek PREEMPT_RT . . . 63
5.2 System operacyjny . . . 65
5.2.1 Środkowisko pracy . . . 66
5.2.2 Przygotowanie nośnika . . . 66
5.2.3 Programy rozruchowe . . . 67
5.2.4 Jądro Linux . . . 68
5.2.5 *Device tree* . . . 68
5.2.6 System plików . . . 70
5.3 Aplikacja . . . 70

5.4 System ALSA - biblioteka *libasound* 71
5.5 Biblioteki użytkownika 73
5.5.1 Biblioteka *libdsp* 73
5.5.2 Biblioteka *libsignal* 75
5.6 Główna pętla programowa 77

6 Testy i badania **79**
6.1 Weryfikacja części sprzętowej 79
6.1.1 Sekcja zasilania 80
6.1.2 Wyjściowe sekcje analogowe i przetworniki 80
6.1.3 Wejściowe sekcje analogowe i przetworniki 82
6.2 Weryfikacja części programowej 84
6.2.1 Identyfikacja toru wtórnego 85
6.2.2 Badania procesu ANC - wpływ parametrów 85
6.2.3 Dodatkowe eksperymenty i podsumowanie badań 97

7 Podsumowanie **103**

Bibliografia **107**

Spis skrótów i symboli **111**

Pliki uzupełniające **113**

Spis rysunków **117**

Spis tabel **119**

Rozdział 1

Wstęp

1.1 Wprowadzenie

Wstępne koncepcje systemów realizujących aktywne tłumienie hałasu (ang. *Active noise control*, ANC) zaczęły pojawiać się już w latach 30. XX wieku, jednak z powodu braku odpowiednich narzędzi pozwalających na praktyczną implementację rozwiązania w tamtych czasach, znaczące postępy zaczęto obserwować dopiero w latach 80. i 90. XX wieku, kiedy raptowny rozwój technologiczny umożliwił projektowane coraz to dokładniejszych i bardziej niezawodnych układów automatyki [11] Powodem takiego przebiegu wydarzeń był wyłaniający się zza horyzontu początek epoki cyfrowej, która udowodniła, że z użyciem kodu binarnego można opisać i wpływać na otaczający nas świat z dużą precyzją i elastycznością, wynikającą głównie z determinizmu błędów przetwarzania, na których wielkość projektant ma pełen wpływ. Zastosowanie układów cyfrowych w systemach ANC umożliwiło ich szersze wykorzystanie w praktyce. Znalazły one swoje zastosowanie w redukcji hałasów akustycznych, rozchodzących się w powietrzu w zamkniętej lub otwartej przestrzeni, redukcji hałasów hydroakustycznych, rozchodzących się w płynach oraz redukcji hałasów wibracyjnych, czyli aktywnym tłumieniu drgań. Jak to zazwyczaj bywa z nowymi technologiami, pierwsze praktyczne realizacje ANC zaobserwowano w militariach, między innymi w tłumieniu hałasu akustycznego propagującego się wewnątrz odrzutowca.

Jak opisuje prawo Moore'a [21], optymalna liczba tranzystorów w układach scalonych w kolejnych latach zwiększa się z zgodnie z trendem wykładniczym i takie zachowanie można zaobserwować do czasów bliskim teraźniejszych (aktualnie prawo nie jest już ściśle zachowane). Ekstremalna miniaturyzacja, optymalizacja zapotrzebowania energetycznego jak i niskie koszta układów cyfrowych spowodowały ich globalizację. Obecnie ANC można zaobserwować na rynku konsumenckim, w postaci słuchawek redukujących hałas z otoczenia, wytłumiaczy otwartych pomieszczeń biurowych czy systemów tworzących przestrzenną strefę ciszy w środku pojazdu. Na tego typu systemy rośnie popyt, spowodo-

wany między innymi wzrostem populacji oraz industrializacji, zwiększajcej ogólny hałas wokół nas.

1.2 Uogólniona koncepcja

Hałas w systemach ANC może rozchodzić się w różnych ośrodkach i w zależności od nich definiowany jest obszar zastosowania, z których wyróżnić można systemy akustyczne, hydroakustyczne lub wibracyjne [11]. Od tego momentu, autor używając zwrotów ANC i hałas, będzie miał na myśli jednokanałowe systemy aktywnej redukcji hałasu w ujęciu akustycznym i wokół tej tematyki skupia się dalsza część pracy.

1.2.1 Obszar zastosowania

Tradycyjne podejście do redukcji hałasu można uogólnić do wykorzystania pasywnych wytłumiaczy, takich jak szczelne bariery odgradzające, tłumiki czy izolacje akustyczne. Takie rozwiązania, pomimo że cenione za wysokie tłumienie w szerokim paśmie częstotliwości, w większości przypadków są drogie oraz wymagają instalacji dużych rozmiarów, mocno przy tym ingerując w środowisko pracy [11].

Największą wadą pasywnych rozwiązań jest jednak kwestia tłumienia hałasów o niskich częstotliwościach. Z racji, że ośrodek, w którym propaguje się fala dźwiękowa wraz z przeszkodami, które napotyka na swojej drodze, zachowują się jak filtry dolnoprzepustowe dla sygnałów akustycznych, to trudno zrealizować ściśle mechaniczny system zdolny wytłumiać hałasy o największej energii w pasmach niskich. Długość fali jest odwrotnie proporcjonalna do jej częstotliwości [7]

$$\lambda = \frac{v}{f}, \tag{1.1}$$

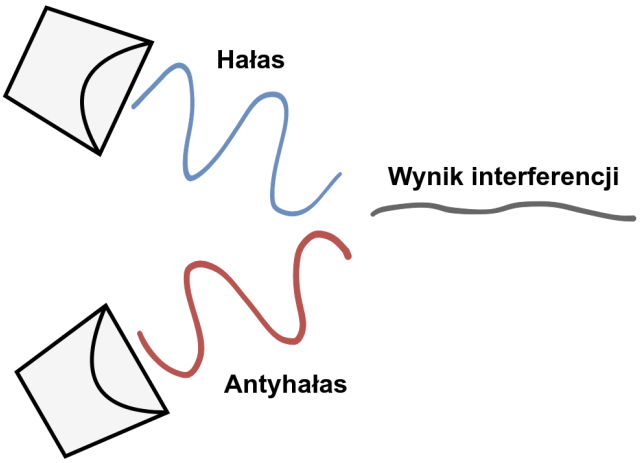
więc wolnozmienna fala akustyczna propagująca w powietrzu ($v \approx 343\ \frac{\mathrm{m}}{\mathrm{s}}$) o przykładowej częstotliwości 343 Hz osiągnie długość jednego metra. Zadanie tłumienia fal długich znacząco zwiększa koszta i złożoność realizacji pasywnego systemu tłumienia, ponieważ takie fale wymagają rozległych, grubych i wykonanych ze specjalnych materiałów, mocno absorbujących niechciany hałas, barier akustycznych.

Zważając na powyższe, systemy ANC wykorzystywane są głównie w celu tłumienia hałasów niskoczęstotliwościowych, tam, gdzie rozwiązania pasywne są najmniej efektywne. Typowe pasmo zainteresowań zamyka się w przedziale **od 50 Hz do 500 Hz** [11]. Niskie częstotliwości powodują, że wymagania czasowe i koszta nałożone na system ANC są z reguły mniejsze, próbkowanie może odbywać się rzadziej, co skutkuje dłuższym czasem na wykonanie obliczeń i wypracowaniem sterowania. Typowo, jeżeli hałas jest tonalny, ANC jest zdolne do osiągania tłumienia na poziomie 30 dB, gdzie w przypadku hałasów sze-

rokopasmowych lub losowych, tłumienie jest zazwyczaj gorsze o połowę. Zakres poziomu ciśnienia akustycznego (ang. *Sound pressure level*, **SPL**) jest dobierany na podstawie charakterystyki źródła i zazwyczaj, ze względu na parametry typowego hałasu i stosowanych urządzeń, zamyka się w przedziale **od 80 dB SPL do 100 dB SPL**.

1.2.2 Idea przewodnia

Fizyczna idea, na której swoje działanie opierają systemy ANC, jest bardzo prosta w zrozumieniu, jednak problematyczna w implementacji. System ma za zadanie wygenerować sygnał o identycznej amplitudzie oraz częstotliwości co niechciany hałas, jednak odwrócony w fazie. Tak wygenerowany sygnał, nazywany antyhałasem, zostaje wprowadzony przez aktuator (głośnik) w tor, którym propaguje się hałas. W wyniku interferencji fal (superpozycji), sygnał zostaje wyciszony. Wizualizację fizycznej idei ANC przedstawiono na rysunku 1.1.



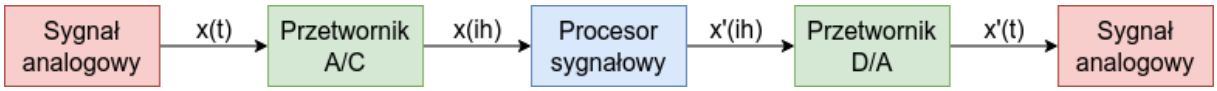
Rysunek 1.1: Wizualizacja fizycznej idei aktywnej redukcji hałasu.

Tor, którym porusza się hałas w omawianej tematyce jest w większości przypadków zmienny w czasie, co powoduje, że częstotliwość, amplituda, faza jak i prędkość rozchodzenia się drgań w ośrodku jest mocno niestacjonarna. Jest to spowodowane aktualną temperaturą, wilgotnością powietrza, ciśnieniem jak i rozmieszczeniem oraz typem elementów wokół, absorobujących lub/i odbijających fale akustyczne w określony sposób. Takie zachowanie często wymaga, żeby projektowany system był adaptacyjny co oznacza, że jego działanie będzie odpowiednio korygowane, w zależności od zaobserwowanych zmian charaketerystyki toru, którym propaguje się niechciany sygnał. Adaptacja realizowana jedynie w sposób analogowy, pomimo że możliwa, posiada duże ograniczenia w zakresie elastyczności i precyzyjności [9], dlatego pożądane jest, aby w systemach tego typu wykorzystać układy cyfrowe, próbkujące sygnały z czujników oraz przetwarzające je z wykorzystaniem odpowiednich algorytmów przetwarzania. Układy cyfrowe przeznaczonego do tego celu muszą być odpowiednio szybkie i precyzjne, aby wszystkie potrzebne

obliczenia wykonać w czasie rzeczywistym. W celu uzyskania wysokiego stopnia tłumienia, niezbędne jest, aby amplituda hałasu i antyhałasu była do siebie zbliżona z dużą precyzją, a faza dokładnie odwrócona. W przeciwnym wypadku wysoce prawdopodobne jest, że system będzie tylko pogarszał sprawę, zwiększając natężenie sygnału, który miał tłumić.

1.3 Określenie problematyki

Układami cyfrowymi często wykorzystywanymi w systemach ANC, ze względu na wymóg przetwarzania sygnałów w czasie rzeczywistym z wysoką precyzją, są specjalnie zaprojektowane do tych celów procesory sygnałowe (DSP, ang. *Digital signal processor*) [6] lub odpowiednio wydajne mikrokontrolery. Takie układy w współpracy z przetwornikami analogowo-cyfrowymi i cyfrowo-analogowymi tworzą klasyczny tor przetwarzania, który przedstawiono na rysunku 1.2:



Rysunek 1.2: Typowy tor przetwarzania.

Procesory sygnałowe charakteryzują się budową, która umożliwia sprzętowe wykonanie operacji często występujących w cyfrowym przetwarzaniu sygnałów. Przykładowo, na tego typu jednostkach istnieje możliwość wykonania operacji mnożenia i odczytu z dwóch niezależnych pamięci RAM w jednym cyklu zegarowym, bufory cykliczne mogą być zaimplementowane na poziomie sprzętowym wraz ze wsparciem dla operacji często wykorzystywanych w liczeniu szybkiej transformaty Fouriera (odwracanie bitów, ang. *bit reversal*). Dodatkowo, instrukcje na tego typu jednostkach obliczeniowych mogą być realizowane są w sposób potokowy (ang. *pipelining*), co pozwala na jednoczesne wykonywanie kilku rozkazów maszynowych (superskalarność)[*superskalarność i pipelining to dwie różne rzeczy. Chodziło mi o to, że równoległość = superskalarność, a nie pipelining.*], znacząco przyśpieszając obliczenia.

Pomimo, że wykorzystanie procesorów sygnałowych lub odpowiednio wydajnych mikrokontrolerów jest z pewnością jednym z najlepszych wyborów przy projektowaniu systemu przetwarzającego sygnały w czasie rzeczywistym, to z takim rozwiązaniem wiąże się szereg problemów:[*tego to teraz nie rozumiem, to na czym można by realizować układy ANC, i przecież realizuje Pan go na odpowiednio wydajnym mikrokontrolerze, bo AM3358 to generalnie mikrokontroler. I ogólnie wiele tych punktów może być dość dyskusyjnych. Tzn. były pewne procesory DSP, gdzie tworzono je z takim podejściem, że chodzi im tylko o DSP, nie projektują tam procesora, by mógł wykonywać różne zadania, by zwiększyć wydajność robią bardzo brzydką architekturę, co później owocowało bym, że były rzadko stosowane, wymagały nietypowych kompilatorów, ... I potem np. Texas Instruments zaczął robić OMAP, które zawierały ich DSP C6000 oraz ARMa dla normalnego kodu, bo ich C6000 to jest tragiczny*

jak chodzi o normalne kody.]

1. **Wysoka cena i mała dostępność** — w większości przypadków zakup musi być realizowany przez globalnego dystrybutora elektroniki. Cena takich układów jest wysoka i z racji niskiej popularności, platformy rozwojowe są drogie i słabo dostępne, jeżeli w ogóle istnieją. Oznacza to, że w większości przypadków, amator lub projektant chcący stworzyć prototyp o niskich kosztach, zmuszony jest na tworzenie dedykowanej elektroniki obsługującej wybrany układ.

2. **Utrudniona programowalność** — z racji dedykowanej budowy procesora, otwartoźródłowe kompilatory, jeżeli w ogóle dostępne, są w najlepszym przypadku wątpliwe. Oznacza to, że w większości przypadków użytkownik takiego układu skazany jest na oprogramowanie producenta, z którym praca jest zazwyczaj nieprzyjemna, ograniczająca i wymaga dodatkowych płatności. Bardzo często się zdarza, że programista koniec końców i tak zmuszony jest do pisania programu w języku asemblera.

3. **Utrudniona przenoszalność** — program napisany pod określony procesor sygnałowy można zazwyczaj uruchomić tylko na docelowym układzie lub jego emulatorze. Portowanie programu na inne urządzenie jest często zadaniem karkołomnym i wymaga od programisty praktycznie przepisanie całego rozwiązania. Często słyszy się również o przypadkach, kiedy program napisany na określony model procesora musi zostać załadowany na nowszy model tego samego producenta, z powodu zakończenia produkcji starszego. W trakcie wykonywania zadania okazuje się, że nowszy model posiada istotne zmiany w budowie i funkcjonalności, które uniemożliwiają załadowanie starego programu bez zmian.

1.3.1 Cel projektu

Wymagania stawiane sprzętowi i oprogramowaniu w systemach ANC są surowe i ograniczające. W raz zaimplementowanym rozwiązaniu zazwyczaj trudno wprowadzać modyfikacje, a środowisko programistyczne w którym się pracuje jest trudne, zamknięte i nieskalowalne. z powodu tych ograniczeń wysunięty został cel projektu, polegający na skonstruowaniu układu realizującego aktywne tłumienie hałasu z wykorzystaniem ogólnodostępnych i otwartoźródłowych rozwiązań, takich jak popularna i tania platforma obliczeniowa z załadowanym systemem operacyjnym wyposażonym w jądro Linux, udostępniającym zestaw bibliotek pozwalających na komunikacją z peryferiami po określonych protokołach cyfrowych. Jądro systemu Linux, jako że należy do wolnego oprogramowania, jest w pełni modyfikowalne i rozszerzalne, co autor, będąc świadomy wymagań czasowych systemu ANC, wykorzysta poprzez dodanie rozszerzeń czasu rzeczywistego (ang. *real-time extensions*), zapewniających lepszą kontrolę nad czasem i realizacją procesów. Ich wpływ na

efektywność działania układu, który zweryfikowany zostanie w końcowym etapie pracy, jest jednym z bardziej znaczących aspektów badawczych projektu.

Oprogramowanie realizujące algorytmy ANC zostanie napisane w języku C, natywnym dla jądra Linux, a sprzęt, poza wybraną, gotową platformą obliczeniową i urządzeniami zewnętrznymi (mikrofony, głośniki), zrealizowany samodzielnie, głównie ze względów edukacyjnych. Taka decyzja jednak w żaden sposób nie ogranicza elastyczności rozwiązania. Do komunikacji z samodzielnie sporządzoną elektroniką wykorzystanie zostany, obsługiwany przez wybraną platformę obliczeniową i system operacyjny, standardowy protokół cyfrowy, za pomocą którego można komunikować się innym, kompatybilnymi peryferiami.

1.4 Zwięzła charakterystyka rozdziałów

Poza słowami wstępu zawartymi w aktualnym rozdziale, zakres pozostałej treści pracy można opisać następująco:

- Rozdział 2, Analiza tematu — przedstawienie typów i struktur systemów ANC, szczegółowe opisanie teoretycznych zasad ich działania wraz z wyprowadzeniem schematu blokowego systemu implementowanego w dalszej części pracy.
- Rozdział 3, Założenia i proponowane rozwiązanie — opisanie założeń funkcjonalnych stawianych projektowanemu układowi z podzieleniem na część sprzętową i programową oraz przedstawienie proponowanego rozwiązania z opisem na ich podstawie.
- Rozdział 4, Implementacja części sprzętowej — szczegółowy opis implementacji zaprojektowanej i sporządzonej elektroniki z podziałem na określone sekcje.
- Rozdział 5, Implementacji części programowej — szczegółowy opis implementacji oprogramowania z podzieleniem na system operacyjny i aplikację działającą w trybie użytkownika.
- Rozdział 6, Testy i badania — zweryfikowanie poprawności sporządzonego rozwiązania poprzez pomiary zasilania, analizę przebiegów czasowych określonych sygnałów analogowych oraz cyfrowych. Przeprowadzenie badań z użyciem pełnego systemu na rzeczywistych lub symulowanych sygnałach, ocena wydajności oraz ocena wpływu wybranych parametrów i konfiguracji na wynikowe tłumienie wraz z komentarzem.
- Roadział 7, Podsumowanie — komentarz zamykający treść pracy. Finalna ocena wykonanego systemu, obiektywna ocena jego działania oraz pomysły na dalsze prace.

Rozdział 2

Analiza tematu

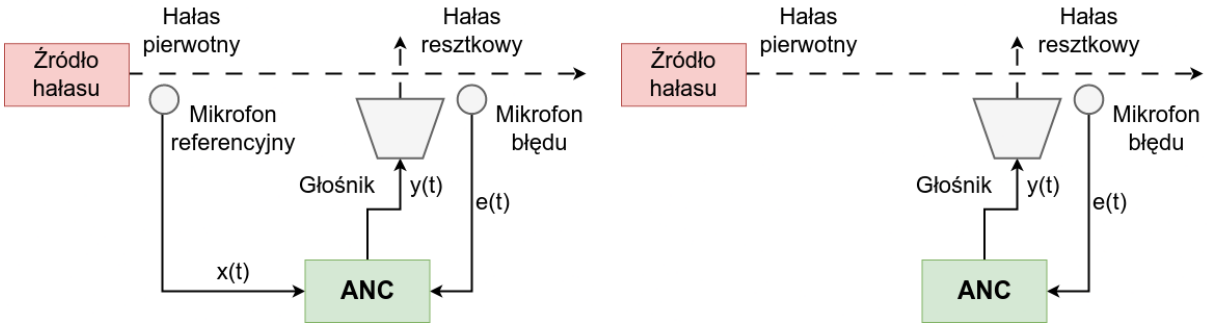
2.1 Struktury systemów

Podział systemów ANC definiowany jest głównie ze względu rodzaj wykorzystywanego sterowania, z którego można wyróżnić [11]:

1. **Sterowanie ze sprzężeniem w przód** — hałas jest mierzony przed dotarciem do mikrofonu błędu z możliwie dużym wyprzedzeniem, tworząc w ten sposób strukturę kompensacji zakłóceń.

2. **Sterowanie ze sprzężeniem w tył** — hałas jest mierzony mikrofonem błędu i stanowi jedyny pomiar przeprowadzany w systemie.

Hałas występujący przed mikrofonem błędu przyjęło nazywać się hałasem pierwotnym i w nomenklaturze automatyki traktowany jako sygnał referencyjny, gdzie ten mierzony przez mikrofon błędu, nazywany hałasem resztkowym, traktowany jest jak błąd (uchyb) W związku z przyjęciem takiego nazewnictwa, można napisać, że systemy ANC ze sprzężeniem w przód opierają swoje działanie na pomiarze referencji i błędu, a system ze sprzężeniem w tył tylko na pomiarze błędu. Omawiane struktury w postaci schematów ideowych przedstawiono na rysunkach 2.1.



(a) ANC ze sprzężeniem w przód.

(b) ANC ze sprzężeniem w tył.

Rysunek 2.1: Porównanie struktur systemów.

Instynktownie można stwierdzić, że skoro systemy ze sprzężeniem w przód opierają swoje działanie na pomiarze dwóch wielkości, to ze względu na większą ilość informacji, sterowanie może zostać wypracowane z większą dokładnością. Jest to stwierdzenie poprawne, jednak należy pamiętać, że takie układy wymagają dodatkowego pomiaru (czujnika), co zwiększa koszta oraz komplikuje strukturę. Dodatkowy pomiar jest też nie zawsze możliwy, więc wybór struktury często jest kompromisem.

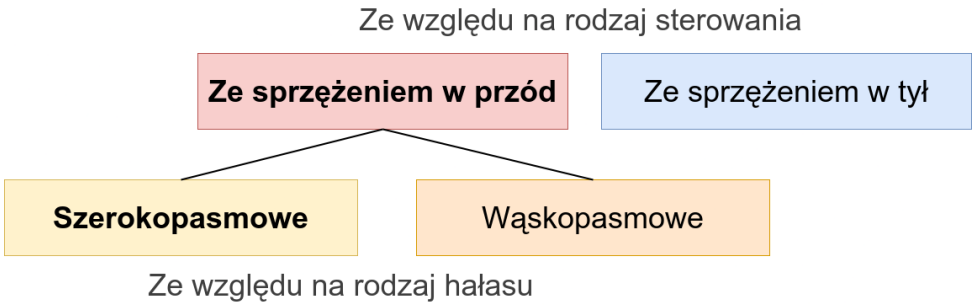
Projektując układ ze sprzężeniem w przód można dokonać dodatkowego założenia, które wprowadza możliwe modyfikacje struktury. Opiera się one charakterystyce hałasu pierwotnego, na podstawie której można zdefiniować:

1. **Hałas szerokopasmowy** — losowy, wywołany przez niespodziewane zaburzenia ośrodka. Jako, że jest losowy, jego energia jest dość równo rozprowadzona w całym paśmie częstotliwości.

2. **Hałas wąskopasmowy** — skoncentrowany w specifycznych pasmach. Może być wywołany przez maszyny pracujące w sposób okresowy, przykładowo silniki lub obracające się wały. W związku, że praca takich elementów jest periodyczna, to hałas, który dystrybuują, również.

W każdym przypadku wymagane jest, żeby element pomiarowy mierzący hałas na wejściu systemu był zdolny do zapewnienia odpowiedniej ilość informacji do wytworzenia adekwatnego sterowania. W przypadku hałasów szerokopasmowych wybór pada na mikrofon referencyjny, w przypadku hałasów wąskopasmowych, wybór czujnika można zaoptymalizować na rzecz nieakustycznego, takiego jak tachometr lub enkoder, kiedy mamy do czynienia z hałasami wygenerowanymi przez obracający się mechanizm. Z wykorzystaniem mikrofonu jako czujnika mierzącego hałas pierwotny często napotyka się na problem, że generowany antyhałas wpływa na pomiar hałasu pierwotnego, tworząc w układzie dodatkową pętlę sprzeżenia zwrotnego. Problem ten można łagodzić poprzez odpowiednie ustawienia kierunków mikrofonów i głośników, tak, aby jak najmniej na siebie wpływały, lub, co jest typowym rozwiązaniem, poprzez rozszerzenie systemu o dodatkową pętlę i neutralizacja jej wpływu w sposób programowy lub sprzętowy.

Klasyfikację struktur systemów ANC można przedstawić jak na rysunku 2.2:

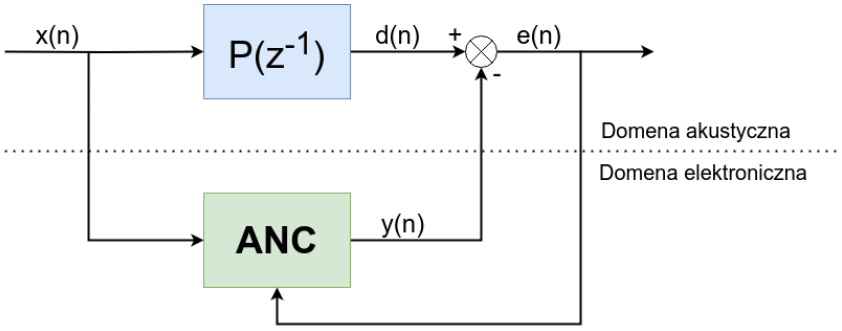


Rysunek 2.2: Podział struktur systemów ANC.

Każda z przedstawionych struktur różni się od siebie znacząco na poziomie sprzętowym jak i programowym. Grubszą czcionką na diagramie 2.2 zaznaczono strukturę, którą zdecydowano się wybrać do dalszej analizy z myślą o finalnej implementacji i jest to **szerokopasmowe ANC ze sprzężeniem w przód**, głównie ze względu na źródło i charakterystykę hałasu, która w tym przypadku może być łatwo symulowana[*tej łatwości symulacji to nie rozumiem.*].

2.2 Zasada działania

Diagram 2.1a można przedstawić za pomocą uogólnionego schematu blokowego 2.3. W celu łatwiejszej analizy, wszystkie sygnały i układy w systemie potraktowano jako dyskretne, jednak należy mieć na uwadze, że postaci sygnałów są zmieniane na ich napięciowe, akustyczne lub cyfrowe odpowiedniki w trakcie przekraczania domen i wchodzenia w blok **ANC**.



Rysunek 2.3: Wyprowadzenie schematu systemu ANC, 0.[*te rysunki dobrze by było jakoś lepiej ponazywać, a nie, że 0, 1, 2, ... I jeszcze na końcu są rysunki o tej samej nazwie. A bez sensownych nazw to spis rysunków traci sens.*]

Transmitancja $P(z^{-1})$ opisuje dynamikę toru, którym porusza się hałas pierwotny i przyjęło się go nazywać torem pierwotnym. Blok **ANC** na ten moment jest uogólnionym układem elektronicznym, a sygnały występujące na schemacie można opisać następująco:

- $x(n)$ — hałas pierwotny trafiający na wejście systemu, mierzony przez mikrofon referencyjny.
- $d(n)$ — hałas pierwotny przefiltrowany przez transmitancję toru podstawowego.
- $y(n)$ — sterowanie głośnika, antyhałas mający wynikowo wytłumić sygnał $d(n)$.
- $e(n)$ — hałas resztkowy, mierzony przez mikrofon błędu.

2.2.1 Identyfikacja toru pierwotnego

Modelowanie obiektów dynamicznych w treści pracy uogólnione zostaje do filtrów SOI (filtr o skończonej odpowiedzi impulsowej, ang. *Finite impulse filter, FIR filter*). Tego

typu modele bardzo często stosowane są w zagadanieniach związanych z systemami ANC, głównie ze względu na wygodę. Są zawsze stabilne, łatwe w implementacji jak i posiadają liniową odpowiedź fazową, w przypadku symetrii współczynników.[*tylko, że obiekty akustyczne nie będą miały symetrycznych współczynników, więc trudno mówić, że to jest jakiś plus modeli FIR.*]

Transmitancja takiego modelu (odpowiedź impulsowa) dana jest zależnością [18]:

$$K(z^{-1}) = \frac{Y(z^{-1})}{X(z^{-1})} = \sum_{l=0}^{L} w_l z^{-l}, \tag{2.1}$$

gdzie w_l to współczynnik filtra o numerze l (ang. *tap*), L to długość odpowiedzi impulsowej, a z^{-l} to operator opóźnienia o l próbek.

W związku z 2.1, wyjście filtra SOI dane jest:

$$y(n) = \sum_{l=0}^{L-1} w_l x(n-l) \implies y(n) = \mathbf{w}^T \mathbf{x}(n), \tag{2.2}$$

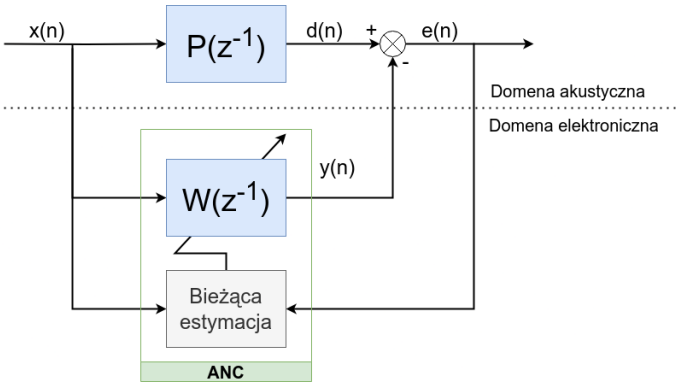
gdzie $\mathbf{w}$ to wektor współczynników filtra:

$$\mathbf{w} \equiv \begin{bmatrix} w_0 & w_1 & \cdots & w_{L-1} \end{bmatrix}^T, \tag{2.3}$$

a $\mathbf{x}$(n) to wektor wejść w chwili n:

$$\mathbf{x}(n) \equiv \begin{bmatrix} x(n) & x(n-1) & \cdots & x(n-L+1) \end{bmatrix}^T. \tag{2.4}$$

Analizując schemat 2.3 można dojść do logicznego wniosku, że jedyną rolą układu **ANC** jest zidentyfikowanie transmitancji $P(z^{-1})$, przefiltrowanie przez nią sygnału $x(n)$ oraz odwórcenie znaku wyjścia. Nie jest to jednak proste zadanie, ponieważ tak jak opisywano wcześniej, charakterystyka toru pierwotnego, czyli drogi, którą propaguje się docelowo tłumiony hałas, może być niestacjonarna. W celu identyfikacji takiego obiektu niezbędny jest mechanizm pozwalający na bieżącą estymację parametrów, czyli innymi słowy, układ musi się adaptować do zmieniającej się charakterystyki toru podstawowego. Mając to na uwadze, zaktualizowany o filtr $W(z^{-1})$, będący bieżącą estymacją obiektu $P(z^{-1})$, schemat blokowy przedstawiono na rysunku 2.4:



Rysunek 2.4: Wyprowadzenie schematu systemu ANC, 1.

2.2.2 Algorytm LMS

Techniką często wykorzystywaną do bieżącej estymacji parametrów obieków niestacjonarnych jest algorytm LMS (ang. *Least mean squares*) [4]. Jego działanie opiera się na zastosowaniu gradientowej metody najniższego spadku (ang. *method of steepest descent*) do minimalizacji błędu średniokwadratowego.

Błąd w analizowanym przypadku jest równy:

$$e(n) = d(n) - y(n) = d(n) - \mathbf{w}^T(n)\mathbf{x}(n). \tag{2.5}$$

Jeżeli założy się, że wektor $\mathbf{w}(n)$ jest sekwencją deterministyczną, to minimalizując wartość oczekiwaną kwadratu błędu [11]:

$$E[e^2(n)] = E[d^2(n)] - 2\mathbf{p}^T\mathbf{w}(n) + \mathbf{w}^T(n)\mathbf{R}\mathbf{w}(n), \tag{2.6}$$

gdzie $\mathbf{p}$ to odniesiony do wejścia wektor korelacji wzajemnej:

$$\mathbf{p} \equiv E[d(n)\mathbf{x}(n)] = \begin{bmatrix} r_{dx}(0) & r_{dx}(1) & \cdots & r_{dx}(L-1) \end{bmatrix}^T, \tag{2.7}$$

$$r_{dx}(k) \equiv E[d(n)x(n-k)], \tag{2.8}$$

[*kontynuacja akapitu musi być bez wcięcia*]a $\mathbf{R}$ jest macierzą autokorelacji sygnału wejściowego $x(n)$:

$$\mathbf{R} \equiv E[\mathbf{x}(n)\mathbf{x}^T(n)] = \begin{bmatrix} r_{xx}(0) & r_{xx}(1) & \cdots & r_{xx}(L-1) \\ r_{xx}(1) & r_{xx}(0) & \cdots & r_{xx}(L-2) \\ \cdots & \vdots & \ddots & \vdots \\ r_{xx}(L-1) & r_{xx}(L-2) & \cdots & r_{xx}(0) \end{bmatrix}, \tag{2.9}$$

$$r_{xx}(k) \equiv E[x(n)x(n-k)], \tag{2.10}$$

metodą najniższego spadku, otrzymuje się równanie aktualizacji parametrów:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu \mathbf{x}(n) e(n), \tag{2.11}$$

gdzie $\mathbf{w}$(n) to wektor obliczonych współczynników filtra w chwili n:

$$\mathbf{w}(n) \equiv \begin{bmatrix} w(n)_0 & w(n)_1 & \cdots & w(n)_{L-1} \end{bmatrix}^T, \tag{2.12}$$

a $\mathbf{x}$(n) to wektor wejść w chwili n:

$$\mathbf{x}(n) \equiv \begin{bmatrix} x(n) & x(n-1) & \cdots & x(n-L+1) \end{bmatrix}^T, \tag{2.13}$$

Parametr μ przyjęło się nazywać krokiem algorytmu LMS. Jego wartość decyduje o szybkości adaptacji algorytmu i ograniczona jest kryterium stabilności zdefiniowanym:

$$0 < \mu < \frac{2}{\lambda_{max}}, \tag{2.14}$$

gdzie λ_{max} zdefiniowana jest jako maksymalna wartość własna macierzy $\mathbf{R}$ 2.9. Zastosowanie warunku stabilności w formie 2.14 jest niepraktyczne z powodu problematycznych obliczeń λ_{max} dla dużych rzędów filtra $W(z^{-1})$. W związku z tym, w praktycznym kryterium stabilności algorytmu LMS przyjęło się stosować estymację λ_{max} na podstawie śladu macierzy $\mathbf{R}$ [11]:

$$\lambda_{max} \leq \sum_{l=0}^{L-1} \lambda_l = L r_{xx}(0) = L P_x, \tag{2.15}$$

$$P_x \equiv r_{xx}(0) = E[x^2(n)], \tag{2.16}$$

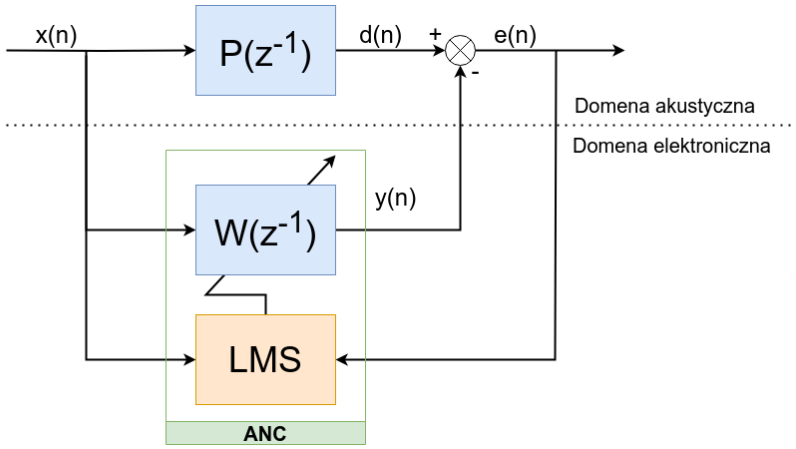
w związku z czym:

$$0 < \mu < \frac{2}{L P_x}, \tag{2.17}$$

i dla sygnałów o rozkładzie normalnym:

$$0 < \mu < \frac{2}{3 L P_x}, \tag{2.18}$$

Posiadając zdefiniowany aparat obliczeniowy bieżącej estymacji parametrów filtra $W(z^{-1})$ schemat można zaktualizować jak na rysunku 2.5:



Rysunek 2.5: Wyprowadzenie schematu systemu ANC, 2.

Bezpośrednio z 2.5 można zapisać:

$$E(z^{-1}) = D(z^{-1}) - Y(z^{-1}) = P(z^{-1})W(z^{-1}) - W(z^{-1})X(z^{-1}). \tag{2.19}$$

Ideowo, jeżeli $W(z^{-1})$ będzie w przybliżeniu zbieżny:

$$E(z^{-1}) = 0 \implies P(z^{-1}) = W(z^{-1}), \tag{2.20}$$

to gdy $X(z^{-1}) \neq 0$:

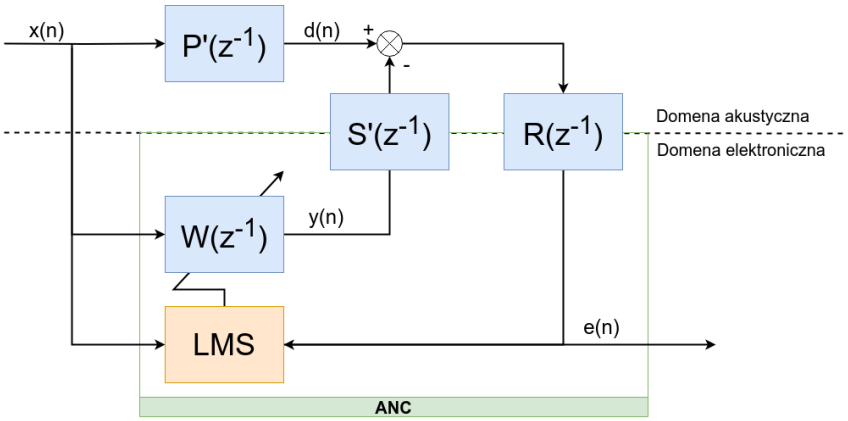
$$y(n) = d(n) \implies e(n) = d(n) - y(n) = 0. \tag{2.21}$$

Zerowy błąd świadczy o idealnym tłumieniu hałasu.

2.2.3 Wpływ toru wtórnego

Podejście realizowane do tej pory można określić jako naiwne, ponieważ nie uwzględniono w nim wpływu dodatkowych obiektów dynamicznych występujących w systemie. Sygnały $x(n)$ i $e(n)$ mierzone są za pomocą mikrofonów i sygnał akustyczny, docierając do nich, musi przebyć pewną drogę. Dodatkowo, w domenie elektronicznej występuje szereg układów przetwarzających, takich jak filtry, wzmacniacze i przetworniki. Finalnie, wygenerowane wyjście układu musi wysterować głośnik i końcowo trafić w tor propagującego się hałasu.

Wszystkie wyszczególnione wyżej elementy posiadają swoją dynamikę i zaniechanie ich opisów w przypadku rzeczywistego układu powoduje gorszą zbieżność realizowanej adaptacji, a co za tym idzie, gorsze tłumienie (lub jego całkowity brak). Schemat po uwzględnieniu dodatkowych obiektów dynamicznych przedstawiono na rysunku 2.6:



Rysunek 2.6: Wyprowadzenie schematu systemu ANC, 3.

Nowe transmitancje na schemacie 2.6 można opisać następująco:

- $S'(z^{-1})$ — reprezentuje dynamikę toru od wyjścia filtru adaptacyjnego $W(z^{-1})$ do węzła sumacyjnego w domenie akustycznej. Z elementów wchodzących w jego skład można wyróżnić przetwornik cyfrowo-analogowy, filtr rekonstrukcjny, wzmacniacz mocy, głośnik i tor od głośnika do punktu interferencji. Jako, że tor składa się z elementów elektronicznych jak i akustycznych, jego obecność na schemacie zaznaczono na pograniczu domen.

- $R(z^{-1})$ — reprezentuje dynamikę toru od węzła sumacyjnego w domenie akustycznej do wejścia algorymu adaptacyjnego. W jego skład wchodzi tor dźwięku od węzła sumacyjnego do mikrofonu błędu, mikrofon błędu, przedwzmacniacz, filtr antyaliasingowy oraz przetwornik analogowo-cyfrowy. Analogicznie jak w przypadku transmitancji $S'(z^{-1})$, tor jest kompozycją elementów akustycznych i elektronicznych.

Po dodaniu dodatkowych obiektów z układu wyłaniają się dwa, znaczące z perspektywy ANC tory przetwarzania. Pierwszym z nich jest dokładniej zdefiniowany tor podstawowy, tym razem określony jako:

$$P(z^{-1}) = P'(z^{-1})R(z^{-1}), \tag{2.22}$$

natomiast drugim jest tak zwany tor wtórny, zdefiniowany jako:

$$S(z^{-1}) = S'(z^{-1})R(z^{-1}). \tag{2.23}$$

Mając na uwadze 2.22 i 2.23, zależność definiująca błąd $E(z^{-1})$ zostaje zmodyfikowana do postaci:

$$E(z^{-1}) = R(z^{-1}) \left[P'(z^{-1}) - S'(z^{-1})W(z^{-1})\right] X(z^{-1}). \tag{2.24}$$

Jeżeli $W(z^{-1})$ jest zbieżne, to w tym przypadku:

$$E(z^{-1}) = 0 \implies W(z^{-1}) = \frac{P'(z^{-1})}{S'(z^{-1})}. \tag{2.25}$$

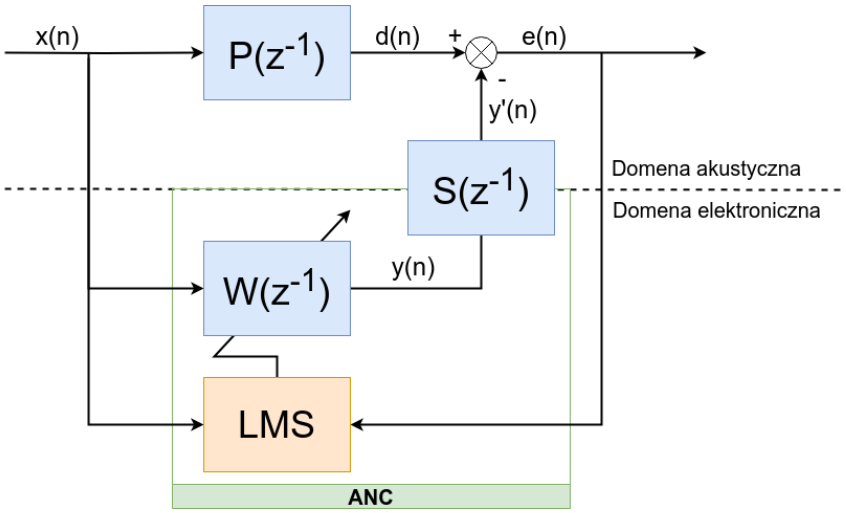
Bezpośrednio z zależności 2.25 można wywnioskować, że zadaniem filtru adaptacyjnego $W(z^{-1})$ w takiej sytuacji jest nie tylko bieżąca estymacja parametrów toru $P(z^{-1})$, ale również bieżąca estymacja parametrów toru $S'(z^{-1})$ wraz z jego odwróceniem. Jest to mocno problematyczne, ponieważ:

1. Niemożliwym jest kompensować opóźnienia $S'(z^{-1})$, jeżeli $P(z^{-1})$ nie zawiera takiego samego lub mniejszego.

2. Jeżeli istnieje częstotliwość, dla której $S'(\omega) = 0$, to filtr staje się niestabilny. Taką częstotliwość nazywa się niesterowalną.

Dodatkową obserwacją jest kwestia, że jeżeli istnieje taka częstotliwość, dla której $R(\omega) = 0$, to system również staje się całkowicie nieefektywny. Taką częstotliwość nazywa się nieobserwowalną, błąd jest wtedy (błędnie) zerowy i nie sposób wypracować odpowiedniego sterowania przy braku wiedzy o jego efektach.

Schemat 2.6 może zostać uproszczony, bazując na informacji, że transmitancje 2.22 i 2.23 mają wspólny składnik w postaci transmitancji $R(z^{-1})$. Tak zmodyfikowany schemat przedstawiono na rysunku 2.7:



Rysunek 2.7: Wyprowadzenie schematu systemu ANC, 4.

Dla schematu 2.7 filtr adaptacyjny $W(z^{-1})$ określony jest zależnością:

$$E(z^{-1}) = \left[P(z^{-1}) - S(z^{-1})W(z^{-1})\right] X(z^{-1}) = 0 \implies W(z^{-1}) = \frac{P(z^{-1})}{S(z^{-1})}. \tag{2.26}$$

Czyli sytuacja, mimo że uproszczona, jest analogiczna jak w 2.25. Wykorzystanie standardowego algorytmu LMS, ze względu na transmitancję $S(z^{-1})$, jest znacząco utrudnione,

dlatego, aby układ ANC działał prawidłowo, wymagane jest wprowadzenie zmian w jego strukturze.

2.2.4 Algorytm FXLMS

Transmitancja toru wtórnego $S(z^{-1})$ wprowadza dodatkową filtrację wyjścia filtru adaptacyjnego $W(z^{-1})$. Błąd w takim przypadku można zdefiniować jako:

$$e(n) = d(n) - y'(n) = d(n) - s(n) * y(n) = d(n) - s(n) * [\mathbf{w}^T(n)\mathbf{x}(n)], \tag{2.27}$$

gdzie $\mathbf{w}(n)$ to wektor współczynników filtra $W(z^{-1})$ w chwili n:

$$\mathbf{w} \equiv \begin{bmatrix} w_0(n) & w_1(n) & \cdots & w_{L-1}(n) \end{bmatrix}^T, \tag{2.28}$$

$\mathbf{x}$(n) to wektor wejść w chwili n:

$$\mathbf{x}(n) \equiv \begin{bmatrix} x(n) & x(n-1) & \cdots & x(n-L+1) \end{bmatrix}^T, \tag{2.29}$$

a $s(n)$ to odpowiedź impulsowa toru wtórnego $S(n^{-1})$ w chwili n. Operator $*$ oznacza konwolucję liniową, czyli filtrację.

Jeżeli dla błędu 2.27 zostanie wyprowadzony algorytm LMS, otrzymane zostanie zmodyfikowane równanie aktualizacji współczynników:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu\mathbf{x'}(n)e(n), \tag{2.30}$$

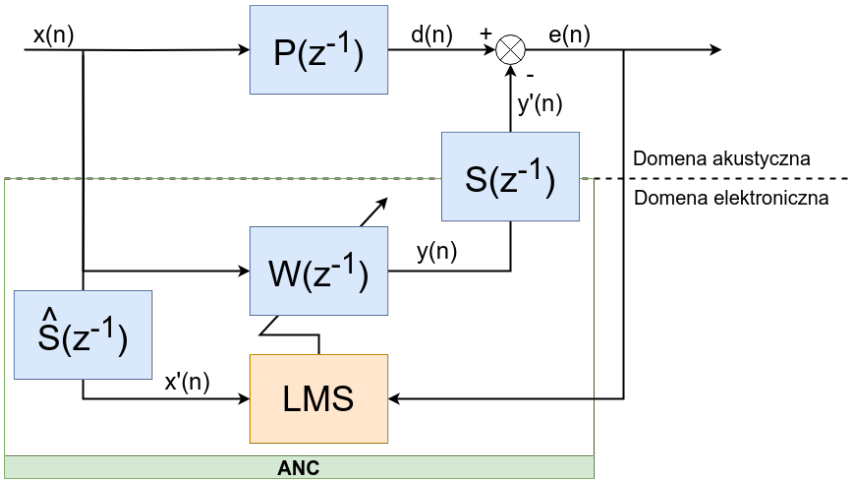
$$\mathbf{x'}(n) = s(n) * \mathbf{x}(n). \tag{2.31}$$

Z równań 2.30 i 2.31 bezpośrednio wynika, że jeżeli za filtrem adaptacyjnym znajduje się dodatkowa transmitancja, to musi ona pojawić się w torze wejściowym algorytmu. Innymi słowy, wektor wejściowy $x(n)$ musi być filtrowany przez transmitancję na wyjściu filtra, skąd bierze się nazwa algorytmu (ang. *FXLMS, Filtered x least mean square*).

Aby wykorzystać 2.30 niezbędna jest wiedza o postaci transmitancji $S(z^{-1})$. Bazując na założeniu, że tor wtórny jest stacjonarny i mniej skomplikowany niż tor podstawowy, estymację jego transmitancji można wyznaczyć korzystając z technik identyfikacji dla obiektów stacjonarnych. Posiadając estymację toru wtórnego $S(z^{-1})$, równanie 2.31 można zmodyfikować do postaci:

$$\mathbf{x'}(n) = \hat{s}(n) * \mathbf{x}(n), \tag{2.32}$$

Schemat systemu z wykorzystaniem algorytmu FXLMS przedstawiono na rysunku 2.8:



Rysunek 2.8: Schemat systemu ANC.

Jeżeli $W(z^{-1})$ zmienia się wolno, to filtry $W(z^{-1})$ i $S(z^{-1})$ można ze sobą połączyć. Wtedy, jeżeli $\hat{S}(z^{-1}) = S(z^{-1})$, to schemat 2.8 można znacząco uprościć, usuwając filtr $S(z^{-1})$ z toru wyjścia filtru $W(z^{-1})$ i podstawiając $S(z^{-1})$ za $\hat{S}(z^{-1})$. Przy takim układzie można stosować metody analizy określone dla tradycyjnego algorytmu LMS, które będą dokładniejsze, im wolniejsza adaptacja (im mniejsze μ).

W praktyce algorytm FXLMS jest stabilny nawet jeżeli współczynniki filtra $W(z^{-1})$ zmieniają się znacznie w odniesieniu do skali czasowej toru wtórnego. Maksymalny krok ograniczony jest przez:

$$\mu_{max} = \frac{1}{P_{x'}(L+\Delta)}, \tag{2.33}$$

gdzie $P_{x'} = E[x'^2(n)]$, a Δ to opóźnienie występujące w torze wtórnym mierzone w ilości próbek. Jak można wnioskować z 2.33, największym ograniczeniem szybkości adaptacji algorytmu jest opóźnienie występujące w torze wtórnym.

2.2.5 Usprawnienie procesu doboru kroku

Wnioskując z równań 2.17 i 2.33, określona przez krok μ stabilność oraz zbieżność algorytmu LMS jest uniwersalnie odwrotnie proporcjonalna do długości filtra L oraz mocy sygnału referencyjnego $P_x(n)$. Jako, że długość filtra L jest stała, istotnym ograniczniem na dobór kroku jest wartość $P_x(n)$, która typowo zmienia się adekwatnie do sygnału referencyjnego $x(n)$. Zmiany mocy $P_x(n)$ w trakcie działania algorytmu mogą spowodować, że początkowo dobrany krok przestaje być poprawny, co skutkuje gorszą zbieżnością lub nawet destabilizacją adaptacji [11].

Aby rozwiązać problem wpływu mocy sygnał wyjściowego na proces uczenia algorytmu, krok μ przy każdej iteracji może zostać znormalizowany w oparciu o estymację mocy sygnału $x(n)$:

$$\mu(n) = \frac{\alpha}{L\hat{P}_x(n)}, \tag{2.34}$$

gdzie α jest nowo zdefiniowanym, znormalizowanym krokiem algorytmu spełnającym założenie

$$0 < \alpha < 2 \tag{2.35}$$

Matematycznie dowieść można, że optymalna wartość kroku α w przypadku braku zakłóceń dana jest jako:

$$\alpha_{opt} = \frac{E\left[|y(n) - \hat{y}(n)|^2\right]}{E\left[|e(n)|^2\right]} = 1, \tag{2.36}$$

W celu wyznaczenia $\hat{P}_x(n)$ można posłużyć się metodą opartą na zostosowaniu prostokątnego okna o długości M:

$$\hat{P}_x(n) = \frac{1}{M}\sum_{m=0}^{M-1} x^2(n-m), \tag{2.37}$$

Równanie 2.37 w przypadku dobrania $M = L$ [*to podejście to chyba wymaga stacjonarności, NLMS jest też dobry dla niestacjonarnych, i tam po prostu przy normalizacji M musi się równać L.*] może zostać sprowadzone do postaci:

$$\hat{P}_x(n) = \frac{1}{L}\sum_{l=0}^{L-1} x^2(n-l) \ = \frac{\mathbf{x}^T(n)\mathbf{x}(n)}{L}, \tag{2.38}$$

Podstawiając 2.38 do 2.34 otrzymuje się:

$$\mu(n) = \frac{\alpha}{\mathbf{x}^T(n)\mathbf{x}(n)}. \tag{2.39}$$

Ostatecznie, podstawiając 2.39 do 2.11 otrzymuje się zmodyfikowane równanie aktualizacji parametrów:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu(n)\mathbf{x}(n)e(n) = \mathbf{w}(n) + \alpha\frac{\mathbf{x}(n)}{\mathbf{x}^T(n)\mathbf{x}(n)}e(n). \tag{2.40}$$

Dzięki tym działaniom, dobór kroku α jest uniezależniony od mocy sygnału $x(n)$. Z powodu zastosowanej normalizacji, algorytm z tak wprowadzoną modyfikacją przyjęło się nazywać znormalizowanym algorytmem NLMS (ang. *Normalized LMS*, NLMS) i jest on również znany jako *Metoda Uczenia Rekurencyjnego* [4].

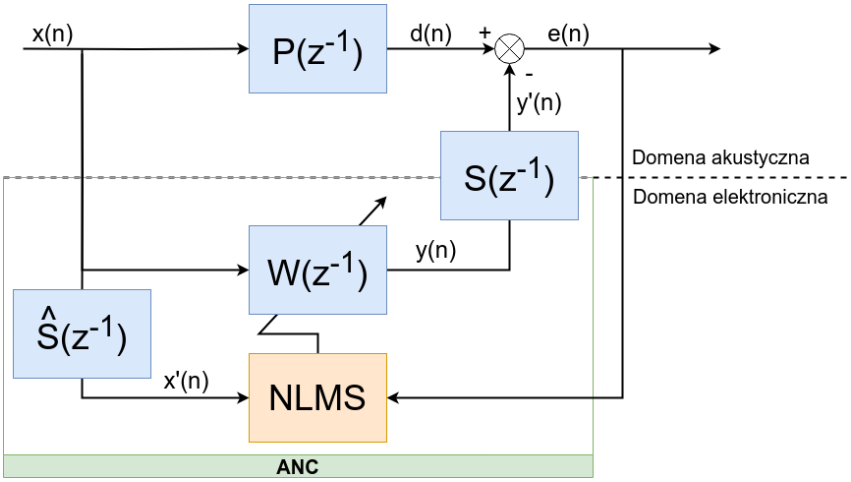
Normalizację kroku, definiującą algorytm NLMS można zastosować w algorytmie FXLMS, otrzymując następujące równiania

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu(n)\mathbf{x'}(n)e(n) = \mathbf{w}(n) + \alpha\frac{\mathbf{x'}(n)}{\mathbf{x'}^T(n)\mathbf{x'}(n)}e(n), \tag{2.41}$$

$$\mathbf{x'}(n) = s(n) * \mathbf{x}(n). \tag{2.42}$$

Tak określony algorytm nie posiada ogólnie przyjętej nazwy, dlatego postępując zgod-

nie z konwencją nazewnictwa modyfikacji algorytmu LMS, autor odwołując się do równań 2.41 i 2.42 będzie używał akronimu FXNLMS (*Filtered X Normalised Least Mean Squares*). Zaktualizowany o normalizację danych wejściowych schemat systemu ANC przedstawiono na rysunku 2.9



Rysunek 2.9: Schemat systemu ANC z normalizacją danych wejściowych.

2.2.6 Metody identyfikacji toru wtórnego

Metod stosowane do identyfikacji toru wtórnego można podzielić na dwie kategorie, które opisano poniżej.

1. **Metody offline** — opierają się na założeniu, że identyfikowany obiekt jest stacjonarny. Identyfikacja odbywa się w kontrolowanym środowisku, na bazie pomiarów uzyskanych w systemie bez oddziaływania w jego strukturę w trakcie pracy. Mogą być wykonywane w systemie w procesie inicjalizującym, lub zewnętrznie, w oparciu o wcześniej zebrane pomiary.

2. **Metody online** — identyfikacja przeprowadzana jest periodycznie w trakcie pracy systemu. Metody z tej kategorii można stosować w przypadku, kiedy obiekt jest niestacjonarny i wymagane jest bezpośrednie oddziaływanie na zmieniającą się charakterystykę.

Jednym z założeń systemu ANC przedstawionego na schemacie 2.8 jest stacjonarność transmitancji toru wtórnego $S(z^{-1})$, więc wykorzystanymi metodami są te z kategorii offline. Trzy z nich opisano poniżej:

1. Identyfikację odpowiedzi impulsowej metoda korelacyjną [4]

$$\hat{s}(n) = \frac{R_{xy}(n)}{\lambda^2}. \tag{2.43}$$

Na wejście identyfikowanego procesu podaje się sygnał identyfikujący o wariancji λ^2[*brakuje to chyba założenia, że ten sygnał identyfikacyjny musi być biały. Natomiast w metodzie korelacyjnej wcale nie musi, tylko wtedy jest do rozwiązania układ równań.*], przez który dzielona jest funkcja korelacji wzajemnej $R_{xy}(n)$ wejścia obiektu $x(n)$ i wyjścia $y(n)$. Uzyskaną w ten sposób odpowiedź impulsową ogarnicza się do długości, której wartość można określić wizualnie na podstawie przebiegu funkcji $\hat{s}(n)$ i jest to typowo moment, w którym osiąga ona małe wartości (wygasza się).

2. Wykorzystanie doświadczalnego estymatora transmitancji [26]:

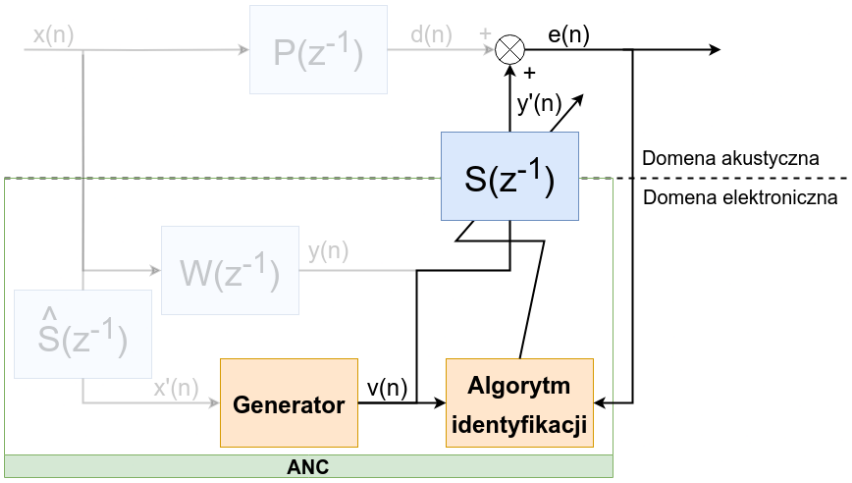
$$\hat{S}(j\omega) = \frac{\hat{Y}(j\omega)}{\hat{X}(j\omega)}. \tag{2.44}$$

Liczone są transformacje Fouriera wejścia $x(n)$ i wyjścia $y(n)$, a następnie przez siebie dzielone. Zależność na $\hat{s}(n)$ otrzymuje się poprzez zastosowanie odwrotnej transformacji na $\hat{S}(j\omega)$.

3. Zastosowanie algorytmu LMS, obserwowanie aktualizowanych współczynników transmitancji i zatrzymanie obliczeń w przypadku, kiedy osiągną one odpowiedni poziom zbieżności.

Do identyfikacji obligatoryjne jest dobranie odpowiedniego sygnału wejściowego. W analizowanym przypadku dobrymi sygnałami są takie, których widmo jest płaskie, czyli ich składowe częstotliwościowe są równomiernie rozłożone w całym paśmie zainteresowań. Przykładami takich sygnałów może szum biały o wariancji λ^2 i sygnał wielosinusoidalny, będący sumą sygnałów sinusoidalnych, z których każdy jest okresowy wewnątrz okna czasowego o wybranej długości [4].

Schemat eksperymentu identyfikacji metodami offline przedstawiono na rysunku 2.10:



Rysunek 2.10: Schemat eksperymentu identyfikacji metodami offline.

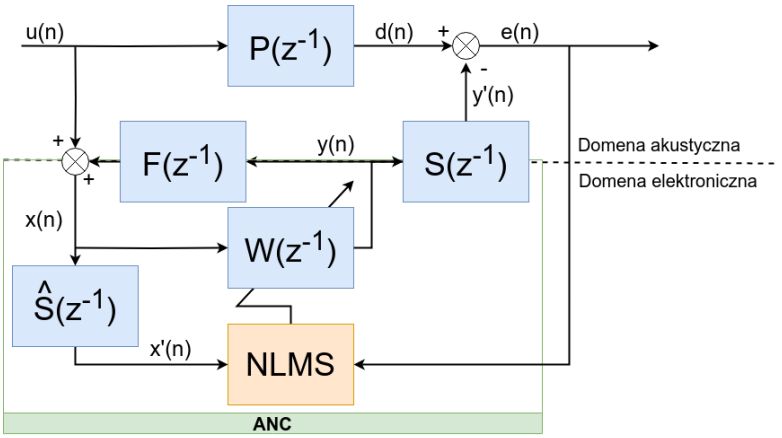
Bezpośrednio ze schematu 2.10 można zapisać:

$$e(n) = y'(n) = v(n) * s(n). \tag{2.45}$$

Identyfikacja toru wtórnego w systemie ANC opiera się więc na wprowadzeniu określonego sygnału identyfikującego $v(n)$ poprzez głośnik emitujący antyhałas i zebranie pomiaru mikrofonem błędu. Sygnał, poprzednio definiowany jako błąd $e(n)$ w tym przypadku jest wynikiem filtracji sygnału $v(n)$ przez transmitancję toru wtórnego $S(z^{-1})$. Przeprowadzając identyfikację zalecane jest, aby przeprowadzić kilka eksperymentów i końcową odpowiedź impulsową $\hat{s}(n)$ uzyskać poprzez uśrednienie rezultatów uzyskanych w poszczególnych eksperymentach. Należy mieć na uwadze, że na charakterystykę torów akustycznych wpływa wiele czynników, które w trakcie pracy systemu mogły ulec zmianie i jeżeli istnieje podejrzenie że tak się stało, to proces identyfikacji należy powtórzyć. W przeciwnym wypadku, adaptacja będzie przebiegała gorzej, a system spełniał swoje zadanie mniej efektywnie. Typowo, proces identyfikacji toru wtórnego realizuje się w procesie inicjalizującym, na początku działania systemu.

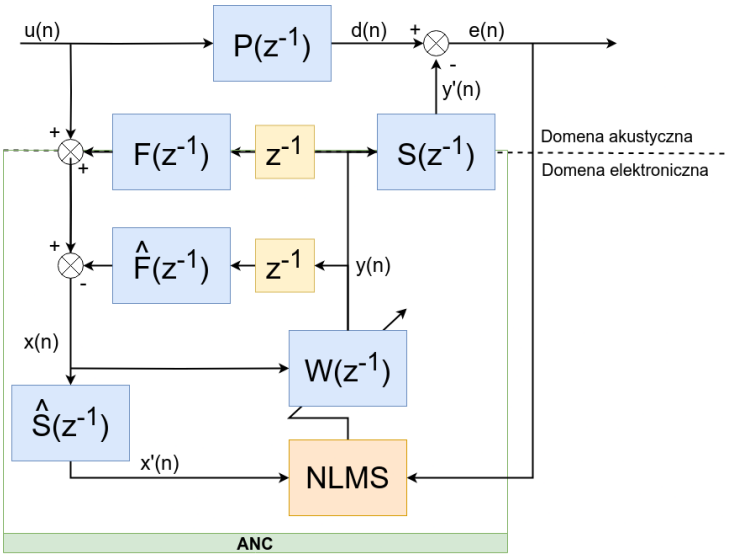
2.2.7 Neutralizacja sprzężenia — możliwe usprawnienie

Jak wspominano wcześniej, częstą przyczyną destabilizacji układów ANC może być wpływ generowanego antyhałasu na mierzony sygnał referencyjny, który interferując z hałasem pierwotnym, zaburza pomiar wejścia. Taka sytuacja powoduje pojawienie się dodatkowej pętli sprzężenia zwrotnego, którą uwzględniono na rysunku 2.11:



Rysunek 2.11: Schemat systemu ANC z uwzględnieniem dodatkowego sprzężenia.

Transmitancja $F(z^{-1})$, podobnie jak $S(z^{-1})$ znajduje się na pograniczu domen sygnałowych, ponieważ składają się na nią układy elektroniczne, jak i droga akustyczna, którą propaguje się antyhałas docierając do mikrofonu referencyjnego. Istnienie $F(z^{-1})$ może spowodować niestacjonarność sygnału $x(n)$ podczas adaptacji filtra $W(z^{-1})$, co z kolei może zdestabilizować system. Jedną z najprostszych metod niwelacji powstałego sprzężenia jest zastosowanie dodatkowego filtra, kompensującego jego wpływ na sygnał $x(n)$. Taki objekt przyjęło się nazywać filtrem neutralizującym [11] i zaktualizowany z jego uwzględnieniem schemat przedstawiono na rysunku 2.12:



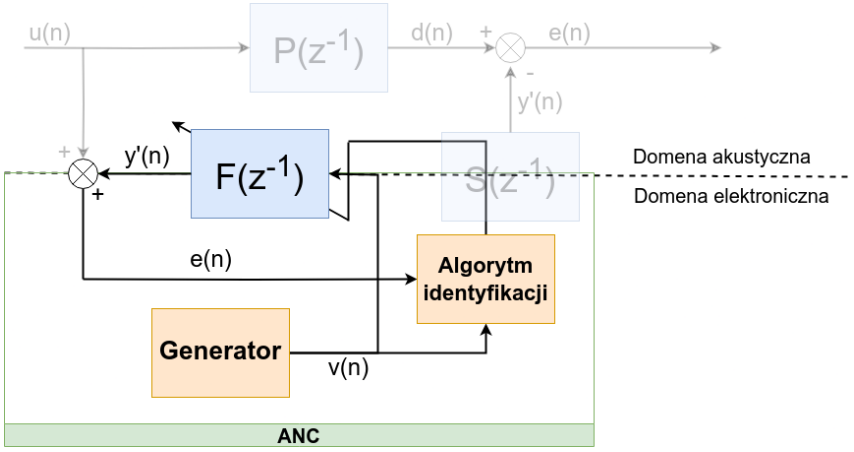
Rysunek 2.12: Schemat systemu ANC z niwelacją sprzężenia.

Transmitancja $\hat{F}(z^{-1})$, analogicznie jak $\hat{S}(z^{-1})$, stanowi zidentyfikowany obiekt $F(z^{-1})$. Jeżeli $F(z^{-1}) = \hat{F}(z^{-1})$, to wpływ sprzężenia pozostaje całkowicie zneutralizowany. Sygnał $x(n)$ wchodzący na wejście filtra $\hat{S}(z^{-1})$ przyjmuje postać:

$$x(n) = u(n) - \sum_{m=0}^{M-1} \hat{f}_m y(n-m-1), \tag{2.46}$$

gdzie $u(n)$ jest nowo oznaczonym sygnałem referencyjnym, M jest długością zidentyfikowanego filtra $\hat{F}(z^{-1})$, a $\hat{f}_m$ jego zidentyfikowaną odpowiedzią impulsową. Należy zauważyć, że w schemacie wprowadzono dodatkowe opóźnienie o jedną próbkę przed blokiem zidentyfikowanego toru sprzężenia. Jest to spowodowane faktem, że w czasie kiedy system zbiera próbkę sygnału referencyjnego zaznaczonego na schemacie jako $u(n)$, to wyjście filtra $W(z^{-1})$ nie jest jeszcze obliczone.

Transmitancję $F(z^{-1})$ można identyfikować w operciu o te same metody co $S(z^{-1})$. Schemat eksperymentu identyfikacji przyjmuje w tym przypadku postać jak na rysunku 2.13:



Rysunek 2.13: Identyfikacja sprzężenia.

Sprzężenie można również niwelować fizycznie, ustawiając głośniki i mikrofony w odpowiednich odstępach i kierunkach, tak, aby jak najmniej na siebie wpływały, co autor wykonał w fizycznej implemetancji projektowanego układu. W późniejszych badaniach nie zauważono znaczących wpływów sprzężenia na stabilność i jakość tłumienia, dlatego programową niwelację sprzężenia potraktowano na tym etapie jako dodatek i możliwe usprawnienie w sytuacji, kiedy projekt będzie rozwijany w przyszłości.

Rozdział 3

Założenia i proponowane rozwiązanie

3.1 Wymagania projektowe

Głównym wymaganiem stawianym całemu projektowi jest wykonanie prostego i wygodnego w obsłudze środowiska, pozwalającego na wykonywanie badań i eksperymentów związanych z tematyką ANC. Poprzez pojęcie środowiska, autor ma na myśli nie tylko platformę sprzętową, ale również strukturę oprogramowania. Sprzęt ma być prosty w obsłudze i możliwe odporny na błędy użytkownika, a struktura oprogramowania powinna udostępniać zestaw narzędzi pozwalających na łatwy proces prototypowania, symulacji, komplilacji i ostatecznie wdrożenia zaimplementowanego rozwiązania.

Jednym z głównych aspektów, który zostanie zweryfikowany w końcowych etapach pracy, jest wpływ wybranych rozszerzeń czasu rzeczywistego jądra Linux na proces aktywnego tłumienia hałasu. Systemy ANC wymagają przetwarzania sygnałów w czasie rzeczywistym i kluczową rolę w efektywności ich działania odgrywa prędkość i precyzja, z którą dane są przetwarzane. Autor ma zamiar sprawdzić, czy takie systemy mają szansę działać pod kontrolą systemu operacyjnego, któremu daleko do systemu czasu rzeczywistego i czy rozszerzenia, o których wspomniano wcześniej, są w stanie pomóc, polepszając efektywność działania procesu. Jeżeli są w stanie pomóc, to jaki jest koszt takiej pomocy i jak wpływa na ogólny proces działania platformy.

Autorowi pracy zależy, żeby wykonany projekt miał jak najszersze horyzonty edukacyjne. W związku z tym, platforma sprzętowa, poza wykorzystaną jednostką obliczeniową, zostanie zaprojektowana i sporządzona samodzielnie w postaci płytki drukowanej. Oprogramowanie napisane na docelową platformę nie będzie korzystało z bibliotek udostępniających gotowe funkcje realizujące algorytmy będącymi znaczącymi elementami pracy. Wykorzystanie zewnętrznych bibliotek zostanie zamknięte na poziomie prototypowania i symulacji. Dodatkowo postanowiono, że wykorzystane narzędzia wspomagające proces projektowania sprzętu jak i implementacji oprogramowania będą dystrybuowane na licencjach otwartego oprogramowania.

3.1.1 Część sprzętowa

Cele projektowe

1. Obsłużenie i przetworzenie wejściowych sygnałów analogowych z mikrofonów pomiarowych pod kowersję do domeny cyfrowej z użyciem osadzonych na płytce przetworników analogowo-cyfrowych. Wykorzystywane będą dwa kanały wejściowe.

2. Obsłużenie i przetworzenie wyjściowych sygnałów analogowych pochodzących z osadzonych na płytce przetworników cyfrowo-analogowych pod poprawne dostarczenie ich na głośniki. Wykorzystywane będą dwa kanały wyjściowe.

3. Zapewnienie działającej komunikacji z wybraną platformą obliczeniową poprzez wybrany protokół cyfrowy, przez który przesyłane będą przekonwertowane z użyciem osadzonych na płytce przetworników analogowo-cyfrowych do domeny cyfrowej wejściowe, pochodzące z mikrofonów, sygnały analogowe, oraz końcowo trafiające na głośniki, dane wyjściowe do przetworników cyfrowo-analogowych.

4. Zapewnienie odpowiednich zasilań dla sekcji analogowych oraz cyfrowych jak i możliwość zasilania wybranej platformy obliczeniowej z poziomu płytki.

W skład projektowanego układu elektronicznego wchodzą cztery sekcje, z których można wyróżnić:

1. Sekcję zasilania, dystrybuujacą energię na cały układ.

2. Wejściową sekcję analogową, obsługującą kanały mikrofonowe.

3. Wyjściową sekcję analogową, obsługującą kanały głośnikowe.

4. Sekcję przetworników, komunikujacą się z sekcjami analogowymi oraz z wybraną platformą obliczeniową.

Poza sekcjami znajdującymi się na projektowanej płytce drukowanej, w skład wykorzystywanego sprzętu wchodzić będą urządzenia zewnętrzne, czyli dwa mikrofony, dwa zestawy głośnikowe oraz wybrana platforma obliczeniowa.

Wymagania sekcji zasilania

1. Generacja wszystkich potrzebnych zasilań na podstawie jednego, dodatniego doprowadzonego z zewnątrz.

2. Generacja symetrycznych zasilań dla wzmacniaczy operacyjnych pracujących w sekcjach analogowych o względnie wysokiej dokładności.

3. W przypadku wyboru mikrofonów pojemnościowych, generacja odpowiedniego zasilania phantomowego, typowo mieszczącego się w zakresie od +12 V do +48 V.

4. Generacja zasilań dla przetworników i zewnętrznej platformy obliczeniowej.

5. Generacja dokładnych napięć referencyjnych dla przetworników (jeżeli będą potrzebne).

Wymagania wejściowych sekcji analogowych

1. Obsłużenie symetrycznych wejść sygnałowych z mikrofonów oraz ich konwersję do odpowiednio wzmocnionej sumy.

2. Filtrację dolnoprzepustową, ograniczającą pasmo sygnału wejściowego do przedziału zainteresowań, mieszczącego się w zakresie od 50 Hz do 500 Hz.

3. Odpowiednie wyskalowanie przetworzonego sygnału, aby mieścił się w pełnym zakresie dynamicznym wybranych przetworników analogowo-cyfrowych.

4. Zabezpieczenie przetworników analogowo-cyfrowych przed uszkodzeniem, poprzez zastosowanie odpowiednich układów ograniczających napięcie wejściowe.

Wymagania wyjściowych sekcji analogowych

1. Obsłużenie wyjść przetworników cyfrowo-analogowych.

2. Odpowiednie przetworzenie sygnału wyjściowego, opierające się na skalowaniu i filtracji rekonstrukcyjnej.

3. Przekształcenie wyjściowego sygnału na symetryczną parę, zdefiniowaną przez standard złącza XLR.

4. Finalne dostarczenie przygotowanego sygnału wyjściowego na wykorzystany zestaw głośnikowy.

Wymagania sekcji przetworników

1. Relatywnie wysoka rozdzielczość, wynosząca minimalnie 12 bitów.

2. Wsparcie dla identycznego protokołu cyfrowego.

3. Jak najmniejsze opóźnienia wynikające z konwersji.

4. Identyczny format danych, pozwalający na globalną konfigurację protokołu według jednej wytycznej.

Wymagania protokołu cyfrowego

1. Synchronizacja wszystkich przetworników poprzez wystawienie wspólnego sygnału zegarowego o konfigurowalnej częstotliwości.
2. Jednoczesna obsługa dwóch kanałów wejściowych (RX) i dwóch kanałów wyjściowych (TX). Oznacza to, że protokół musi wspierać multipleksowanie kanałów, jeżeli wykorzystane zostaną pojedyncze linie danych na odpowiednim kierunku transmisji, lub odpowiednią konfigurację selektorów (CS), ewentualnie selektora (FS), jeżeli zostaną wykorzystane pojedyncze linie danych na kanał.
3. Konfigurowalność. Możliwość zmian częstotliwości zegarów, co implikuje wpływ na częstotliwość próbkowania, jak i sposobu, w jaki dane są czytane/zapisywane, czyli na jakie zbocza zegarów odbywa się zapis/odczyt.

Wymagania mikrofonów

1. Niska cena i wysoka dostępność.
2. Typ — standardem w omawianej tematyce są mikrofony pojemnościowe. Znane są z wysokiej czułości i płaskiej charakterystyki częstotliwościowej, jednak wymagają dodatkowego zasilania, zwanego zasilaniem phantom.
3. Charakterystyka kierunkowa — superkardoidalne lub hiperkardoidalne mogą lepiej skupić się na kierunkowym źródle dźwięku, lecz w przypadku hałasu w otwartej przestrzeni odbierany dźwięk będzie słabszy, jeżeli jego źródło znajdzie się poza docelowym obszarem działania. Tematyka projektu skupia się na hałasie w przestrzeni, a źródła dźwięku ustawione są względem siebie pod pewnym kątem, więc docelową charakterystyką jest charakterystyka kulista.
4. Odpowiedź częstotliwościowa — ideowo płaska. Zakres nie odgrywa znaczącej roli, bo analizowane sygnały są wolnozmienne (typowe zakresy posiadają górną granicę definiowaną w dziesiątkach kiloherców).

Wymagania zestawów głośnikowych

1. Rozsądna cena i dobra dostępność.
2. Z racji, że zrezygnowano z projektowania końcówki mocy, wymagany jest wbudowany wzmacniacz. Innymi słowy, zestawy muszą być aktywne.
3. Analizowane sygnały są wolnozmienne (pasmo 50 do 500 Hz), więc wymagane jest wsparcie dla tonów niskich. Innymi słowy, na wyposażeniu zestawu musi być głośnik niskotonowy (ang. *woofer*), zdolny do emisji sygnałów akustycznych o niskiej częstotliwości.

4. Rozsądny rozmiar, pozwalający na manipulację położeniem i kierunkiem.
5. Z uwagi na zakres poziomu akustycznego analizowanych sygnałów, zestaw musi być zdolny do osiągnięcia jego górnego limitu, czyli użytecznego 100 dB SPL.

Wymagania platformy obliczeniowej

1. Duża dostępność i stosunkowo niska cena.
2. Dostępny kompilator języka C.
3. Możliwość instalacji wybranej dystrybucji systemu operacyjnego GNU/Linux.
4. Wsparcie sprzętowe dla wybranego protokołu cyfrowego.

Poza opisanymi wyżej wymaganiami, porządane jest (lecz nie wymagane), aby wykorzystana platforma posiadała sprzętową realizację działań zmiennoprzecikowych (FPU), wsparcie dla architektury SIMD (ang. *Single Instruction, Multiple Data*) wspomagającą obliczenia wektorowe, port szeregowy oraz złącze Ethernet.

3.1.2 Część programowa

Cele projektowe

1. Pełna kontrola nad wykorzystywanym protokołem cyfrowym. Możliwość jednoczesnego odczytu z dwóch kanałów wejściowych, zapisu do dwóch kanałów wyjściowych, konfiguracji częstotliwości głównego zegara oraz zachowania selektorów/selektora.
2. Pełna kontrola nad sporządzoną elektroniką poprzez wykorzystany protokół cyfrowy.
3. Realizacja filtracji cyfrowej oraz algorytmu FXLMS.
4. Symulacyjna generacja hałasu w postaci typowych sygnałów okresowych (sinus, cosinus, prostokąt, trójkąt, piła i szum) o konfigurowalnych parametrach.
5. Generacja odpowiedniego sygnału identyfikującego (biały szum, PRBS), pozwalającego na identyfikację toru wtórnego.
6. Możliwość rejestracji i zapisu określonych sygnałów do pliku w celu dalszej analizy i wizualizacji.
7. Udostępnienie środowiska pozwalającego no prosty proces kompilacji krzyżowej oraz symulacji prototypowanych rozwiązań.

W skład projektowanego oprogramowania wchodzi aplikacja działająca w trybie użytkownika oraz odpowiednio skonfigurowany system operacyjny.

Wymagania systemu operacyjnego

1. Wsparcie dla wybranego interfejsu cyfrowego w postaci modułu jądra.
2. Możliwość interakcji ze sporządzoną elektorniką poprzez moduł jądra wybranego interfejsu cyfrowego lub osobny sterownik reprezentujący sporządzoną płytkę.
3. Zapewnienie zestawu bibliotek pozwalających na komunikację wybranym interfejsem cyfrowym przez moduł jądra z poziomu aplikacji działającej w trybie użytkownika.
4. Wpływ na opóźnienia wynikające z synchronizacji procesów, buforowania danych oraz czasu przetwarzania.
5. Obecność narzędzi ułatwiających ogólną prace w systemie (edytor tekstowy, obsługa internetu, debugger) oraz pozwalających na określenie wydajności zaimplementowanej aplikacji.

Wymagania aplikacji

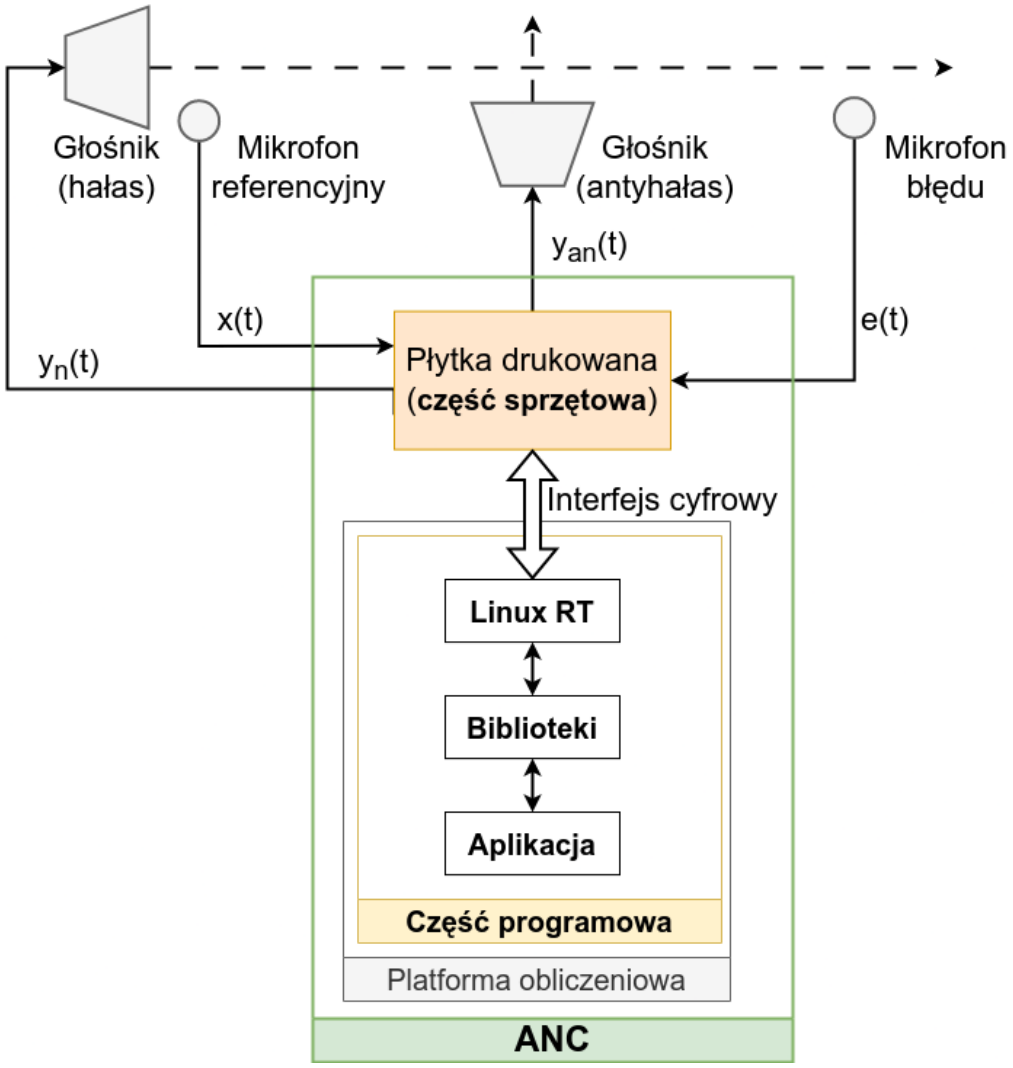
1. Programowa obsługa sporządzonej elektroniki za pośrednictwem bibliotek komunikujących się z odpowiednimi modułami jądra.
2. Możliwość symulacji źródła hałasu poprzez generację odpowiedniego sygnału w trakcie działania pętli ANC.
3. Odpowiednio szybkie przetwarzanie danych, zapobiegające nieciągłościom transferów i destabilizacji pętli ANC.
4. Wykorzystanie rozszerzeń czasu rzeczywistego.
5. Wykorzystanie możliwych realizacji sprzętowych określonych operacji (FPU, SIMD).

3.2 Architektura systemu

Tak jak opisywano w sekcji dotyczącej teoretycznej analizy tematu, wybraną strukturą jest ANC ze sprzężeniem w przód dla hałasów szerokopasmowych. Układ elektroniczny, uogólniony na schemacie 2.3 jak blok **ANC** został podzielony na dwie części, stale komunikujące się ze sobą po określonym interfejsie cyfrowym, zapewniającym synchronizację przetworników oraz przesyłanie i odbieranie danych będących cyfrowymi odpowiednikami sygnałów analogowych występujących w systemie. Sporządzona płytka drukowana, będąca częścią sprzętową projektu, traktowana jest przez platformę obliczeniową jak zewnętrzna karta pomiarowa obsługująca dwa kanały wejściowe i dwa kanały wyjściowe.

Dodatkowy kanał wyjściowy umożliwia generację hałasu, co pozwala na przeprowadzenie testów i badań bez obecności rzeczywistego źródła.

Diagram architektury zaprojektowanego systemu przedstawiono na rysunku 3.1



Rysunek 3.1: Architektura zaprojektowanego systemu

Aplikacja działająca w przestrzeni użytkownika komunikuje się z jądrem poprzez biblioteki systemowe, komunikujące się z odpowiednimi sterownikami wspierającymi obsługę wybranego protokołu cyfrowego oraz bezpośrednie wywołania (ang. *syscall*). Jądro, poprzez wykorzystanie planisty (ang. *scheduler*) zarządza i kontroluje alokację zasobów procesora platformy obliczeniowej potrzebnych do wykonywania procesów, wśród których wyróżniony jest proces zaimplementowanej aplikacji. Dzięki wykorzystanym rozszerzeniom czasu rzeczywistego aplikacja ma bezpośredni wpływ na przydzielone zasoby, głównie priorytet, sposób wywłaszczania i realizacji przerwań. Wpływ aplikacji na sposób zarządzania jądra może zostać wykorzystany do minimalizacji opóźnień występujących w torze cyfrowym, wynikających z zastosowania systemu operacyjnego, nie będącego systemem czasu rzeczywistego.

3.3 Wykorzystane narzędzia

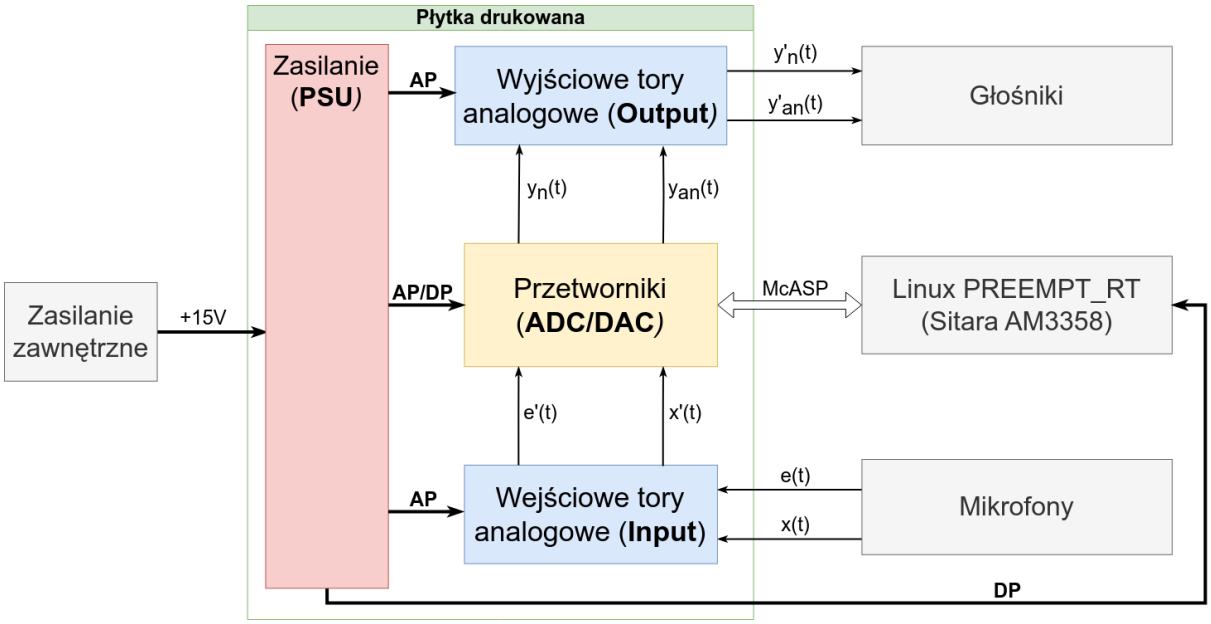
Tak jak opisywano w głównych założeniach projektowych, wykorzystane narzędzia programowe oparte są na licencjach otwartoźródłowych. Spośród nich można wyróżnić:

- KiCAD — wykorzystane EDA (ang. *Electronic Design Automation*).
- Język programowania Python — wykorzystywany do prototypowania, symulacji i wizualizacji danych.
- Język programowania C — wykorzystany do implementacji.
- GCC — jako zestaw kompilatorów.
- CMake i GNU Make — jako systemy budowania.
- Docker — jako narzędzie do wirtualizacji środowiska pracy
- Git — jako system kontroli wersji.

Rozdział 4

Implementacja części sprzętowej

Diagram opisujący zaimplementowaną architekturę sprzętową przedstawiono na rysunku 4.1:



Rysunek 4.1: Architektura sprzętowa - najwyższy poziom

Jak można zauważyć, na architekturę zaprojekowanej płytki drukowanej składają się cztery główne sekcje, które komunikują się z elementami zewnętrznymi zaznaczonymi na szaro. Z sekcji można wyróżnić:

- Sekcję zasilania (**PSU**, ang. *Power supply unit*)
- Sekcję wejściowych torów analogowych (**Input**, wejście)
- Sekcję wyjściowych torów analogowych (**Output**, wyjście)
- Sekcję przetworników (**ADC/DAC**)

Na diagramie konektory o najmniejszej grubości reprezentują sygnały analogowe, z których można wyróżnić:

- $x(t)$ — sygnał z mikrofonu referencyjnego przy źródle hałasu.
- $e(t)$ — sygnał z mikrofonu błędu, mierzącego superpozycję hałasu i antyhałasu.
- $y_n(t)$ — sygnał reperezentujący wygenerowany hałas.
- $y_{an}(t)$ — sygnał reperezentujący obliczony antyhałas.

Każdy z wyszczególnionych wyżej sygnałów analogowych posiada swoje przetworzone odpowiedniki, które na schemacie zaznaczono z apostrofem. Pogrubionymi konektorami zaznaczono szyny zasilania i są to odpowiednio:

- **AP** (ang. *Analog power*) — zasilanie sekcji analogowych.
- **DP** (ang. *Digital power*) — zasilanie sekcji cyfrowych.

Dużą strzałką zaznaczono wykorzystywany protokół cyfrowy, czyli McASP (ang. *Multichannel serial port*), który dokładniej opisywany będzie w sekcji interfejsu cyfrowego.

4.1 Urządzenia zewnętrzne

4.1.1 Mikrofony

W projekcie wykorzystano mikrofony Behringer ECM8000. Są one bardzo często wybierane przez amatorów jak i profesjonalistów w zastosowaniach pomiarowych oraz cieszą się dobrymi opiniami. Ich droższym odpowiednikiem jest flagowy Beyerdynamic MM1, a najbardziej znaczące parametry [19] przedstawiono poniżej:

1. Typ — pojemnościowy, phantom +15 V do +48 V.
2. Charakterystyka kierunkowa — kulista.
3. Odpowiedź częstotliwościowa — relatywnie płaska, z pofalowaniami w paśmie 5 kHz do 10 kHz i zakresie 20 Hz do 20000 Hz.
4. Czułość — 15 $\frac{\text{mV}}{\text{Pa}}$.

Czułość mikrofonów informuje o wartości napięcia dla danego ciśnienia aktustycznego i opisuje ją zależność:

$$S_{\text{dBV}} = 20\log_{10}\frac{S_{\text{mV/Pa}}}{U_{\text{ref}}}, \qquad U_{ref} = 1000\ \frac{\text{mV}}{\text{Pa}}. \tag{4.1}$$

Na podstawie tego parametru dobiera się wzmocnienie przedwzmacniacza [13]. Dla wybranych mikrofonów z czułością na poziomie 15 $\frac{\text{mV}}{\text{Pa}}$, uzyskuje się 15 mV dla 1 Pa, co jest równe 94 dB SPL. Pracując na hałasach w zakresie 85 do 105 dB SPL, dla górnej granicy otrzymuje się około 3.5 Pa, czyli odpowiednio około 52.5 mV. W przypadku przetwornika analogowo-cyfrowego pracującego w przedziale od 0 V do 5 V (2.5 V amplitudy sygnału przy nałożonej składowej stałej 2.5 V), 52.5 mV z mikrofonu powinno odpowiadać maksymalnemu zakresowi 2.5 V[*nie do końca, 52.5 mV to będzie RMS, dla sinusa 1 kHz zwykle się określa, a 2.5 V to musi być maksymalna wartość*], co pozwala na obliczenie wzmocnienia:

$$k = \frac{U_{out}}{U_{mic}} = \frac{2.5}{0.0525} = 47.6190 \approx 42 \text{ dB}, \tag{4.2}$$

4.1.2 Zestaw głośnikowy

Na użytek prywatny, autor posiada parę monitorów studyjnych KRK Classic 8, spełniających wszystkie wyszczególnione dla zestawu głośników wymagania. Nie wchodząc w techniczne szczegóły, z najbardziej znaczących parametrów [10] można wyróżnić:

1. Typ i konfiguracja — aktywny, głośnik wysoko i nisko tonowy (ang. *woofer, tweeter*).
2. Maksymalny zakres poziomu akustycznego — 103 dB SPL, 106 dB SPL w piku.
3. Charakterystyka częstotliwościowa do ±3 dB — 42.5 Hz do 35000 Hz, płaska.
4. SNR — powyżej 100 dB.

4.1.3 Platforma obliczeniowa

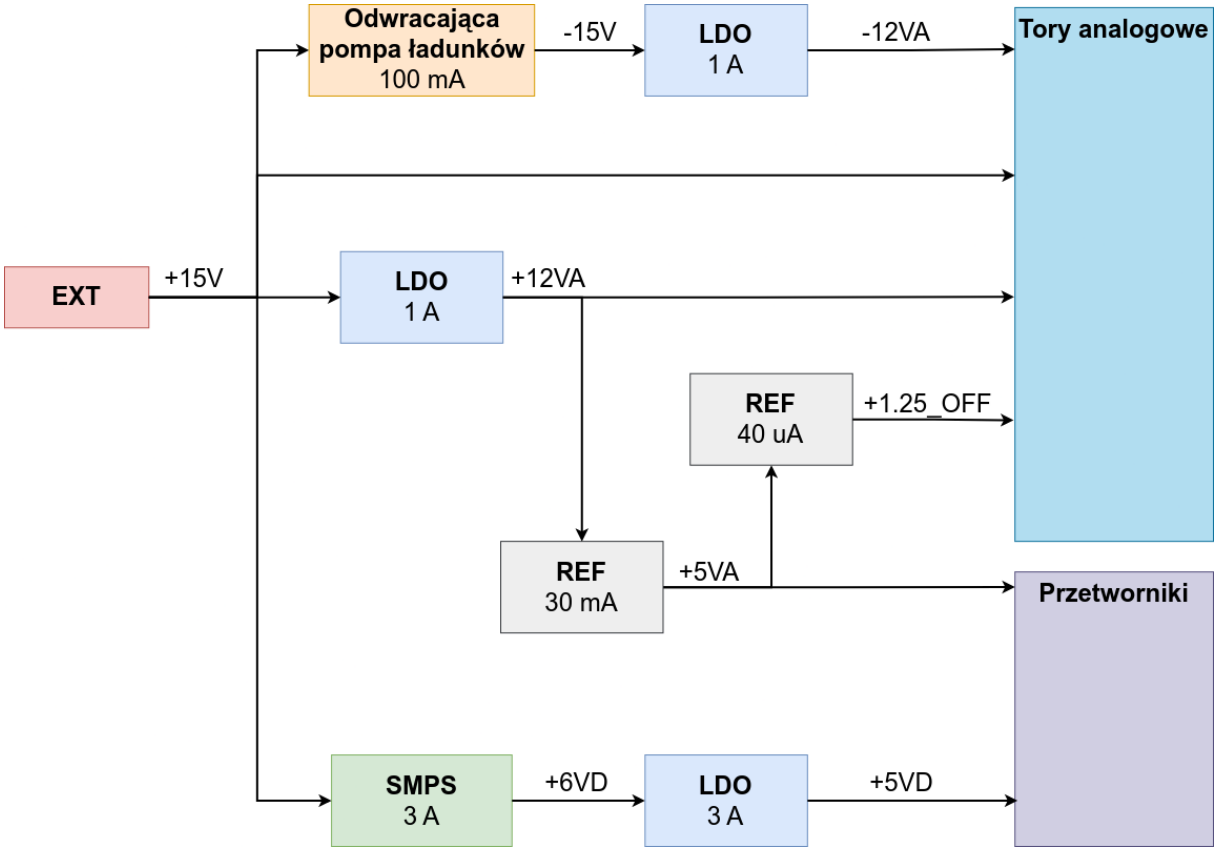
Analizując rynek w poszukiwaniu tanich i popularnych układów spełniających założone wymagania, wyboru można dokonać tak właściwie spośród dwóch platform. Jest to oparte na module BCM2837B0 firmy Broadcom Raspberry Pi [24] oraz oparty na module Sitara AM3358 firmy Texas Instruments BeagleBone® Black [3]. Platformy są w podobnych cenach i wykorzystuje się je w równie podobnych zastosowaniach.

Argumentem przemawiającym za wyborem jednej z nich były wspierane interfejsy cyfrowe, których przeglądu i analizy dokonano na podstawie założeń projektowych. W trakcie tego przeglądu, przeanalizowana została nota aplikacyjna [25], opisująca akwizycję danych z macierzy mikrofonowej w oparciu o interfejs McASP (ang. *Multi-channel audio serial port*) z wykorzystaniem procesora sygnałowego oraz układu BeagleBone® Black. Opisywany interfejs jest dedykowanym mechanizmem komunikacji w wielokanałowych systemach audio realizowanych w układach firmy Texas Instruments i nie posiada swojego odpowiednika na platformach Raspberry Pi. Z tego powodu wybór okazał się prosty i jako wykorzystaną platformą obliczeniową wykorzystano płytkę BeagleBone® Black.

Z kwestii dodatkowych, moduł Sitara AM3358 posiada sprzętową realizację działań zmiennoprzecinkowych (FPU) oraz koprocesor NEON™ (SIMD).

4.2 Zasilanie

Diagram dystrybucji zasilania przedstawiono na rysunku 4.2:



Rysunek 4.2: Diagram dystrybucji zasilania

Wykorzystane symbole zostały wyjaśnione poniżej:

- **EXT** (ang. *External*) — zasilanie zewnętrzne
- **LDO** (ang. *Low dropout regulator*) — linowy regulator napięcia niskiego spadku
- **REF** (ang. *Reference voltage*) — generator napięcia referencyjnego
- **SMPS** (ang. *Switching-mode power supply*) — zasilacz impulsowy

4.2.1 Sekcja wejściowa

Układ zasilany jest z zewnętrznego zasilacza impulsowego o napięciu wyjściowym 15 V. Zasilacze impulsowe znane są wysokiej, wynikającej z przełączania wewnętrznego klucza tranzystorowego, fluktuacji napięcia wyjściowego (ang. *ripple*), które w przypadku

dobranego zasilacza wynosi maksymalnie 240 mV p-p [33]. Na uwadze należy również mieć zakłócenia pochodzące z sieci, dlatego typowym rozwiązaniem stosowanym w sekcjach wejściowych układów elektronicznych jest zastosowanie dolnoprzepustowego filtru LC [28], mającego za zadanie wygładzenie zewnętrznego zasilania jak i zapewnienie bufora do dalszych konwersji. Taki filtr opisany jest zależnością, wynikającą z uogólnionego równania dzielnika napięciowego:

$$U_{wy}(s) = U_{we}(s)\frac{Z_1}{Z_1 + Z_2} = U_{we}(s)\frac{\frac{1}{Cs}}{Ls + \frac{1}{Cs}} \implies K_{LC}(s) = \frac{1}{LCs^2 + 1}. \tag{4.3}$$

W dziedzinie symbolicznej natomiast ($s = j\omega$):

$$K_{LC}(j\omega) = \frac{1}{1 - \omega^2 LC}. \tag{4.4}$$

Częstotliwość rezonansowa takiego układu wynosi:

$$\omega_c = 2\pi f_c \implies f_c = \frac{1}{2\pi\sqrt{LC}}. \tag{4.5}$$

Zależność 4.6 przedstawia idealny model układu LC. W rzeczywistości każdy element pasywny posiada swoją rezystancję, przez co układ LC staje się układem RLC, określonym zależnością:

$$U_{wy}(s) = U_{we}(s)\frac{Z_1}{Z_1 + Z_2} = U_{we}(s)\frac{\frac{1}{Cs}}{R + Ls + \frac{1}{Cs}} \implies K_{LP}(s) = \frac{1}{LCs^2 + RCs + 1}. \tag{4.6}$$

Rezystor w takim układzie wprowadza tłumienie, z czego wynika współczynnik tłumienia oraz dobroć. Wielkości te określone są odpowiednio:

$$\zeta = \frac{1}{2Q} = \frac{1}{2R\sqrt{\frac{C}{L}}}, \tag{4.7}$$

$$Q = \frac{1}{R}\sqrt{\frac{L}{C}}. \tag{4.8}$$

[*czy ten wzór u góry jest na pewno dobry? Bo z niego wynika* $2Q = 2R\sqrt{\frac{C}{L}}$*, co nie zgadza się z tym drugim, a drugi jest na pewno dobry.*]

Współczynnik tłumienia może znajdować się w czterech typach zakresów:

1. $\zeta = 0$ — brak tłumienia, nieskończone oscylacje
2. $\zeta > 0$ i $\zeta < 1$ - tłumienie subkrytyczne, oscylacje są wygaszane po pewnym czasie
3. $\zeta = 1$ — tłumienie krytyczne, powrót do stanu ustalonego jak najszybciej i bez oscylacji

4. $\zeta > 1$ — nadmierne tłumienie

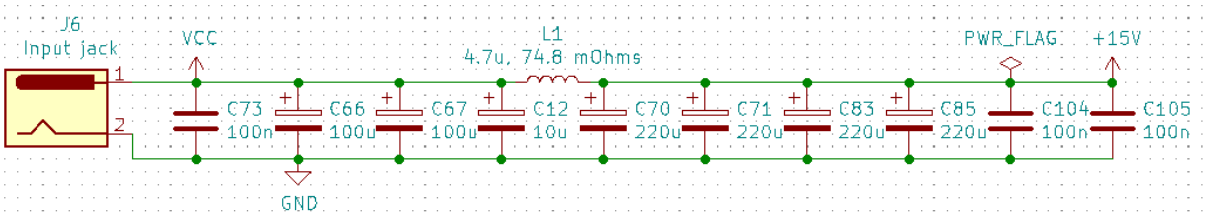
Przy projektowaniu tego typu układów oczekuje się, że będą tłumione krytycznie. Oznacza to, że układ jest na granicy oscylacji, a pik rezonansowy na charakterystyce częstotliwościowej jest nabliższy zakresowi pasma przenoszenia, które nie jest tłumione (możliwe płaskie) [20]. Podstawiając $\zeta = 1$ w równaniu 4.7 otrzymuje się:

$$L = \frac{R\sqrt{2}}{\omega_0}, \qquad C = \frac{1}{\omega_0 R\sqrt{2}}. \tag{4.9}$$

Jako rezystancję przyjmuje się rezystancję wewnętrzną cewki[*nie zawsze, bo czasami dodaje się specjalnie opornik, by ją zwiększyć, lub mieć lepiej kontrolowaną, cewki często mają* ±30%]. W zaprojektowanym układzie $L = 4.7$ μH i $C = 1080$ μF, z czego wynika $f_c \approx 2.47$ kHz oraz warunek na rezystancję cewki:

$$R = \sqrt{\frac{L}{2C}} = \sqrt{\frac{4.7}{2160}} \approx 46.6 \text{ m}\Omega. \tag{4.10}$$

W układzie dobrano dostępną wartość najbliższą obliczeniom i wynosi ona 74.8 mΩ. Warto wspomnieć, że w przypadku doboru rzeczywistych wartości elementów, w tym przypadku lepiej wybrać element o wartości zawyżonej niż zaniżonej, ponieważ bardziej oczekiwane jest uzyskanie większego tłumienia, niż wzmocnienia pasma rezonansowego. Omawiany schemat układu wraz z dołożonymi kondensatorami zapewniającymi dodatkowe buforowanie i filtrację średnich częstotliwości przedstawiono na rysunku 4.3:



Rysunek 4.3: Sekcja wejściowa

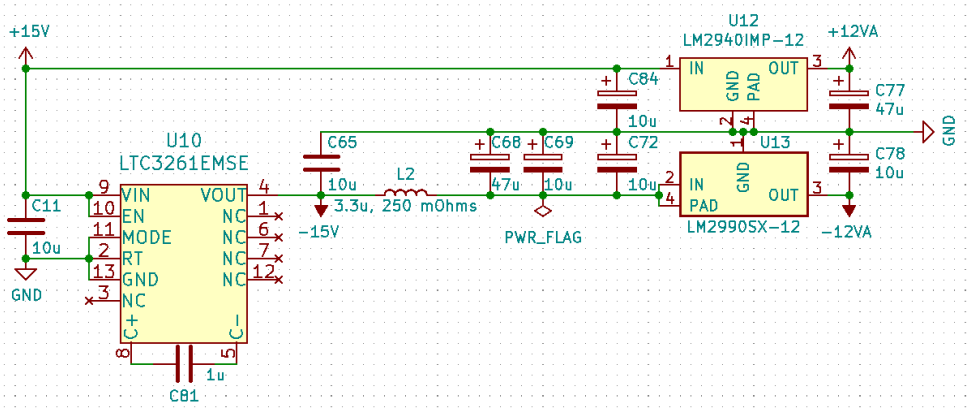
Tak przygotowanie napięcie 15 V jest podstawą do generacji pozostałych zasilań w układzie, jak i również wykorzystywane jest jako zasilanie phantomowe mikrofonów.

4.2.2 Symetryzacja szyn zasilających

Przetwarzany sygnał akustyczny jest symetryczny wokół zera, więc jednym z możliwych rozwiązań jest generacja symetrycznego zasilania dla wzmacniaczy w sekcji torów analogowych, odpowiednio wysokiego, aby nie doporowadzać do saturacji ich wyjść. Wybranym zasilaniem jest ±12 V, a szczegóły dotyczące tego wyboru będą dokładniej opisywane w sekcji torów analogowych.

Stosunkowo sporo czasu poświęcono kwesti generacji ujemnego zasilania. W literaturze [27] [28] opisanych jest wiele sposobów implementacji tego typu układów, z czego najbardziej popularnymi są inwertery napięciowe zaimplementowane w oparciu o wzmacniacze operacyjne lub dedykowane przetwornice (ang. *buck-boost*). Rozwiązanie oparte o skonstruowanie układu inwerterów zostało odrzucone z powodu dodatkowego nakładu pracy. Taki system, poza układem odwracającym napięcie (ang. *rail splitter*) wymagałby dodatkowego wzmacniacza mocy na wyjściu, co jest spowodowane zapotrzebowaniem prądowym wzmacniaczy, według specyfikacji wynoszącym typowo 1.4 mA na układ. Jako, że w projekcie wykorzystywanych jest pięć wzmacniaczy operacyjnych na kanał, a kanałów jest cztery, to wymaganiem jest, aby wydajność prądowa wykorzystanego generatora ujemnego napięcia wynosiła 28 mA. Wyborów dedykowanej przetwornicy został odrzucony z powodu niskiej dostępności układów zgodnych z założoną specyfikacją na rynku.

Wynikowo zdecydowano się na wykorzystanie układu odwracającej pompy ładunków, który spełnia wszystkie założenia projektowe i jest wydajna prądowo (maksymalny pobierany prąd równy 100 mA). Minusem układu jest mała dokładność napięcia wyjściowego z fluktuacjami, które według dokumentacji [8] dla wybranego trybu pracy wyniosą 10 mV. Dodatkowo, układ generuje spadek napięcia w zależności od wyjściowego prądu, który w przypadku 30 mA wyniesie około 750 mV. Dopuszczenie takiego napięcia na wzmacniacze w sekcji analogowej jest niedopuszczalne, między innymi zważając na wskaźnik PSRR, który dla wybranych wzmacniaczy wynosi około 40 dB dla częstotliwości 500 kHz [29] oraz niską dokładność. W związku z tym, napięcie wyjściowe pompy ładunków należy filtrować i stabilizować. Jako filtr zastosowano ponownie układ LC (rzeczywisty RLC) o niższej częstotliwości rezonansowej, ponieważ celem jest filtracja fluktuacji wprowadzanych przez pompę ładunków. Dla dobranego $L = 3.3\ \mu$H i $C = 57\ \mu$F uzyskano $f_c \approx 12.18$ kHz oraz warunek na rezystancję cewki $R \approx 170\ \text{m}\Omega$, który spełniono wybierając najbliższą dostepną wartość równą 250 mΩ. Przefiltrowane ujemne napięcie trafia następnie wraz z jego dodatnim odpowiednikim na układy liniowych regulatorów napięcia o małym spadku (LDO, ang. *Low-dropout*), których celem jest stabilizacja oraz zwiększenie dokładności. Wykorzystane układy według specyfikacji dla zakresu pracy od 12 V do 15 V posiadają dokładność równą 2.5%, wydajność prądową równą maksymalnie 1 A oraz średni stosunek tłumienia zakłóceń równy 66 dB dla częstotliwości powyżej 100 kHz i prądu wyjściowego równego 10 mA [15] [16]. Omawiany układ wraz z dołożonymi kondensatorami odsprzęgającym, których wartości dobierane były według not aplikacyjnych producenta przedstawiono na rysunku 4.4.



Rysunek 4.4: Symetryzacja szyn zasilających

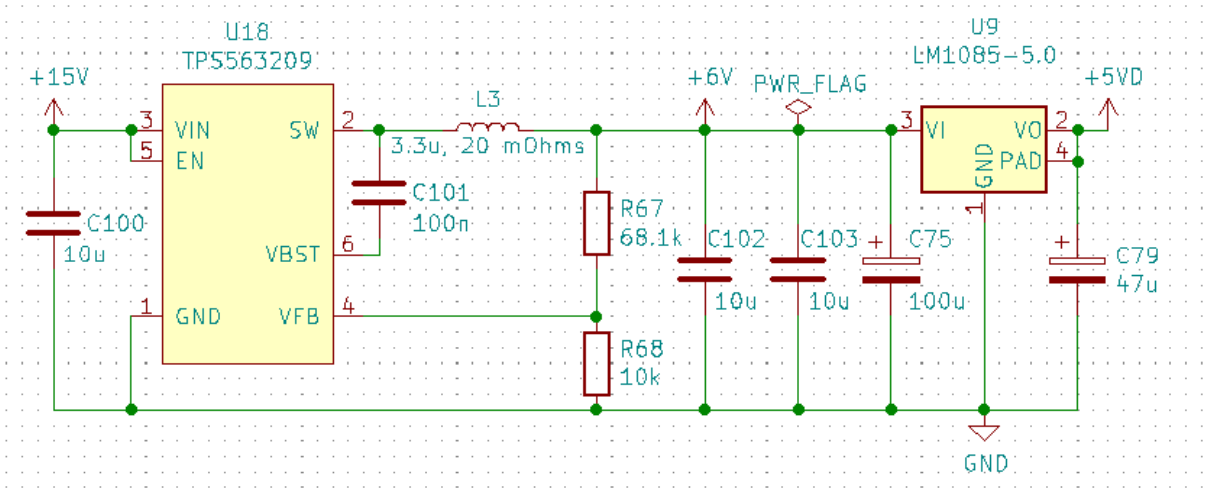
4.2.3 Zasilanie domeny cyfrowej

Celem sekcji jest wygenerowanie napięcia równego 5 V, będącego zasilaniem przetworników cyfrowo-analogowych oraz wybranej platformy obliczeniowej. Maksymalne zapotrzebowanie prądowe tych elementów wynosi odpowiednio 200 μA [29] (100 μA na układ wzmacniacza) oraz około 2 A na platformę [3]. Wykorzystywana jednostka obliczeniowa posiada własną sekcję stabilizacji i filtracji napięcia wejściowego, a wybrane przetworniki cyfrowe-analogowe, w przeciwieństwie do analogowo-cyfrowych, generują wyjście na podstawie dodatkowego sygnału referencyjnego, więc wymagania nałożone na dokładność wygenerowanego zasilania są zdecydowanie mniej rygorystyczne niż w przypadku zasilań i referencji domeny analogowej. W celu generacji napięcia wykorzystano układ impulsowej przetwornicy napięcia (**SMPS**, ang. *Switched mode power supply*), który jest tani, prosty w implementacji oraz zapewnia wysoką wydajność prądową, według dokumentacji wynoszącą do 3 A [30].

Zależność opisująca napięcie wyjściowe wykorzystanego układu podana jest przez producenta i zdefiniowana jest poniższym równaniem:

$$U_{out} = 0.765 \cdot (1 + \frac{R_1}{R_2}). \tag{4.11}$$

Jak opisywano w sekcji dotyczącej filtracji i stabilizacji zasilania wejściowego, przetwornice generują fluktuacje na wyjściu, które należy filtrować. W przypadku tego układu, producent w nocie aplikacyjnej [30] przedstawił zalecane parametry wyjściowego filtru LC, z których skorzystano. Napięcie z przetwornicy trafia końcowo na regulator napięcia o wydajności prądowej równej 3 A [14] w celu stabilizacji oraz zostaje wyprowadzone na pozostałe sekcje. Omowiany układ przedstawiono na rysunku 4.6.



Rysunek 4.5: Zasilanie domeny cyfrowej

Napięcie wyjściowe przetwornicy zgodnie z zależnością 4.11 dla wykorzystanych rezystorów wynosi 5.97 V, co pozwala poprawną stabilizację z wykorzystaniem regulatora niskiego spadku. Do wygenerowanego zasilania równolegle wraz z rezystorem ograniczającym prąd dodatkowo podłączono diodę LED, działającą jako indykator informujący o włączonym układzie.

4.2.4 Referencja domeny analogowo-cyfrowej

Wykorzystane przetworniki wymagają zasilania referencyjnego, na bazie którego przeprowadzane są konwersje. W przypadku przetworników cyfrowo-analogowych, zależność wynosi [2]:

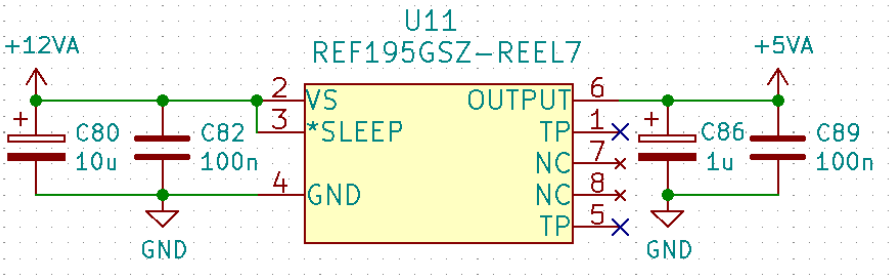
$$U_{out} = \frac{DU_{ref}}{2^N}, \tag{4.12}$$

gdzie D to dane załadowane do rejestru przetwornika, a N to rozdzielczość.

W przypadku przetwornika analogowo-cyfrowego, referencja wpływa bezpośrednio na rozmiar kwantu [1]:

$$q = \frac{U_{ref}}{2^N}. \tag{4.13}$$

Z powyższych równań wynika, że sygnał referencyjny musi być bardzo dokładny. Aby to uzyskać, zastosowano precyzyjny regulator napięcia niskiego spadku, zdolny do generacji napięcia wyjściowego z maksymalnym błędem równym 2 mV i maksymalnym obciążeniem równym 30 mA [23], wystarczającym dla wykorzystanych przetworników. Zakres wejściowych napięć układu zawiera się w przedziale od -0.3 V do +18 V, więc wejściowe napięcie (równe +12 V) jest odpowiednie. Omawiany układ wraz z kondensatorami odsprzęgającymi przedstawiono na rysunku 4.6.



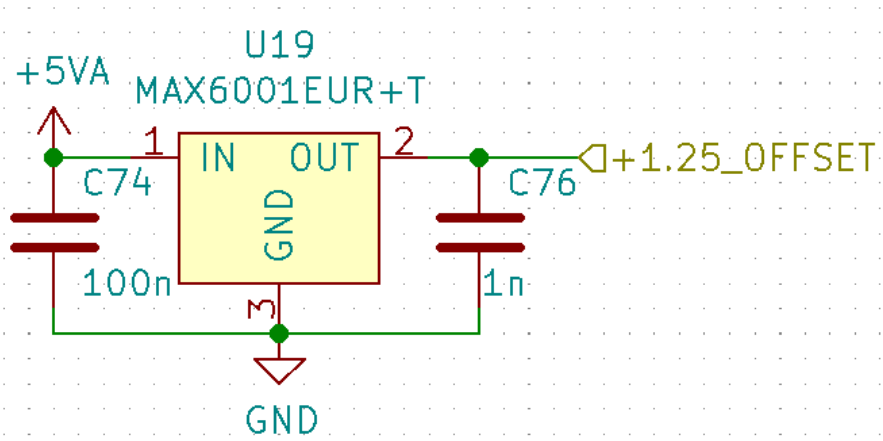
Rysunek 4.6: Referencja domeny analogowo-cyfrowej

Warto dodać, że napięcie referencyjne, prócz przetworników, trafia również na ograniczniki diodowe zastosowane w sekcji wejściowych torów analogowych.

4.2.5 Generacja składowej stałej

Jak dokładniej będzie opisywane w sekcji torów analogowych, w celu zapewnienia pełnego zakresu dynamicznego przetworników analogowo-cyfrowych, niezbędny jest dokładny i stabilny sygnał referencyjny zapewniający odpowiednią składową stałą w przetworzonym sygnale dźwiękowym trafiającym na wejście przetwornika. Od układu generującego taki sygnał nie jest wymagana obsługa dużych obciążeń, ponieważ sygnał na jego wyjściu w zaimplementowanym rozwiązaniu trafia na wejście nieodwracające wzmacniaczy operacyjnych w sekcji analogowej, których impedancja wejściowa jest liczona w MΩ (teoretycznie impedancja wejściowa wzmacniacza operacyjnego jest nieskończona [27]).

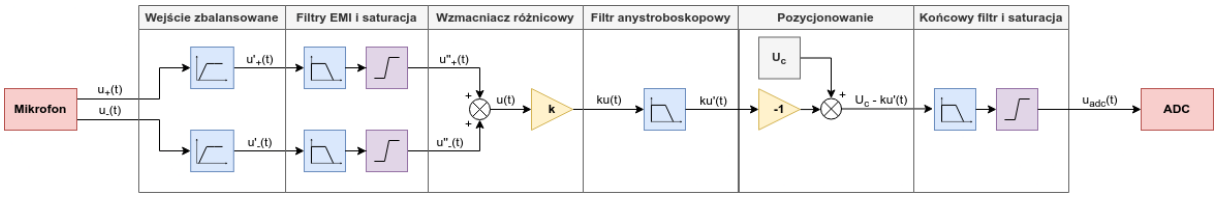
W celu generacji takiego sygnału posłużono się układem napięcia odniesienia [17], który wraz z niezbędnymi kondensatorami odsprzęgającymi przedstawiono na rysunku 4.7.



Rysunek 4.7: Układ napięcia odniesienia

4.3 Wejściowe tory analogowe

Schemat blokowy wejściowego toru analogowego przedstawiono na rysunku 4.8:

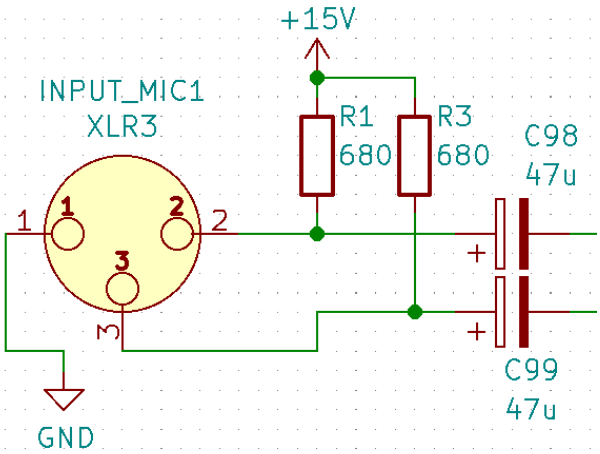


Rysunek 4.8: Schemat blokowy wejściowego toru analogowego

4.3.1 Wejście zbalansowane

Dźwięk trafia na sekcję torów analogowych poprzez trzy-pinowe złącze XLR. Oprócz masy, ze złącza odprowadzane są dwa sygnały, z czego drugi jest odwrócony w fazie względem pierwszego. Tak zrealizowane połączenia, wraz z odpowiednim ekranowaniem w przewodzie, pozwalają na minimalizację zakłóceń oraz strat.

Bezpośrednio po doprowadzeniu sygnałów na układ, poprzez rezystory podciągające (ang. *pull-up resistors*) do sygnałów doprowadzone jest zasilanie phantomowe o wartości +15 V. Standard IEC 61938 definiuje wartości rezystancji dla poszczególnych wartości napięc zasilania phantomowego i wynoszą one 6.81 kΩ dla zasilania +48 V, 1.2 kΩ dla zasilania +24 V i 680 Ω dla zasilania +12 V. Jako, że wybrana wartość zasilania jest niestandardowa ze względu na specyfikację mikrofonów, to jako rezystancje dobrano te, dla których wartości napięcia w standardzie jest najbliższa, czyli 680 Ω. Po doprowadzeniu zasilania sygnały trafiają na kondensatory odcinające składową stałą. Omawianą część schematu przedstawiono na rysunku 4.9.



Rysunek 4.9: Wejście zbalansowane

4.3.2 Początkowa filtracja i saturacja

W celu zabezpieczenia się przed wysokoczęstotliwościowymi zakłóceniami z zewnętrznych źródeł, następnym etapem przetwarzania jest zastosowanie dolnoprzepustowych filtrów EMI (ang. *Electromagnetic-interference filtering*). Obecność wysokoczęstotliwościo-

wych szumów w przetwarzanym sygnale wchodzącym na wejście wzmacniacza różnicowego może doprowadzić do niekorzystnych zmian sygnału wyjściowego. Takie zachowanie opisywane jest przez wskaźnik CMRR (ang. *Common-mode rejection ratio*), który zdefiniowany jest jako stosunek wzmocnienia różnicowego do wzmocnienia sygnału zgodnego [27]. Dla wykorzystywanych wzmacniaczy powyżej częstotliwości 100 kHz, CMRR osiąga wartość około 80 dB. Rzeczywiste CMRR, poza obecnością wysokoczęstotliwościowych szumów w sygnale wejściowym można również pogorszyć niezbalansowaniem układów przed wzmacniaczem, co należy mieć na uwadze projektując układ składający się z szeregowego połączenia układów scalonych.

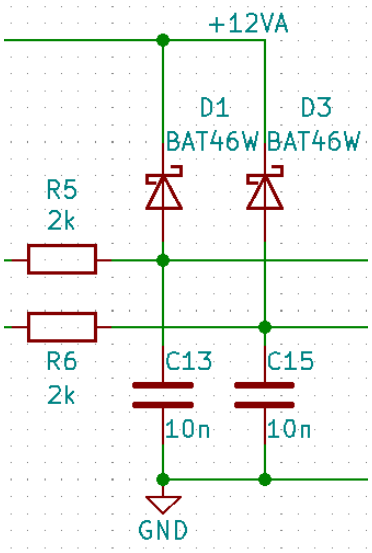
Zaprojektowany filtr jest dolnoprzepustowym filtrem RC pierwszego rzędu, którego opisuje następująca zależność [28]:

$$U_{wy}(s) = U_{we}(s)\frac{Z_1}{Z_1 + Z_2} = U_{we}(s)\frac{\frac{1}{Cs}}{R + \frac{1}{Cs}} \implies K(s) = \frac{1}{RCs + 1}, \tag{4.14}$$

Stała czasowa τ oraz częstotliwość odcięcia 3 dB dane są zależnościami

$$\tau = RC, \quad f_c = \frac{1}{2\pi RC}. \tag{4.15}$$

Dobierając $R = 2$ kΩ i $C = 10$ nF uzyskano $f_c \approx 7.957$ kHz. Poza filtrem, w sekcji zaimplementowano saturację w postaci ogranicznika diodowego ograniczającego amplitudy sygnału wejściowego do limitu +12 V. Omawiany układ przedstawiono na rysunku 4.10.

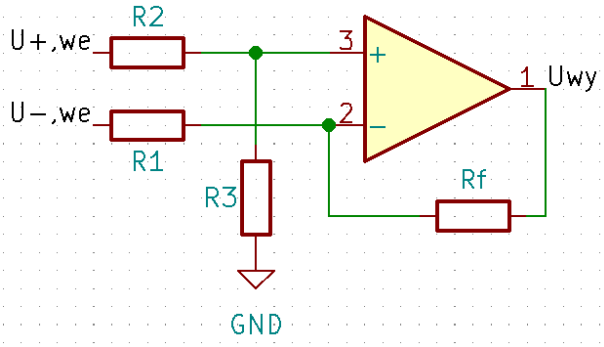


Rysunek 4.10: Filtr EMI wraz z ogranicznikiem diodowym

4.3.3 Wzmocnienie różnicowe

Symetryczny sygnał z mikrofonów w kolejnym etapie należy przekonwertować do pojedynczej, odpowiednio wzmocnionej sumy. Układem wykorzystywanym do tego typu za-

stosowań jest wzmacniacz różnicowy, a jego topologię przedstawiono na rysunku 4.11.



Rysunek 4.11: Wzmacniacz różnicowy

Zależność opisująca układ opiera się na wykorzystaniu zasady superpozycji. Równanie opisujące wyjście w układu w stosunku do wejścia odwracającego przy wirtualnie wyzerowanym wejściu nieodwracającym jest zależnością opisującą wzmocnienie wzmacniacza w topologii odwracającej [28], opisaną jako:

$$U_{-,wy} = -U_{-,we}\left(\frac{R_f}{R_1}\right). \tag{4.16}$$

Wyjście wzmacniacza w stosunku do wejścia nieodwracającego przy wirtualnie wyzerowanym wejściu odwracającym jest zdefiniowane zależnością opisującą wzmacniacz w topologii nieodwracającej w połączeniu z dzielnikiem występującym na wejściu nieodwracającym:

$$U_{+,wy} = U_{+,we}\left(\frac{R_3}{R_2+R_3}\right)\left(1+\frac{R_f}{R_1}\right). \tag{4.17}$$

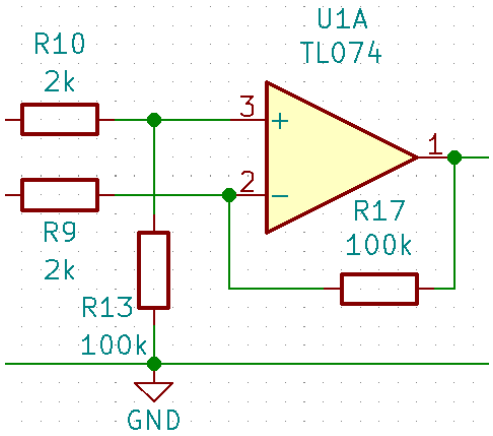
Korzystając z zasady superpozycji otrzymuje się

$$U_{wy} = U_{+,we} - U_{-,wy} = U_{+,we}\left(\frac{R_3}{R_2+R_3}\right)\left(1+\frac{R_f}{R_1}\right) - U_{-,we}\left(\frac{R_f}{R_1}\right). \tag{4.18}$$

Dla $R_1 = R_2$ i $R_3 = R_f$ równanie 4.18 upraszcza się do postaci:

$$U_{wy} = \frac{R_f}{R_1}(U_{+,we} - U_{-,we}). \tag{4.19}$$

Jako wzmocnienie wybrano obliczone wcześniej 4.2 z zaokrągleniem w górę, przyjmując $k = 50$, oraz $R_f = 100$ kΩ i $R_1 = 2$ kΩ w symobolach schematu 4.11. Omawiany układ przedstawiono na rysunku 4.12.



Rysunek 4.12: Wzmocnienie różnicowe

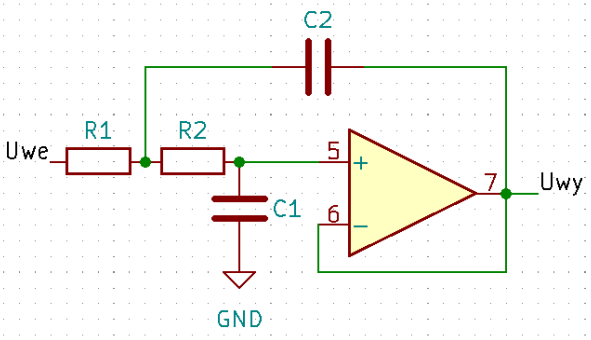
4.3.4 Filtracja antyaliasingowa

Jako, że pasmo interesujących częstotliwości, wynikających z założeń projektowych zamyka się w 500 Hz, zdecydowano się na implementację analogowego filtru Butterwortha szóstego rzędu o częstotliwości granicznej $f_c = 500$ Hz. Filtr ten spełnia rolę filtru antyaliasingowego, umożliwiając próbkowanie [18] wynikające z twierdzenia Nyquista-Shannona

$$f_p > 2f_s. \tag{4.20}$$

Dla $f_s = f_g = 500$ Hz sygnał może zostać spróbkowany z częstotliwościami większymi niż 1 kHz.

Projektując układ posłużono się topologią Sellena-Keya z jednostkowym wzmocnieniem [28], którą przedstawiono na rysunku 4.13.



Rysunek 4.13: Topologia Sellena-Keya z jednostkowym wzmocnieniem

Transmitancję takiego układu, będącego aktywnym filtrem dolnoprzepustowym drugiego rzędu opisuje:

$$K_f(s) = \frac{1}{b_1 s^2 + a_1 s + 1}, \tag{4.21}$$

gdzie:

$$b_1 = R_1 R_2 C_1 C_2, \quad a_1 = C_1(R_1 + R_2). \tag{4.22}$$

Transmitancję filtru szóstego rzędu można zapisać jako iloczyn trzech filtrów: 4.21

$$K(s) = K_1(s)K_2(s)K_3(s) = \prod_{i=1}^{3} \frac{1}{(b_i s^2 + a_i s + 1)}. \tag{4.23}$$

Z wykorzystaniem funkcji wspomagających projektowanie filtrów analogowych języka programowania Python (biblioteka SciPy, funkcje *signal.butter* i *signal.zp2tf*) uzyskano wartości współczynników transmitancji 4.23 dla $f_c = 500$ Hz:

$$b_1 = b_2 = b_3 = 1.0132\text{e-}07, \tag{4.24}$$

$$\mathbf{A} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 6.1493\text{e-}04 \\ 4.5016\text{e-}04 \\ 1.6477\text{e-}04 \end{bmatrix}, \tag{4.25}$$

Mając policzone współczynniki transmitancji 4.23, z równań 4.22 można obliczyć wartości poszczególnych elementów układu. Dla przyjętych pojemności wartości rezystancji określone są zależnościami:

$$R_{1_i,2_i} = \frac{C_2 a_i \pm \sqrt{C_2(C_2 a_i^2 - 4C_1 b_i)}}{2C_1 C_2}, \qquad \text{dla } i = 1, 2, 3. \tag{4.26}$$

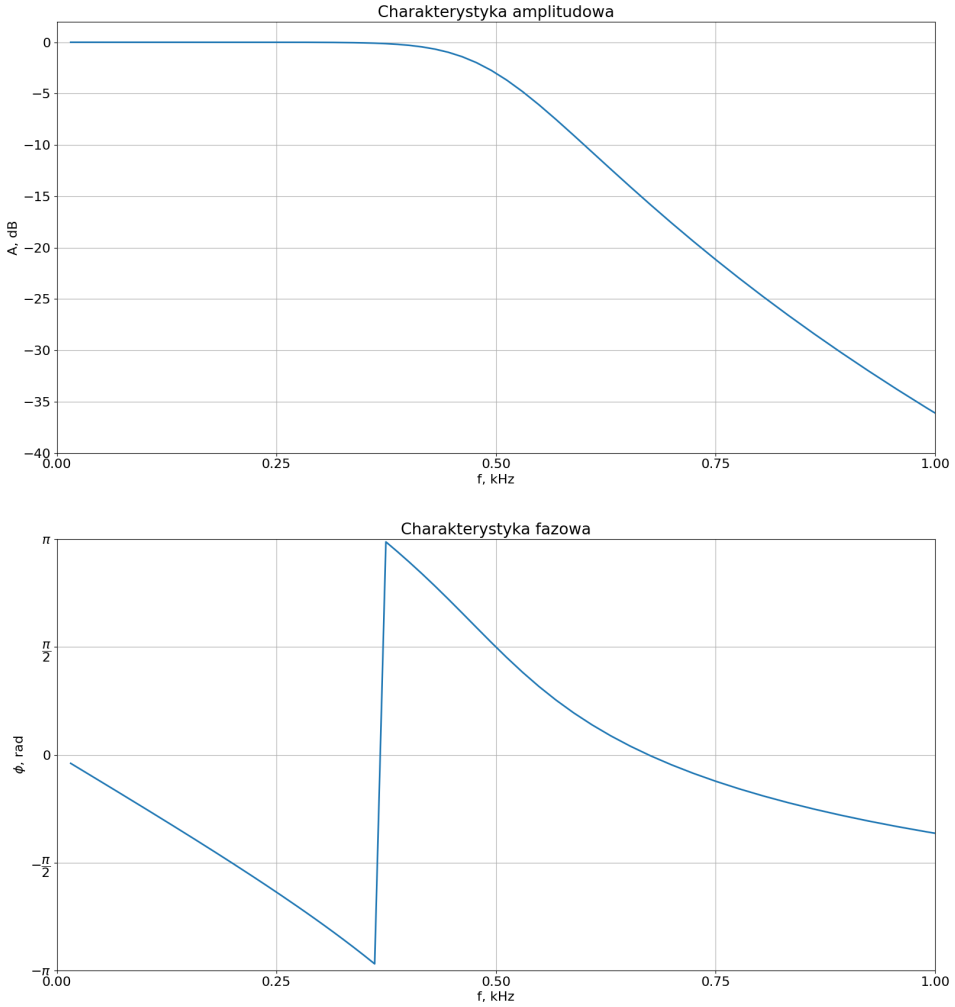
Dla następujących rzeczywistych wartości pojemności:

$$\mathbf{C} = \begin{bmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \\ C_{3,1} & C_{3,2} \end{bmatrix} = \begin{bmatrix} 4.7\text{e-}09 & 5.6\text{e-}09 \\ 2.7\text{e-}09 & 6.8\text{e-}09 \\ 560\text{e-}12 & 22\text{e-}09 \end{bmatrix} \text{ F}, \tag{4.27}$$

otrzymano następujące teoretyczne wartości rezystancji:

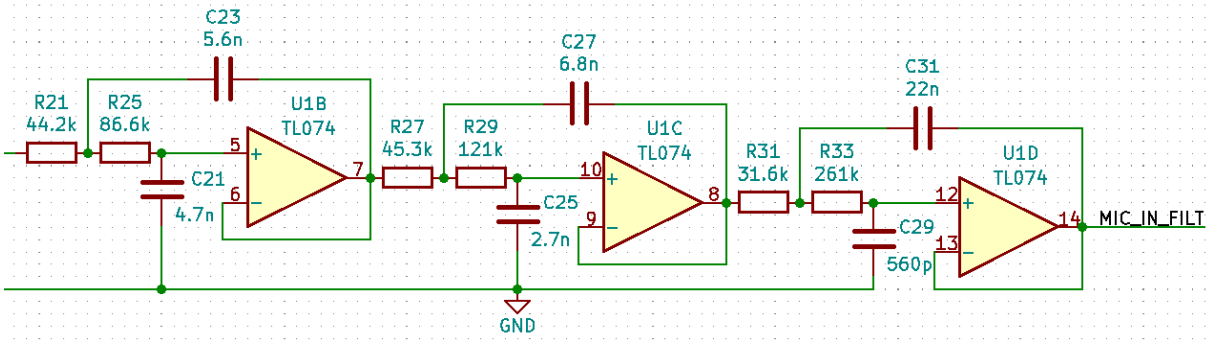
$$\mathbf{R} = \begin{bmatrix} R_{1,1} & R_{1,2} \\ R_{2,1} & R_{2,2} \\ R_{3,1} & R_{3,2} \end{bmatrix} = \begin{bmatrix} 4.4683\text{e+}04 & 8.6151\text{e+}04 \\ 4.5537\text{e+}04 & 1.2118\text{e+}05 \\ 3.1276\text{e+}04 & 2.6295\text{e+}05 \end{bmatrix} \ \Omega. \tag{4.28}$$

Charakterystykę amplitudową i fazową układu przedstawiono na rysunku 4.14:



Rysunek 4.14: Charakterystyka aplitudowa i fazowa układu filtra

Po wybraniu rzeczywistch wartości rezystancji najbliższych teoretycznym, omawiany układ przedstawiono na rysunku 4.15.



Rysunek 4.15: Filtr antyaliasingowy

4.3.5 Pozycjonowanie

Celem układu pozycjonującego jest zapewnienie, że sygnał wejściowy przetwornika analogowo-cyfrowego jest wyskalowany do jego pełnego zakresu dynamicznego, wynoszą-

cego od 0 V do +5 V. Sygnał przed trafieniem na omawianą sekcję jest już odpowiednio wzmocniony i przefiltrowany, więc jedynym zadaniem układu jest dodanie do niego składowej stałej, będącej środkiem zakresu przetwornika.

Zadanie te zrealizowano poprzez zastosowanie układu wzmacniacza różnicowego przedstawionego na rysunku 4.11, z modyfikacją wykluczającą dzielnik napięciowy złożony z rezystorów R_1 i R_2. To implikuje zmianę równania 4.17 na klasyczną zależność opisującą wzmocnienie wzmacniacza w topologii nieodwracającej [28]:

$$U_{+,wy} = U_{+,we}\left(1 + \frac{R_f}{R_1}\right). \tag{4.29}$$

Zależność opisująca wyjście ze względu na wejście $U_{-,we}$ pozostaje niezmienna.

Zważając na powyższe, wyjście układu wynikające z zasady superpozycji jest opisane zmodyfikowaną zależnością 4.18:

$$U_{wy} = U_{+,we} - U_{-,wy} = U_{+,we}\left(1 + \frac{R_f}{R_1}\right) - U_{-,we}\left(\frac{R_f}{R_1}\right). \tag{4.30}$$

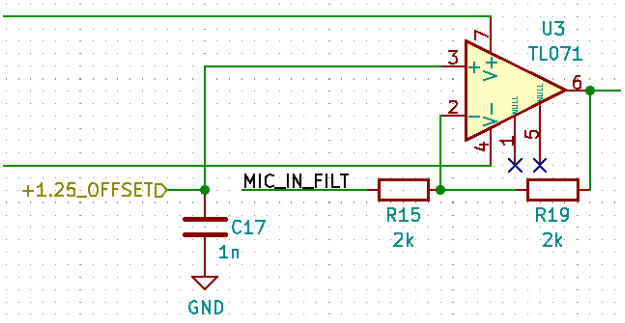
Dla arbitralnej wartości $R_f = R_1$ otrzymuje się:

$$U_{wy} = 2U_{+,we} - U_{-,we}. \tag{4.31}$$

Chcąc uzyskać stałe przesunięcie sygnału $U_{-,we}$ o środek zakresu przetwornika $U_c = 2.5$ V uzyskuje się warunek na pierwszy składnik równania 4.31:

$$2U_{+,we} = 2.5\text{ V} \implies U_{+,we} = 1.25\text{ V}. \tag{4.32}$$

Jak można zauważyć z równiania 4.31 sygnał $U_{-,we}$ zostaje w tym etapie odwrócony. Omawiany układ wraz z kondensatorem pełniącym rolę lokalnego bufora składowej stałej przedstawiono na rysunku 4.16.



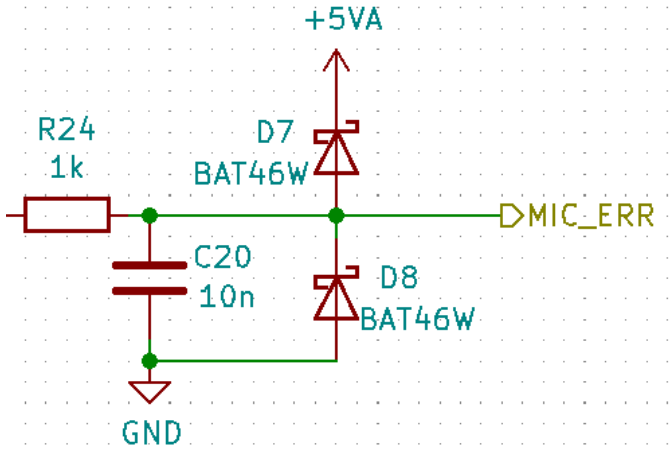
Rysunek 4.16: Układ pozycjonujący

4.3.6 Filtr końcowy i saturacja

Bezpośrednio przed wejściem przetwornika dołączono dodatkowy filtr dolnoprzepustowy RC pierwszego rzędu 4.14, odcinający możliwe szumy powstałe w wyniku zastosowanych wzmacniaczy operacyjnych oraz zapewniający lokalny bufor energii dla kondensatora próbkującego wewnątrz przetwornika. Wymagania na taki filtr nie są surowe, ponieważ niechciane pasmo zostało odfiltorwane przez wcześniejszy filtr antyaliasingowy, dlatego zdecydowano się na wartości $R = 1$ kΩ i $C = 10$ nF, uzyskując w ten sposób $f_c \approx 15.91$ kHz.

Wejście przetwornika analogowo-cyfrowego należy zabezpieczyć przed napięciami przekraczającymi jego zakres. O układzie realizującym to zadanie wspominano już w sekcji wejścia zbalansowanego, lecz w tym przypadku, limiter ogranicza amplitudy sygnału wejściowego do zakresu przetwornika. Z tego wynika, że diody w układzie będą połączone z zasilaniem +5 V i masą (0 V).

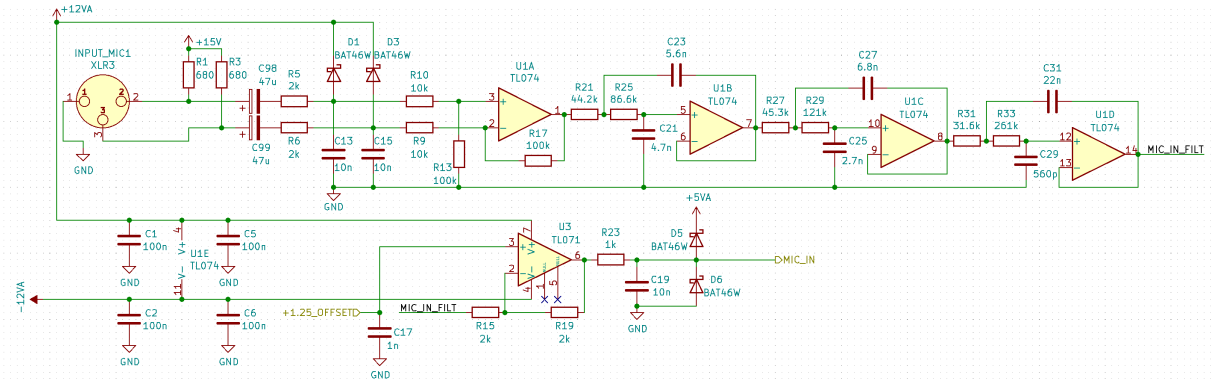
Omawiany układ przedstawiono na rysunku 4.17:



Rysunek 4.17: Filtr dolnoprzepustowy i limiter[*KiCAD umożliwia zapis schematów jako pdf, jakoś wtedy byłaby dużo lepsza.*]

4.3.7 Pełen schemat sekcji

Pełen schemat omawianej sekcji przedstawiono na rysunku 4.18

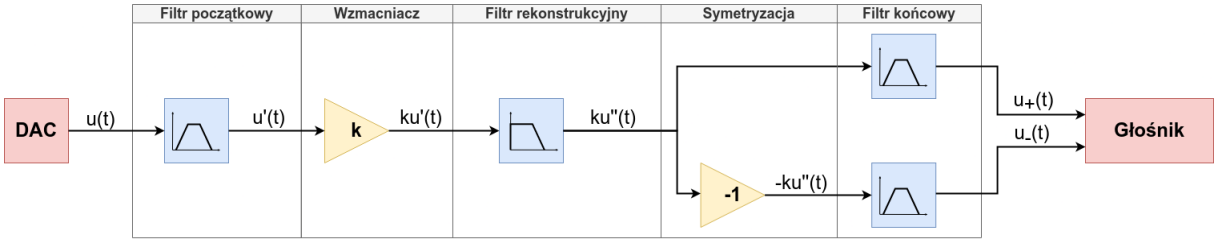


Rysunek 4.18: Pełen schemat wejściowej sekcji analogowej

Do symetrycznych linii zasilających wzmacniaczy operacyjnych dołączono kondensatory odsprzęgające, filtrujące szumy zasilania. Jako wartości dobrano typowe 100 nF na każdy pin zasilający wzmacniacza.

4.4 Wyjściowe tory analogowe

Schemat blokowy wyjściowego toru analogowego przedstawiono na rysunku 4.19:

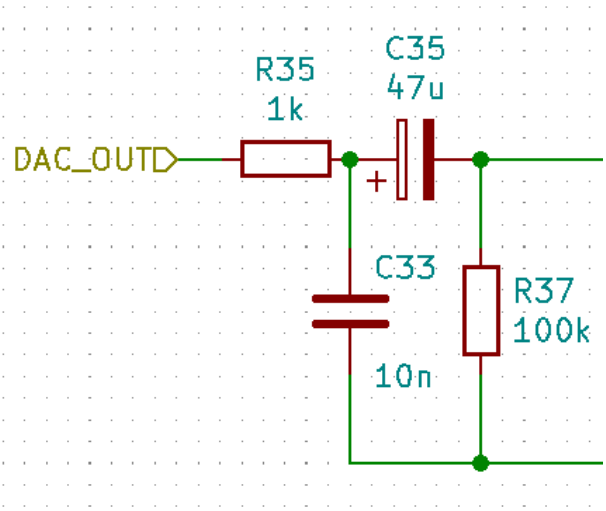


Rysunek 4.19: Schemat blokowy wyjściowego toru analogowego

4.4.1 Filtracja początkowa

Sygnał wychodzący bezpośrednio z przetwornika cyfrowo-analogowego wymaga wstępnej filtracji, mającej na celu usunięcie wysokoczęstotliwościowych artefaktów kowersji jak i składowej stałej +2.5 V wynikającej z zakresu od 0 V do +5 V. Jako, że późniejszym etapem przetwarzania jest stromy dolnoprzepustowy filtr rekonstukcyjny, to wymagania stawiane omawianemu układowi nie są surowe.

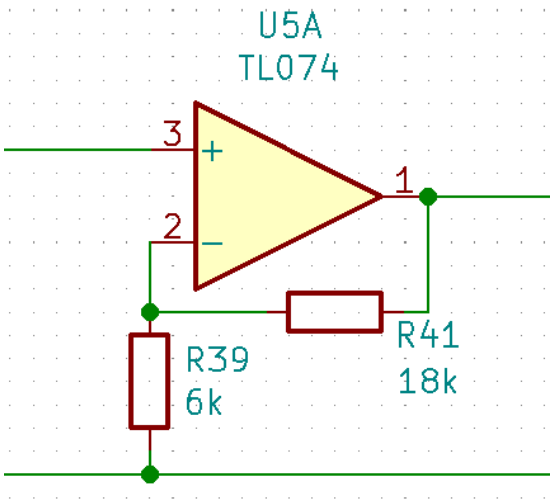
Jako układ realizujący opisane wyżej zadanie wybrano pasywny filtr pasmowoprzepustowy, złożony z szeregowego połączenia dolnoprzepustowego i górnoprzepustowego filtru RC. Dla sekcji dolnoprzepustowej dobrano $R = 1$ kΩ i $C = 10$ nF, uzyskujac w ten sposób $f_c \approx 15.91$ kHz, natomiast dla sekcji górnoprzepustowej dobrano $R = 100$ kΩ i $C = 47$ μF, uzyskując $f_c < 1$ Hz. Omawiany układ przedstawiono na rysunku 4.20:



Rysunek 4.20: Wejściowy filtr pasmowoprzepustowy

4.4.2 Wzmocnienie

Przefiltrowany sygnał w następnej kolejności trafia na sekcję wzmacniacza zaimplementowanego w topologii nieodwracającej 4.29. Z racji, że wykorzystywane głośniki posiadają regulację głośności, jedynym obligatoryjnym wymaganiem przy doborze wzmocnienia układu jest uniknięcie saturacji wyjścia, które ograniczone jest przez symetryczne zasilanie układu, równe ±12 V. W związku z tym, zdecydowano się wzmocnienie $k = 4$ poprzez zastosowanie rezystorów $R_1 = 6$ kΩ i $R_f = 18$ kΩ. Taki wybór skutkuje $U_{wy} = 10$ V dla $U_{we} = 2.5$ V, czyli górnego zakresu przetwornika, co daje zalecane dla tego układu 2 V zapasu. Omawiany układ przedstawiono na rysunku 4.21:



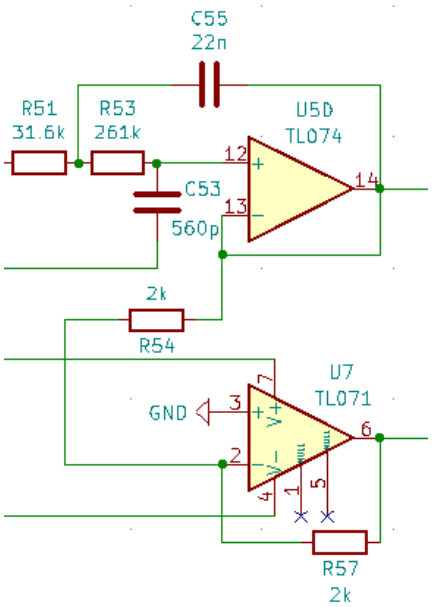
Rysunek 4.21: Wzmocnienie sekcji wyjściowej

4.4.3 Filtracja rekonstrukcyjna

Zaimplementowany filtr rekonstrukcyjny jest powieleniem filtru antyaliasingowego implementowanego w sekcji analogowych torów wejściowych. Jest to typowe podejście przy projektowaniu układów, posiadających analogowe wejście i wyjście [27] będące efektem przetwarzania wejścia. W wyniku tej decyzji, wyjście układu jest ograniczone do założonego pasma wejściowego.

4.4.4 Symetryzacja

Wyjście analogowe projektowane jest z myślą o złączach XLR, co oznacza, że sygnał wyjściowy, tak jak wejściowy, musi być symetryczny (zbalansowany). W celu jego wytworzenia zastosowano układ wzmacniacza operacyjnego w topologii odwracającej z jednostkowym wzmocnieniem 4.16 ($R_1 = R_f$). Sygnał wyjściowy $(-)$ takiego układu jest odwrócony w fazie względem wejściowego $(+)$, co pozwala na uzyskanie symetrycznej pary wyprowadzanej docelowo na złącze XLR. Przy wykorzystaniu takiego układu należy mieć na uwadze, że wzmacniacz generujący sygnał odwrócony $(-)$ jest elementem nie mającym odpowiednika w torze sygnału nieodwróconego $(+)$. Układy sterujące wyjściem tej sekcji, czyli odpowiednio ostatni wzmacniacz w układzie filtra dla sygnału nieodwróconego $(+)$ oraz wzmacniacz odwracający dla sygnału odwróconego $(-)$ mogą zachowywać się znacząco inaczej. Możliwym rozwiązaniem byłoby dodanie wtórnika napięciowego w ścieżce sygnału nieodwróconego $(+)$, lecz wiązałoby się to z dodatkowymi wzmacniaczami operacyjnymi na kanał wyjściowy. Jako, że wykorzystywanych wzmacniaczy na tym etapie jest już stosunkowo dużo (pięć na kanał), z pomysłu zrezygnowano, mając świadomość kompromisu. Omawiany fragment schematu przedstawiono na rysunku 4.22:

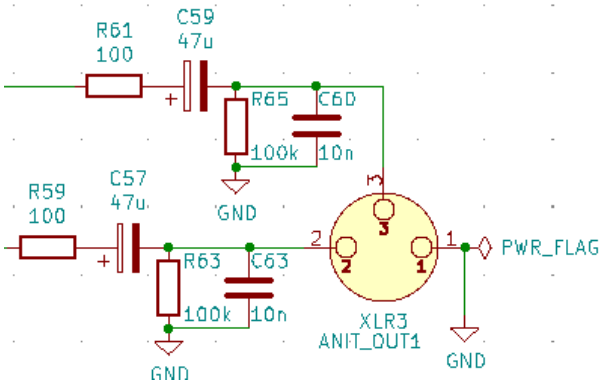


Rysunek 4.22: Układy sterujące wyjściem analogowym

4.4.5 Filtracja końcowa

Bezpośrednio przed złączem XLR zastosowano filtr pasmowoprzepustowy, chroniący przed możliwie wystąpującymi w przetworzonym sygnale szumami powstałymi w wyniku zastosowanych wzmacniaczy operacyjnych, jak i możliwie występującą składową stałą, której obecność może okazać się destrukcyjna dla głośnika.

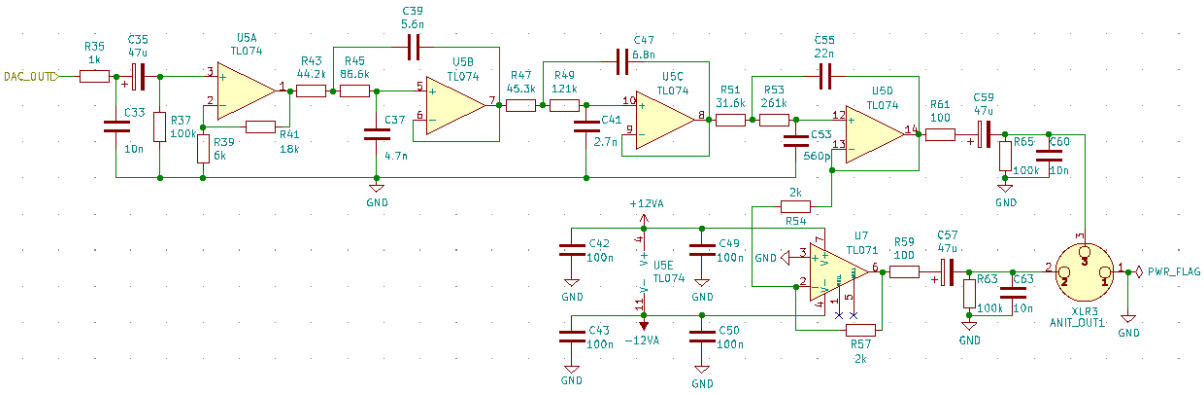
Przyjęte parametry są podobne do filtra na wejściu sekcji, jednak w tym przypadku zmniejszono rezystancję opornika w układzie dolnoprzepustowym, aby nie zwiększać impedancji wyjściowej toru. Skutkuje to przemnożeniem częstotliwości granicznej filtru przez 10, co daje $f_c \approx 159.1$ kHz. Dodatkowo, zmieniono kolejność elementów w układzie, starając się umieścić kondensator równolegle jak najbliżej złącza. Omawianą część schematu przedstawiono na rysunku 4.23:



Rysunek 4.23: Filtracja końcowa toru analogowego

4.4.6 Pełen schemat sekcji

Pełen schemat omawianej sekcji przedstawiono na rysunku 4.24:



Rysunek 4.24: Pełen schemat wyjściowej sekcji analogowej

Analogicznie jak w wejściowej sekcji analogowej, na każdy pin zasilania wzmacniaczy zastosowano kondensator odsprzęgający o typowej wartości 100 nF.

4.5 Sekcja cyfrowa

4.5.1 Interfejs cyfrowy

McASP (ang. *Multi-channel audio serial port*) jest cyfrowym interfejsem komunikacyjnym opracowanym przez firmę Texas Instruments w celu obsługi wielokanałowych systemów audio. W zależności od aplikacji, interfejs wykorzystywany jest do realizacji transferów multipleksowanych metodą TDM (ang. *Time-division multiplexing*) z wyszczególnieniem konfiguracji I^2S (ang. *Inter-IC sound*), będących standardowym protokołem komunikacyjnym z urządzeniami wysyłającymi dane cyfrowe reprezentujące dźwięk w formacie PCM (ang. *Pulse code modulation*). Piny obsługiwane przez interfejs McASP można podzielić na trzy grupy [31]:

1. Piny zegarowe - dzielone na trzy podgrupy:
   (a) Wysokoczęstotliwościowe (**AHCLK**) — wykorzystywane do generacji zegarów bitowych (**ACLK**) i selektorów (**AFS**) poprzez wykorzystanie odpowiednio skonfigurowanych dzielników częstotliwości.
   (b) Bitowe (**ACLK**) — wykorzystywane do taktowania danych wejściowych i wyjściowych względem odpowiednich zboczy.
   (c) Selektory (**AFS**) — definują aktualnie wysyłany kanał. W terminologii dla trybu stereo (dwukanałowego), w stanie niskim przesyłane jest lewy, a w wysokim prawy kanał. Oznacza to, że na jednej linii danych można wysłać dane pochodzące z dwóch kanałów audio.
2. Piny resetujące — umożliwiają wyłączanie (w kontekście audio wyciszanie) peryferiów w zależności od zdefiniowanej logiki. Sygnały zazwyczaj podpinane są do wejść resetujących dane cyfrowe układy audio.
3. Piny danych (**ARX**) — nazwa jest samo wyjaśniająca.

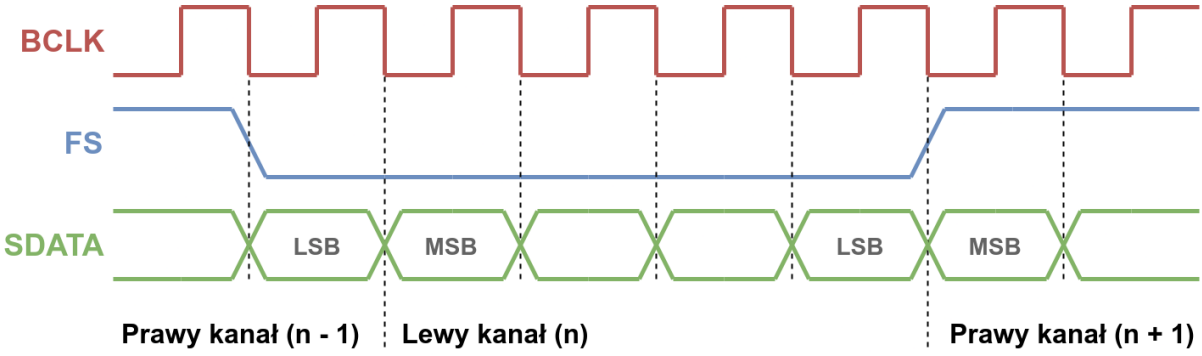
Każdy pin z przedstawionych wyżej grup może zostać skonfigurowany w trybie wejściowym lub wyjściowym. Dla trybu wejściowego, terminologia McASP definuje nazwę z postfiksem R, a dla wyjściowego z postfiksem X (przykładowo **ACLKX** jako wyjściowy zegar bitowy, **AFSR** jako wejściowy selektor). Interfejs pracuje na bazie modelu *master-slave*, gdzie *master* jest urządzeniem zapewniający główny zegar (**AHCLK**) służący do generacji pozostałych.

Metody komunikacji

Transfery z wykorzystaniem trybu I^2S i jego pochodnych realizowane są z wykorzystaniem linii zegarowej (ang. *Bit clock*, **BCLK**), selektora kanału (ang. *Frame select*,

FS) oraz danych (ang. *Serial data*, **SDATA**). Dane przesyłane są w formacie definiującym najbardziej znaczący bit na początku transmisji (ang. *MSB first*), której rozpoczęcie definiowane jest przez określony stan selektora kanału (**FS**). Typowo zbocze opadające selektora definuje transmisję kanału lewego, a narastające prawego, lecz możliwe są inne konfiguracje.

Zapis może odbywać na zboczu narastającym lub opadającym zegara (**BCLK**), jednak wymagane jest, aby odbiornik czytał dane na zboczu narastającym, co powoduje pewne komplikacje, kiedy zapis odbywa się w identyczny sposób. Domyślnie, zmiany selektora kanału (**FS**) odbywają się na zboczach opadających. Standardowy tryb posiada charakterystyczne, zdefiniowane w standardzie opóźnienie o jeden cykl zegara (**BCLK**), powodujące, że najbardziej znaczący bit jest odczytywany dopiero na jego drugim zboczu narastającym, liczonym od momentu rozpoczęcia transmisji dla określonego kanału. Diagram czasowy transmisji z wykorzystaniem trybu I^{2}S w domyślnej konfiguracji przedstawiono na rysunku 4.25.



Rysunek 4.25: Diagram czasowy komunikacji I^{2}S

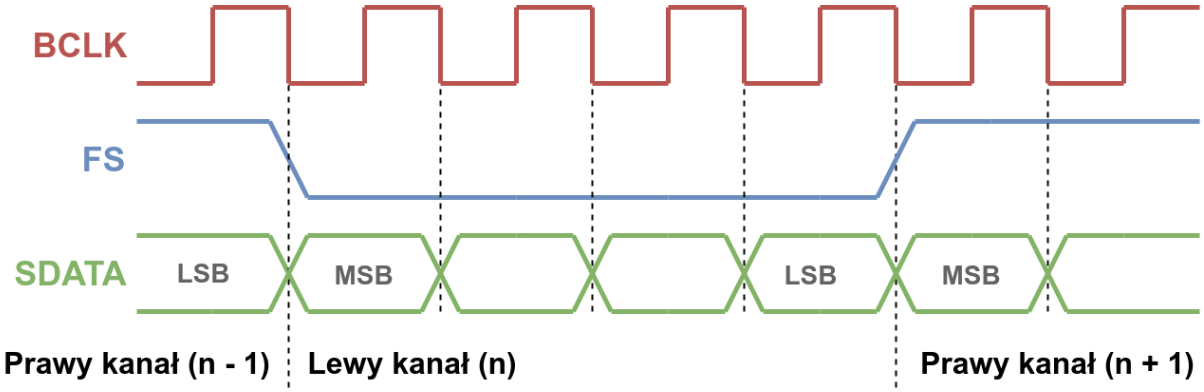
Zależność opisująca częstotliwość zegara bitowego (**BCLK**) zdefiniowana jest:

$$f_{\mathrm{BCLK}} = NLf_s, \tag{4.33}$$

gdzie N to ilość kanałów (od 1 do 2 dla jednej linii), L to bitowa długość transferu (typowo od 16 bitów do 32 bitów) a f_s to częstotliwość próbkowania, za którą bezpośrednio odpowiada częstotliwość selektora kanałów:

$$f_s = f_{FS}. \tag{4.34}$$

W przypadku, kiedy komunikacja odbywa się z peryferiami, dla których opóźnienie standardowego trybu jest problematyczne, istnieje możliwość skorzystania z trybu pochodnego (ang. *Left justified* I^{2}S), który umożliwa odczyt danych na pierwszym wystąpieniu zbocza narastającego liczonego od momentu zmiany selektora. Elimiuje to opóźnienie jednego cyklu, gdzie pozostała charakterystyka komunikacji pozostaje taka sama. Diagram czasu z wykorzystaniem omawianego trybu pochodnego przedstawiono na rysunku 4.26.



Rysunek 4.26: Diagram czasowy komunikacji *Left justified* I^2S

4.5.2 Interakcja z przetwornikami

Wykorzystanie interfejsu McASP jest niezwykle elastyczne, głównie ze względu na dostępne opcje zegarowe i ilość możliwych linii danych. Nota [31] specyfikuje 16 serializatorów, co umożliwia uzyskanie aż 32 kanałów, jeżeli wykorzystywany jest tryb. Główny zegar (**AHCLK**), na podstawie którego generowane są pozostałe, pozwala na pełną synchronizację wszystkich podpiętych urządzeń w systemie, co jest niezbędne dla systemu wymagającego stałej częstotliwości próbkowania.

Kwestią o którą należy zadbać, jest sposób realizacji transferów. W przypadku przetworników analogowo-cyfrowych i cyfrowo-analogowych, najwcześniej wykorzystywanym interfejsem jest SPI (ang. *Serial Peripheral Interface*). Transmisja z użyciem takiego interfejsu odbywa się synchronicznie w sposób dwukierunkowy z wykorzystaniem linii danych wyjściowych (ang. *Master output, slave input*, **MOSI**), danych wejściowych (ang. *Master input, slave output*, **MISO**), zegarowej (ang. *Serial clock*, **SCLK**), oraz selektora układu (ang. *Chip select*, **CS**). Interfejs SPI nie posiada ściśle zdefiniowanego standardu działania. Charakterystykę transmisji typowo definiuje peryferium podrzędne (*slave*), do której układ nadrzędny (*master*) musi się przystosować. Charakterystyki transmisji przyjęto definiować w postaci czterech trybów, które zależą od wartości parametrów opisujących polaryzację zegara w stanie bezczynności (*idle state*, parametr **CPOL**), oraz fazy zegara (parameter **CPHA**), czyli rodzaju zbocza, na którym odbywa się zapis/odczyt.

Wpływ parametrów na charakterystykę transmisji przedstawiono w tabelach 4.1 i 4.2:

Wartość CPOL	**Wpływ**
0	Zegar w stanie niskim (stan bezczynności)
1	Zegar w stanie wysokim (stan bezczynności)

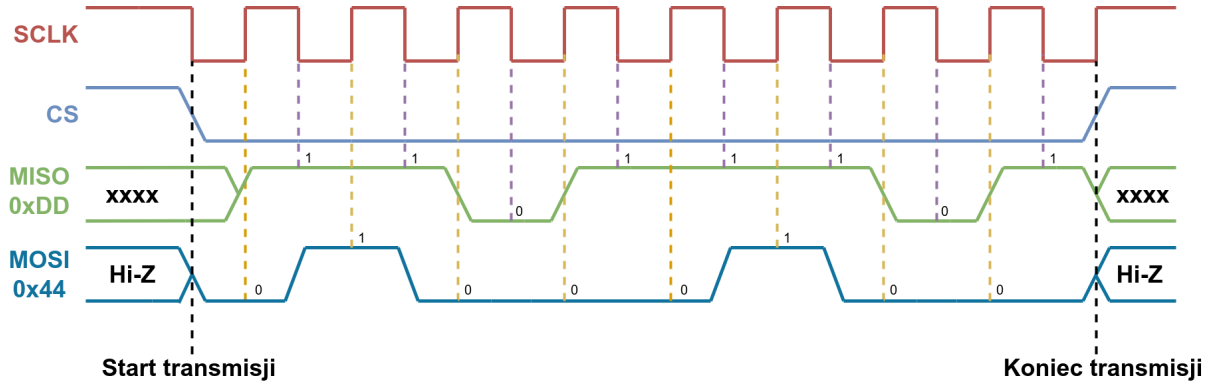
Tablica 4.1: Wpływ wartości CPOL na charakterystyke transmisji

Wartość CPHA	Wpływ
0	Zapis na zboczu opadającym, odczyt na narastającym
1	Zapis na zboczu narastającym, odczyt na opadającym

Tablica 4.2: Wpływ wartości CPHA na charakterystyke transmisji

Numer trybu otrzymywany jest poprzez połączenie wartości **CPOL** i **CPHA** w postać dwubitową, gdzie wartość **CPOL** definuje najbardziej znaczący bit, a **CPHA** najmniej. Przykładowo, ustawienie **CPOL** = 1 i **CPHA** = 0 da wynikowo tryb 2.

Transmisja SPI rozpoczyna się typowo od wyboru układu linią selektora (**CS**), który schodząc na stan niski, inicjalizuje komunikację. Niezależne selektory, których może być więcej niż jeden, pozwalają na wykorzystanie pozostałych linii do komunikacji z wieloma układami. Przykładowy diagram czasowy komunikacji SPI w trybie 3 przedstawiono na rysunku 4.27:



Rysunek 4.27: Ideowy diagram transmisji SPI w trybie 3

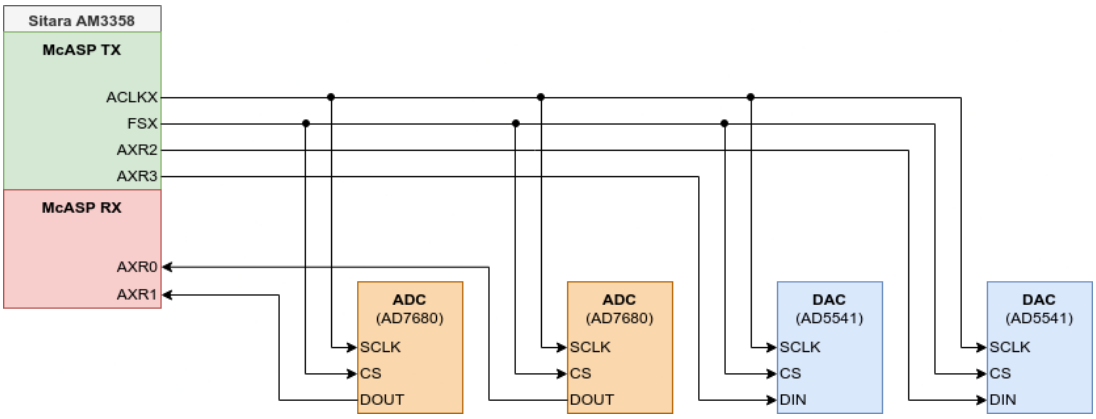
Bazując na powyższym opisie można stwierdzić, że transfer I^2S (w trybie *Left justified* I^2S). jest tak właściwie szczególnym, jednokierunkowym transferem SPI realizowanym w trybie 3 (odczyt na zbocze narastające, zapis na opadające, domyślnie wysoki stan zegara). W związku z tym, interakcja układów komunikujących się po interfejsie SPI w systemie bazującym na transferach I^2S nie powinna być problematyczna. Realizację połączeń po stronie dwóch interfejsów przedstawiono w tabeli 4.3:

Domena SPI	Domena I^2S
SCLK	BCLK
CS	FS
MISO/MOSI	SDATA

Tablica 4.3: Realizacja połączeń SPI do I^2S

Rolę selektora układu (**CS**) w tak zrealizowanym połączeniu pełni selektor kanału

(**FS**) co powoduje, że dane otrzymywane są periodycznie i interpretowane jako jeden kanał (domyślnie lewy). Obligatoryjne jest, żeby wykorzystane w ten sposób układy nie wymagały dwukierunkowych transferów. W przypadku zapisywania/odczytywania danych, układy nie mogą wymagać dodatkowych odczytów/zapisów. Podpinając jeden zegar (**BCLK**) i selektor kanału (**FS**) do wielu układów uzyskuje się synchroniczną transmisję wielokanałową, gdzie jeden układ odpowiada jednemu kanałowi, wejściowemu, jeżeli jest przetwornikiem analogowo-cyfrowym, lub wyjściowemu, jeżeli jest przetwornikem cyfrowo-analogowym. Przy doborze przetworników należy zwrócić szczególną uwagę na zgodność transferów. Tryb I^2S jest konfigurowany globalnie dla całego interfejsu McASP, co oznacza, że konwencja zapisu na zbocze narastające, odczytu na opadające oraz ilości bitów na transfer zostaje przyjęta przez wszystkie linie danych. Diagram połączeń sekcji przetworników przedstawiono na rysunku 4.28:



Rysunek 4.28: Diagram połączeń sekcji przetworników

Wykorzystane przetworniki w obu przypadkach posiadają rozdzielczość 16 bitów, więc zapewniony jest wspólny format przetwarzanych danych.

Przetworniki analogowo-cyfrowe

W celu minimalizacji opóźnień w torze przetwarzania, wybrane przetworniki analogowo-cyfrowe zrealizowane są w architekturze SAR i posiadają niski czas konwersji, wynoszący 8 μs dla transferu 24 bitowego. Ich specyfikacja komunikacji określa, że każdy transfer rozpoczynany jest czterami bitami zerowymi liczonymi od momentu pojawienia się zbocza opadającego na selektorze, po których następuje zapisanie wyniku konwersji. Jeżeli transmisja jest dłuższa niż optymalne 20 bitów, to w dodatkowe miejsca wpisywane są zera. Wykorzystywany przez przetwornik sposób transmisji wprowadza zatem wymaganie transferów równych lub dłuższych niż 20 bitów. Zapis odbywa się na zboczu opadającym, a dane przesyłane są w formacie najbardziej znaczącego bitu na początku (ang. *MSB first*). Maksymalna prędkość transmisji wynosi 100 kSPS, a układ zasilany jest napięciem +5 V, które pełni rolę referencji i określa zakres dynamiczny.

Przetworniki cyfrowo-analogowe

Komunikacja z wybranymi przetwornikami cyfrowo-analogowymi odbywa się poprzez transfery o długości 16 bitów w formacie najbardziej znaczącego bitu na początku, gdzie ładowanie do wewnętrznego rejestru przetwornika odbywa się na zboczach narastających, licząc od momentu wystąpienia zbocza opadającego na selektorze. Wartość wyjściowa wystawiana jest w momencie wystąpienia zbocza narastającego na selektorze i jeżeli transfer jest dłuższy niż 16 bitów, dodatkowe wartości są ignorowane. Umożliwia to komunikację z przetwornikiem transferami dłuższymi niż 16 bitów. Przetwornik poza zasilaniem o wartości +5 V wymaga napięcia referencyjnego, które defniniuje wartości wyjściowe i zakres. Jako napięcie referencyjne wykorzystano również +5 V.

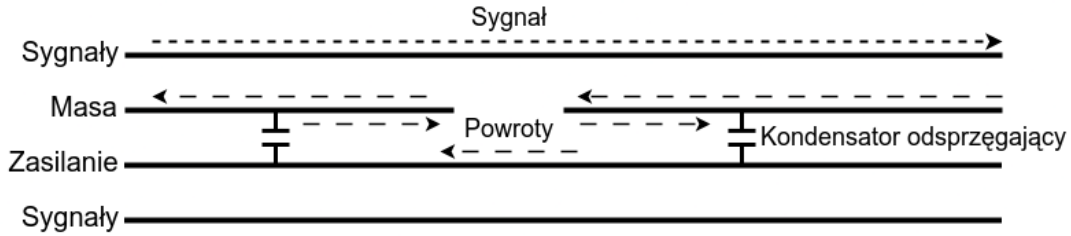
4.6 Dodatkowe kwestie sprzętowe

Projektując płytkę drukowaną autor starał podążać się szeregiem zasad projektowych [22], które opisano poniżej:

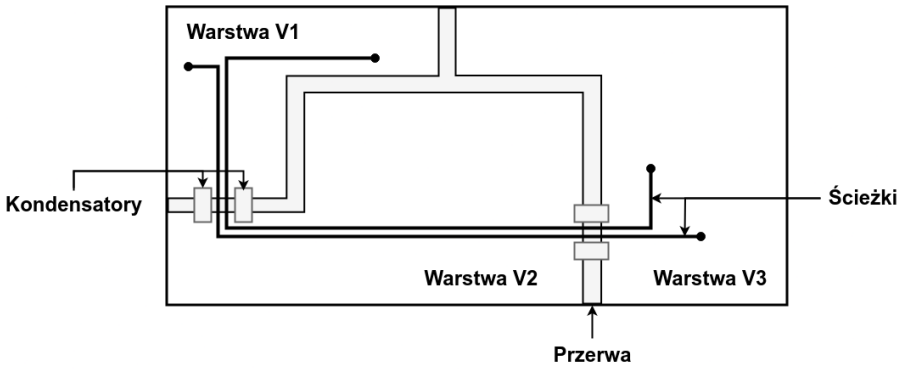
1. Minimalizacja długości scieżek — elementy powinny być umieszczone jak najbliżej siebie. Im dłuższa ścieżka, tym wpływ pojemności i indukcyjności pasożytniczych większy, co powoduje niechciane efekty filtracji. Szczególną uwagę należy zwrócić na sekcje krytyczne, takie jak sprzężenia zwrotne wzmacniaczy operacyjnych i wyjście przełączające przetwornic impulsowych.

2. Grubości ścieżek — im grubsza scieżka, tym jej impedancja mniejsza. Grubość należy dobierać na podstawie płynącego prądu, który zwykle większy jest dla ścieżek którymi poprowadzone jest zasilanie. W projekcie zastosowano 0.5 mm grubości na ścieżki sygnałowe i 1 mm grubości na ścieżki zasilania.

3. Kondensatory odsprzęgające — powinny być możliwie najbliżej docelowych układów. Jako, że mają zapewnić filtrację zasilania i lokalny bufor energii, im dalej od docelowego układu, tym swoje zadanie spełniają gorzej. Mogą również służyć jako mosty pomiędzy wartstwami i płaszczyznami, kiedy droga sygnału może wymagać ich zmiany, co przedstawiono na rysunkach 4.29 i 4.30:



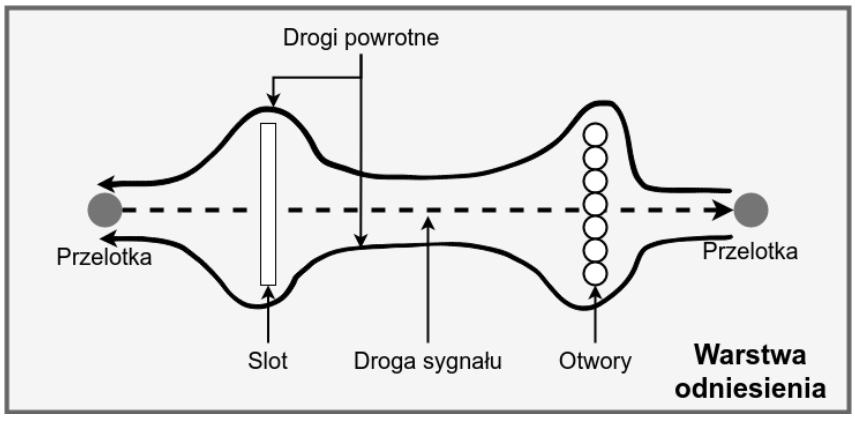
Rysunek 4.29: Zapewnienie powrotów przez kondensatory odsprzęgające



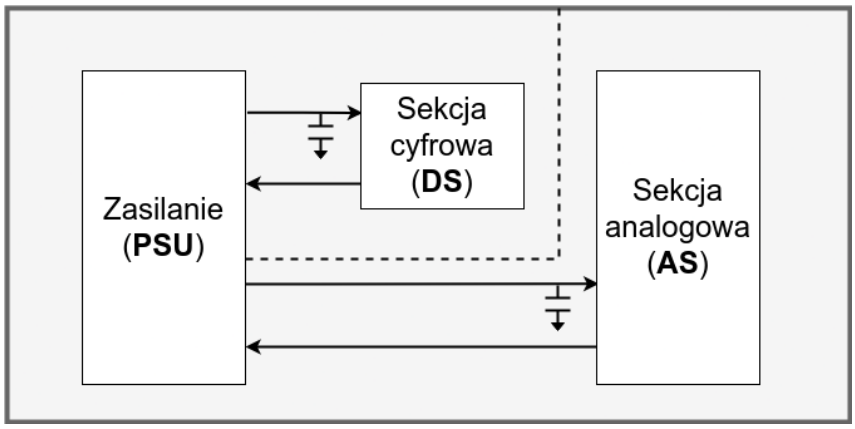
Rysunek 4.30: Zmiany warst przez kondensatory odsprzęgające

Omawianą sytuację w projekcie wykorzystano do zapewnienia częściowego powrotu sygnałów cyfrowych przez bliższą warstwę zasilania, co zaskutkowało krótszą drogą oraz stabilniejszą masą (brak spadków związanych ze szybkim przełączaniem sygnałów cyfrowych).

4. Powroty sygnałów i rozmieszczenie elementów — sygnały najczęściej, jeżeli pod lub nad jest warstwa odniesienia, jako drogę powrotną wybiorą tą bezpośrednio pod lub nad ścieżką sygnałową, ponieważ jest to dla nich droga o najmniejszej induktancji [22]. Mając to na uwadze, elementy i przelotki należy układać tak, aby nie zaburzać drogi powrotnej sygnałów. Duża ilość przelotek lub otworów (przykładowo na elementy THT) może skutkować zablokowaniem bezpośredniego powrotu, tworząc sloty. Dodatkowo, elementy należy ustawiać w sposób zapewniający unikanie warstwowego krzyżowania się sygnałów różnych sekcji i domen. Układając sekcje cyfrową należy zwrócić uwagę, czy sygnały powrotne, jeżeli odniesione są do tej samej płaszczyzny co sygnały analogowe, nie będą przechodzić bezpośrednio pod lub nad ścieżkami oraz układami analogowymi i odwrotnie. Szybkozmienne sygnały cyfrowe w powrocie mogą zakłócać sygnały analogowe. Opisywane sytuacje przedstawiono na rysunkach 4.31 i 4.32:



Rysunek 4.31: Problematyczne powroty - nieciągłość plaszczyzny odniesienia



Rysunek 4.32: Poprawne ułożenie sekcji

5. Rozmieszczenie warstw (ang. *stackup*) — dla płytek wielowarstwowych powinno być realizowane z myślą o optymalnych powrotach sygnałów i najniższej emisji, a grubości warstw dobierane na podstawie docelowej impedancji ścieżek. Przy projektowaniu rozmieszczenia należy kierować się następującymi zasadami [22]:

   (a) Warstwa sygnałowa powinna być zawsze bezpośrednio sąsiadująca z warstwą odniesienia.

   (b) Odległość między warstwą sygnałową i odpowiedającą jej warstwą odniesienia powinna być jak najmniejsza.

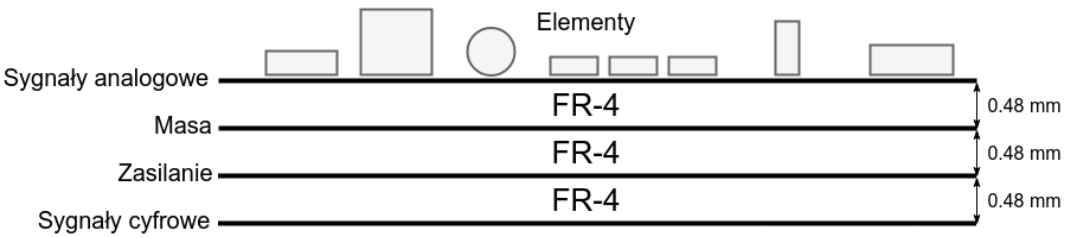
   (c) Warstwy odniesienia (zasilania i masa) powinny być jak nabliżej siebie.

   (d) Wysokoczęstotliwościowe sygnały powinny znajdować pomiędzy warstwami odniesienia.

   (e) Im więcej warstw masy, tym lepiej.

   (f) Kiedy sygnały krytyczne są prowadzone na więcej niż jednej wartswie, to nie powinny przechodzić przez więcej niż dwie bezpośrednio sąsiadujące.

   Większość płytek drukowanych nie może spełniać wszystkich sześciu warunków przedstawionych powyżej, dlatego sposób projektowania jest kompromisem. Przykładowo, projektując płytkę czterowarstwową, nie sposób spełnić dwóch pierwszych zasad jednocześnie. Wykorzystane w projekcie rozmieszczenie przedstawiono na rysunku 4.33:



Rysunek 4.33: Rozłożenie warstw

Rozmieszczenie 4.33 nie jest zbyt dobre, ponieważ każda warstwa odzielona jest od siebie o taką samą odległość. W takim przypadku ustatysfakcjonowana jest tylko pierwsza zasada i znacznie lepiej byłoby, gdyby warstwy sygnałowe umieścić bliżej ich warst odniesienia, tak, aby również spełnić trzecią.

6. Terminatory — należy stosować w celu kontroli impedancji ścieżek transmisyjnych. Dla szybkozmiennych sygnałów i długich ścieżek z powodu pojemności i idukcyjności pasożytniczych w torze mogą wystąpić niechcianie oscylacje, które należy wytłumić. Terminatory docelowo dobiera się na podstawie impedancji charakterystycznej ścieżki. Wykorzystane sygnały cyfrowe w projekcie, mimo że wolnozmienne częstotliwościowo, posiadają bardzo szybkie zbocze, więc wykorzystano terminatory szeregowe w postaci rezystorów o rezystancji 100 Ω bezpośrednio przy ich źródle.

7. Pola masowe — puste powierzchnie na warstwach sygnałowych należy wypełniać masą i łączyć przelotkami z warstwą masy (jeżeli jest). Przy układaniu przelotek należy uważać, żeby nie stworzyć odciętych od reszty powierzchni (slotów), powodujących problematyczne powroty, które przedstawiono na rysunku 4.31. Wypełnienie masą tworzy ekran jak i również może zapewniać dodatkowe drogi powrotne dla sygnałów.

Rozdział 5

Implementacja części programowej

5.1 Rozszerzenia czasu rzeczywistego

Spośród dostępnych rozszerzeń czasu rzeczywistego można wyróżnić trzy, najbardziej popularne i sprawdzone opcje:

1. **Zestaw łatek PREEMPT_RT** — zestaw modyfikacji jądra Linux mających na celu rozszerzenie standardowych funkcjonalności o możliwość realizacji zadań w czasie rzeczywistym.

2. **Xenomai** — środowisko programistyczne pracujące wspólnie z jądrem Linux, umożliwiające tworzenie zadań pracujących w czasie rzeczywistym z deterministycznym czasem realizacji równolegle z zadaniami jądra. Działanie opiera się na stworzeniu mikrojądra obok głównego, obsługującego jedynie krytyczne zadania.

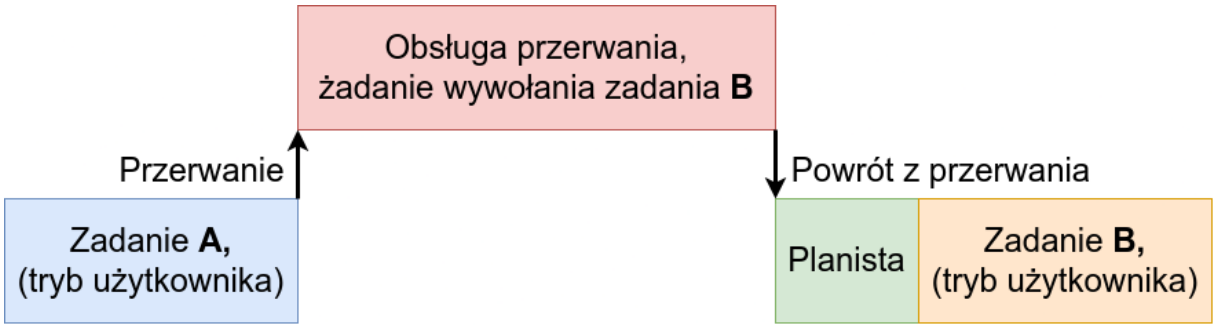
3. **RTAI** (ang. *Real-Time Application Interface*) - podobnie jak Xenomai, zapewnia zestaw interfejsów pozwalających na realizację krytycznych zadań z użyciem drugiego jądra, które w przeciwieństwie do środowiska Xenomai, nie jest realizowane w architekturze mikrojądra.

Dokonując analizy dostępnych opcji zdecydowano się na wykorzystanie zestawu łatek PREEMPT_RT. Xenomai i RTAI, mimo że bardzo cenione i często wykorzystywane nawet w rozwiązaniach komercyjnych, cechują się wysokim progiem wejścia i praca z nimi wymaga dużego nakładu czasowego.

5.1.1 Problemy jądra Linux w kontekście real-time

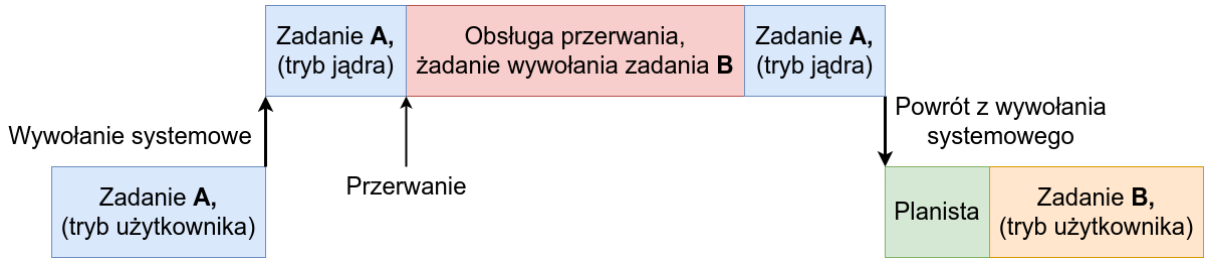
Podstawową kwestią, która dyskredytuje jądro Linux w zastosowaniach czasu rzeczywistego jest niedeterministyczny czas wykonania określonego zadania, który zależy od wielu czynników [32]. Jądro Linux jest wielozadaniowe i implementuje planistę (ang.

scheduler), którego zadaniem jest szeregowanie i obsłużenie zadań na podstawie ich priorytetu. Planista decyduje o przydziale czasu procesora na wykonanie zadań, które może przełączać w zależności od aktualnego zdażenia lub ustalonego planu. Standardowo, jądro Linux zapewnia mechanizm wywłaszczenia (ang. *preemption*) zadania z trybu użytkownia przez zadania o wyższym priorytecie lub przerwania sprzętowe. Przebieg wywłaszczenia przedstawiono na rysunku 5.1:



Rysunek 5.1: Przebieg wywłaszczenia zadania

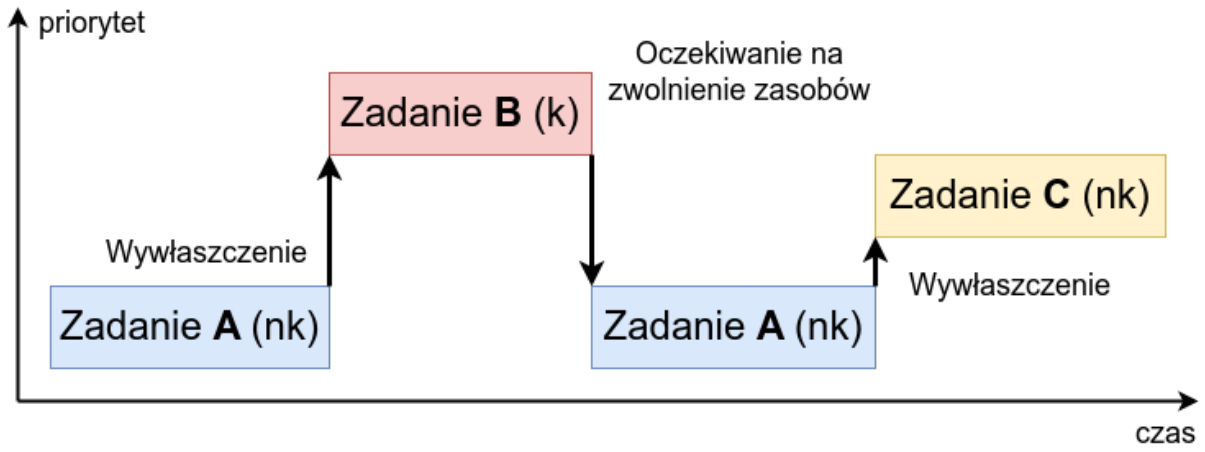
W przypadku jądra bez rozszerzeń czasu rzeczywistego, wywłaszczenie może zostać zablokowane przez mechanizm zapobiegania zakleszczeniom, zaimplemtowany w oparciu blokady wirujące (ang. *spinlock*). Z tego powodu nie trudno wyobrazić sobie sytuację, którą przedstawiono na rysunku 5.2



Rysunek 5.2: Przebieg wywłaszczenia zadania w sytuacji wywołania systemowego

Jeżeli przerwanie żądające wywołanie innego zadania nadejdzie w momencie wykonywania wywołania systemowego, to takie przerwanie nie zostanie obsłużone do momentu jego zakończenia. Oznacza to, że w niektórych sytuacjach nie sposób określić czasu potrzebnego na wykonanie zadania. Dodatkowo, każde zadanie w trybie użytkownika może zostać wywłaszczone przez przerwanie sprzętowe, w którego trakcie obsługi niemożliwa jest obsługa innych przerwań. Oznacza to, że małe ważne przerwania sprzętowe mogą wywłaszczyć krytyczne czasowo zadania czasu rzeczywistego.

Z realizacją wywłaszczeń realizowanych w standardowym systemie Linux wiąże się również problem nazywany inwersją priorytetów (ang. *priority inversion*). Opiera się on na sytuacji, kiedy niekrytyczne zadanie blokuje dostęp (ang. *mutex*) do zasobów zadaniu krytycznemu. Przebieg takiego problemu przedstawiono na rysunku 5.3:



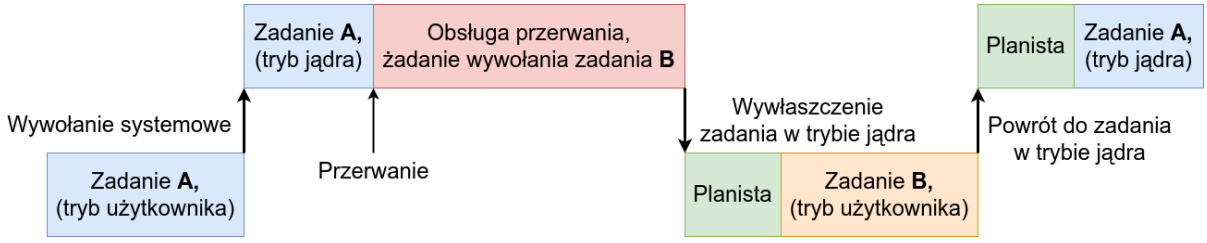
Rysunek 5.3: Przebieg problemu inwersji priorytetów

Krytyczne zadanie **B** musi uzyskać dostęp do zasobów blokowanych przez zadanie **A** o niższym priorytecie. W celu ich odblokowania, planista przełącza kontekst na zadanie **A**, lecz w czasie jego wykonywania zostaje ono wywłaszczone przez zadanie **C** o wyższym priorytecie.

5.1.2 Zestaw łatek PREEMPT_RT

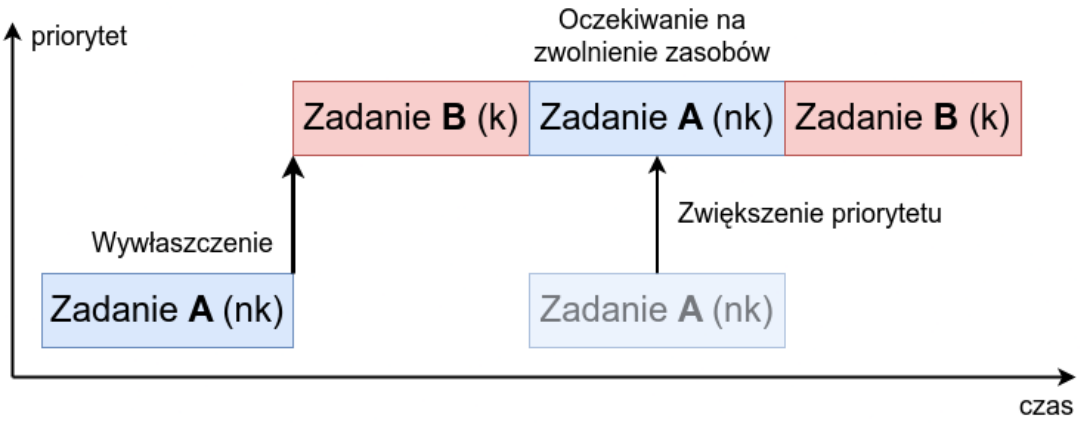
Samoistnie, jądro Linux nigdy nie będzie podstawą do stworzenia systemu operacyjnego czasu rzeczywistego, dlatego celem zestawu łatek PREEMPT_RT jest minimalizacja latencji. Latencja w kontekście obliczeń w czasie rzeczywistym definiowana jest jako czas liczony od momentu wystąpienia zdarzenia, którym może być rządanie realizacji zadania krytycznego, do momentu jego obsługi.

Zestaw łatek w odpowiedzi na problem 5.2 umożliwia wywłaszczenie wszystkich zadań, nawet tych z trybu jądra. Mając taką możliwość, problem przełączenia się na krytyczne zadanie w trakcie wykonywania zadania z trybu jądra może zostać rozwiązany jak przedstawiono na rysunku 5.4:



Rysunek 5.4: Wywłaszczenie zadania z przestrzeni jądra

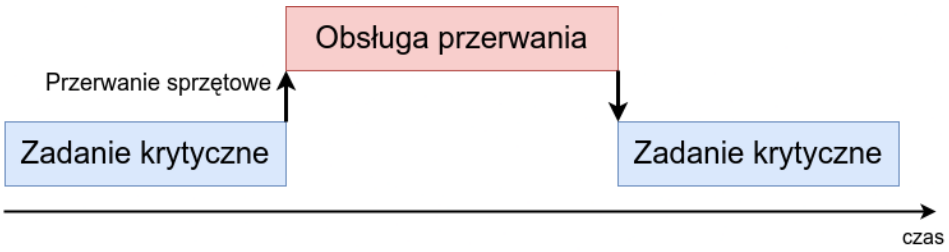
Problem inwersji priorytetów, przedstawiony na rysunku 5.3 zostaje rozwiązany w oparciu o metodę dziedziczenia priorytetów, którą przedstawiono na rysunku 5.5:



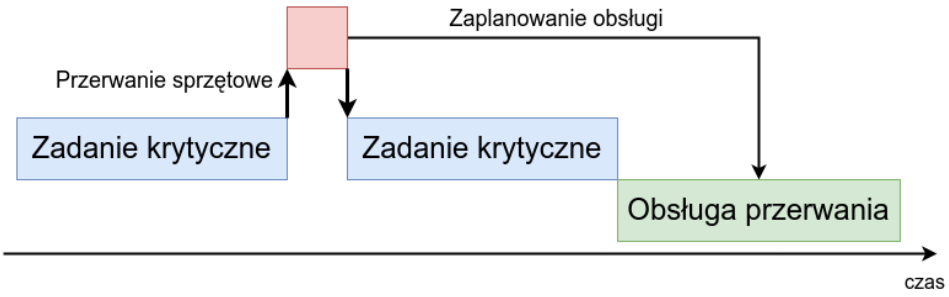
Rysunek 5.5: Dziedziczenie priorytetu

Czyli w przypadku chęci uzyskania dostępu przez zadanie o wyższym priorytecie **B** do zasobów blokowanych przez zadanie o niższym **A**, priorytet zadania **A** zostaje zwiększony, aby w trakcie odblokowywania zasobów nie nastąpiło jego wywłaszczenie.

Dodatkowo, wprawdzenie łatek umożliwia wątką realizację przerwań sprzętowych, które standardowo są niewywłaszczalne. Różnice w standardowej obsłudze przerwania, a wątkowej przedstawiono na rysunkach 5.6:



(a) Standarowa obsługa przerwania



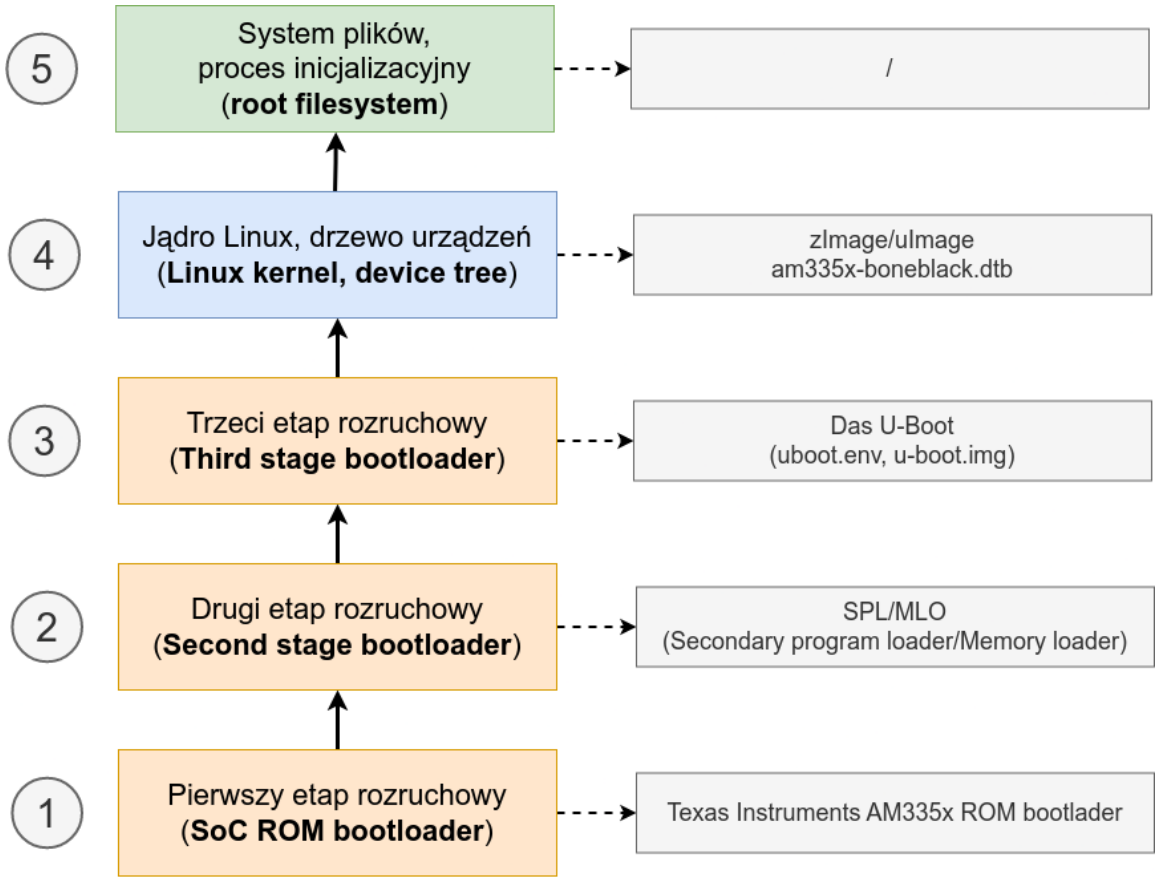
(b) Wątkowa realizacja przerwania sprzętowego

Rysunek 5.6: Porównanie obsługi przerwań sprzętowych

Jako, że PREEMPT_RT jest zestawem łatek jądra Linux, obsługa funkcjonalności, które oferuje, opiera się na wykorzystaniu zestawu funkcji udostępnianych przez POSIX. Konfiguracja krytycznego zadania i planisty odbywa się poprzez funkcje zdefiniowane w nagłówkach *pthread.h* i *sched.h*.

5.2 System operacyjny

Proces uruchomienia systemu operacyjnego GNU/Linux na wybranej platformie obliczeniowej przedstawiono na rysunku 5.7:



Rysunek 5.7: Proces uruchomienia systemu operacyjnego

Opis poszczególnych etapów przedstawiono poniżej:

1. Pierwszy etap rozruchowy (ang. *Boot ROM*) — jego zadanie polega na inicjalizacji podstawowych magistral i peryferiów niezbędnych do dalszych etapów rozruchu. Znajduje się z zafiksowanej przestrzeni adresowej z uprawnieniami tylko do odczytu, z której procesor bezpośrednio po uruchomieniu zaczyna wykonywać instrukcje. Po inicjalizacji, jego zadaniem jest załadowanie następnego w koleji programu rozruchowego (plik SPL lub MLO) z pamięci zewnętrznej (eMMC, karta microSD) lub przez określony protokół cyfrowy (SPI, UART) do wewnętrznej pamięci SRAM procesora. Jako, że program znajduje się w pamięci tylko do odczytu, nie podlega modyfikacji.

2. Drugi etap rozruchowy — kontynuuje inicjalizację peryferiów procesora oraz ładuje trzeci program rozruchowy (plik u-boot.img), wymagający najwięcej miejsca do pamięci DDR. Po załadowaniu, przekazuje mu pełną kontrolę.

3. Trzeci etap rozruchowy — ładuje jądro wraz z *device tree* do pamięci DDR oraz inicjalizuje peryferia, które mogą zostać do tego wykorzystane (I2C, NAND, FLASH, Ethernet). Proces ładowania należy odpowiednio skonfigurować, wskazując programowi miejsce, w który znajduje się obraz jądra oraz adres w pamięci, do którego ma je załadować. Konfiguracji można dokonywać statycznie, poprzez uprzednio przygotowany plik (*uEnv.txt*, *uboot.env*) lub dynamicznie, wpisując polecenia na etapie działania programu.

4. Jądro — wykonuje, planuje i zarządza procesami. Jest pośrednikiem między interakcjami ze sprzętem (peryferiami), kontrolując je za pośrednictwem sterowników, które mogą być osadzone statycznie lub ładowane dynamicznie w postaci modułów. Po uruchomieniu i przeprowadzeniu procesu inicjalizacji, włącza programy startowe i umożliwia użytkownikowi pracę. Zadania wykonywane przez jądro zawierają się w tak zwanej przetrzenii jądra (ang. *kernel space*), która tworzy osobną przestrzeń adresową niedostępną dla aplikacji użytkownika.

5. Główny system plików — zawiera zestaw plików i katalogów, niezbędnych do działania systemu. Z tego miejsca wychodzą pozostałe struktury katalogów, które definiują obszar roboczy użytkowników systemu wraz z aplikacjami, z których korzystają.

5.2.1 Środkowisko pracy

Tworząc oprogramowanie na platformę wbudowaną należy pamiętać, że architektura procesorów na takich urządzeniach różni się od tych typowo wykorzystywanych w komputerach personalnych. Chcać skompilować działający program pod daną architekturę, niezbędny będzie odpowiedni zestaw narzędzi (ang. *toolchain*), w którego skład wchodzi zestaw kompilatorów, konsolidatorów i bibliotek standardowych wykorzystywanego języka. Wykorzystana w projekcie platforma zaopatrzona jest w procesor ARM Cortex-A8, więc niezbędny jest zestaw narzędzi zdolnych budować programy pod taką architekturę. Taki proces przyjęło się nazywać kompilacją skrośną (ang. *cross-compilation*).

W projekcie wykorzystano gotowy zestaw narzędzi udostępnianych przez firmę ARM. W ich skład wchodzi odpowiednio przebudowany GCC oraz zestaw bibliotek standardowych języka C i C++. Wszystkie wyszczególnione na rysunku 5.7 konfigurowalne etapy ładowania systemu operacyjnego wraz z aplikacją działającą w trybie użytkownia są budowane z jego użyciem.

5.2.2 Przygotowanie nośnika

Docelowo, platforma BeagleBone Black ładuje zapisany na etapie produkcji system operacyjny z wewnętrznej pamięci eMMC. Chąc wgrać inny system, wymagane jest zapisanie niezbędnych plików na kartę SD i odpowiednie skonfigurowanie płytki, aby prze-

prowadzała z niej proces ładowania. Aby to wykonać, karta SD z plikami systemu operacyjnego musi być odpowiednio podzielona na partycje. Opis poszczególnych partycji przedstawiono poniżej:

1. Partycja rozruchowa — zawiera pliki drugiego i trzeciego programu rozruchowego. Wymagany jest system plików FAT32.

2. Partycja systemowa — zawiera pliki jądra systemu i drzewa urządzeń. Wymagany jest system plików EXT4[*nie jest wymagany, Linux obsługuje wiele systemów plików.*].

3. Partycja z głównym systemem plików — zawiera wypakowany główny system plików systemu. Wymagany jest system plików EXT4.

5.2.3 Programy rozruchowe

Pierwszy program rozruchowy jest wgrany przez producenta na etapie produkowania układu, więc jest to część systemu, na którą programista nie ma wpływu. Pozostałe dwa programy, czyli odpowiednio drugi i trzeci etap rozruchowy, muszą być odpowiednio skonfigurowane i umiejscowione na odpowiedniej partycji docelowego nośnika, aby proces uruchomienia systemu odbył się prawidłowo. Jako program rozruchowy zapewniający drugi i trzeci etap wybrano pakiet *Das U-Boot*, który stanowi podstawowe rozwiązanie dla wbudowanych systemów operacyjnych.

Proces budowania poszczególnych etapów przedstawionych na rysunku 5.7 zazwyczaj wygląda bardzo podobnie i można go podzielić na trzy główne kroki:

1. Pobranie kodów źródłowych z repozytorium twórców — jeżeli oprogramowanie udostępniane jest na licencjach otwartoźródłowych, to twórcy zwyczajowo magazynują wszystkie kody źródłowe na ogólnodostępnych w internecie repozytoriach.

2. Konfiguracja procesu budowania — może się różnić w zależności od wykorzystywanego przez program systemu budowania. Typowo, procesowi należy wskazać plik konfiguracyjny z charakterystyką używanej platformy.

3. Budowanie — proces kompilacji zakończony generacją odpowiedniego pliku wykonywalnego.

Platforma BeagleBone Black, jako że jest popularnym układem, posiada już odpowiednio przygotowane pliki konfiguracyjne dla drugiego etapu budowania, więc dodatkowa ingerencja w proces konfiguracji nie jest wymagana. Wynikiem pomyślnego zbudowania programu są pliki *MLO* i *u-boot.img*, będące odpowiednio drugim i trzecim programem rozruchowym. Po wgraniu plików na partycję rozruchową nośnika i uruchomieniu ładowania, trzeci program rozruchowy przedstawi się linią poleceń i wymagane będzie wpisanie kilku komend, informujących program, skąd i gdzie ma załadować pliki jądra systemu. Po zapisaniu konfiguracji, programy rozruchowe nie wymagają dodatkowych modyfikacji.

5.2.4 Jądro Linux

Proces budowania jądra Linux jest analogiczny jak proces budowania programów rozruchowych. Dodatkową opcją jest możliwość interaktywnej konfiguracji budowanego systemu w oparciu o narzędzie *menuconfig*, jednak domyślny plik konfiguracyjny, opisujący wybraną platformę na potrzeby realizowanego projektu jest wystarczający.

Jedyną zmianą, którą wprowadzono w kodach źródłowych jądra było dodanie prostego sterownika obsługującego zaprojektowaną płytkę drukowaną na bazie kodeku PCM5102A [25]. z najbardziej znaczących kwestii, sterownik zawiera deklarację struktury opisującą konfigurację płytki dla transferów wejściowych i wyjściowych (wspierane formaty, liczba kanałów) oraz funkcję, która umożliwia jego załadowanie i widoczność w systemie. W wykorzystanym rozwiązaniu napisany moduł bezpośrednio zastąpił sterownik kodeka PCM-5102A, zachowując początkową nazwę, więc dodatkowe modyfikacje w plikach konfiguracyjnych jądra nie były konieczne. Domyślnie, sterownik kodeka PCM5102A ładowany jest dynamicznie jako moduł, więc po zbudowaniu jądra wymagana jest ich instalacja w głównym systemie plików.

Proces zastosowania zestawu łatek PREEMPT_RT można wykonać manualnie, lub skorzystać z gotowej opcji przygotowanej przez twórców repozytorium składującego kody źródłowe jądra systemu Linux przystosowane do pracy na platformie BeagleBone Black. Wersje jądra z zastosowanym zestawem łatek PREEMPT_RT oznaczone są tagami **-rt-**, gdzie gwiazdki oznaczają numery wersji. Aby skorzystać z gotowo połatanego jądra, wystarczy ściągnąć odpowiednią wersję i przeprowadzić na niej znany już wcześniej proces budowania, którego wynikiem będzie plik *zImage*, następnie ładowany na partycję systemową nośnika.

5.2.5 *Device tree*

Wraz z plikami jądra, na partycji systemowej nośnika musi znaleźć się plik informujący system o rozmieszczeniu i konfiguracji urządzeń nie posiadających automatycznej metody wykrywania. Mechanizmem stosowanym do tego celu we wbudowanych systemach operacyjnych GNU/Linux jest *device tree*, z którym praca sprowadza się do modyfikacji następujących plików

- Pliki z kodem źródłowym (rozszerzenie *.dts*)
- Pliki nagłówkowe (rozszerzenie *.dtsi*)
- Pliki będące wynikiem budowania plików źródłowych (rozszerzenie *.dtb*), które finalnie muszą znaleźć się obok obrazu jądra na partycji systemowej nośnika.

Pliki źródłowe zawierają definicje i odwołania do wezłów (ang. *node*), które przypominają struktury języka C. Węzły posiadają zestaw właściwości (ang. *properites*) oraz

możliwe węzły potomne. Każdy węzeł posiada tylko jednego rodzica, za wyjątkiem węzła *root*, który jest węzłem nadrzędnym. Węzły mogą się do siebie nawzajem odwoływać jak i modyfikować swoją zawartość, dlatego z matematycznego punktu widzenia, drzewo urządzeń jest tak właściwie skierowanym i spójnym grafem acyklicznym.

Kody źródłowe jądra Linux posiadają gotowo przygotowane pliki źródłowe *.dts* dla większości obsługiwanych platform, w tym wykorzystanej. Sporządzona układ pomiarowy, obsługiwany przez interfejs McASP, nie posiada mechanizmu automatycznego wykrywania, dlatego niezbędna jest jego deklaracja wraz z niezbędną konfiguracją interfejsu McASP, który domyślnie nie jest zdefiniowany. W tym celu, napisany został dodatkowy plik nagłówkowy *.dtsi* dodany do głównego pliku źródłowego *.dts* z opisem niezbędnej konfiguracji, której zawartość jest następująca:

1. Zdefiniowanie pinów McASP — płytka BeagleBone Black wspiera wiele możliwych konfiguracji wyprowadzeń, które kontrolowane są przez wewnętrzny multilpekser. W ramach projektu wykorzystano McASP0, którego konfigurację wyprowadzeń przedstawiono w tabeli 5.1:

Nazwa	**Adres**	**Numer**	**Domyślny stan**	**Kierunek**	**Tryb multipleksera**
MCASP0_ACLKX	0x990	P9_31	1	OUT	0
MCASP0_FSX	0x994	P9_29	1	OUT	0
MCASP0_AXR0	0x998	P9_30	0	IN	0
MCASP0_AXR1	0x9A8	P9_41	0	IN	0
MCASP0_AXR2	0x99C	P9_28	1	OUT	2
MCASP0_AXR3	0x9A4	P9_27	1	OUT	2

Tablica 5.1: Konfiguracja wyprowadzeń McASP w drzewie urządzeń

2. Zdefiniowanie węzła McASP — stworzenie węzła *mcasp0* z odpowiednim zestawem wartości, z których można wyróżnić:

   - Odwołanie do wyprowadzeń (wartość *pinctrl*) — odwołanie do zadeklarowanych w tabeli 5.1.
   - Kierunki serializatorów (wartość *serial-dir*) — definiowane jako wartość 2 dla trybu wejściowego i wartość 1 dla trybu wyjściowego. Wartość przyjmuje postać tablicy, której indeksy reprezentują odpowiednie piny **AXR** (przykładowo **AXR0** dla indeksu zerowego).

3. Zdefiniowanie węzła zegara i przypisanie go sygnałowi **AHCLK** — jego domyślna częstotliwość wynosi 24.576 MHz i na jego podstawie generowane są pozostałe zegary interfejsu McASP (**ACLKX** i **FSX**)

4. Zdefiniowanie węzła opisującego sporzadzoną płytkę drukowaną - wymaga odwołania do zmodyfikowanego sterownika kodeka PCM5102A, którego opisano w sekcji jądra.

5. Zdefiniowanie węzłą urządzenia dźwiękowego — wykonana płytka drukowana będzie docelowo widoczna w systemie jako urządzenie dźwiękowe (karta dźwiękowa). Deklaracja urządzenia dźwiękowego wymaga zdefiniowanie zestawu wartości, które opisano poniżej:

   - Odniesienie do nadrzędnego sterownika karty — wykorzystano domyślny, czyli *simple-audio-card*.
   - Format transmisji — wykorzystano tryb I^2S w trybie wyrównania w lewo (ang. *Left justified* I^2S), który opisano w sekcji sprzętowej interfejsu cyfrowego.
   - Odniesienia do urządzenia traktowanego jako *master* i jako *slave* — jako *master* traktowany jest interfejs McASP0 (węzeł McASP) ponieważ zapewnia potrzebne zegary, a jako *slave* sporządzona płytka drukowana (węzeł płytki drukowanej).
   - Nazwa karty — nazwa, pod jaką urządzenie będzie widoczne w systemie.

Tak przygotowane źródła można poddać procesowi budowania i wynikowy plik umieścić obok obrazu jądra na partycji systemowej nośnika.

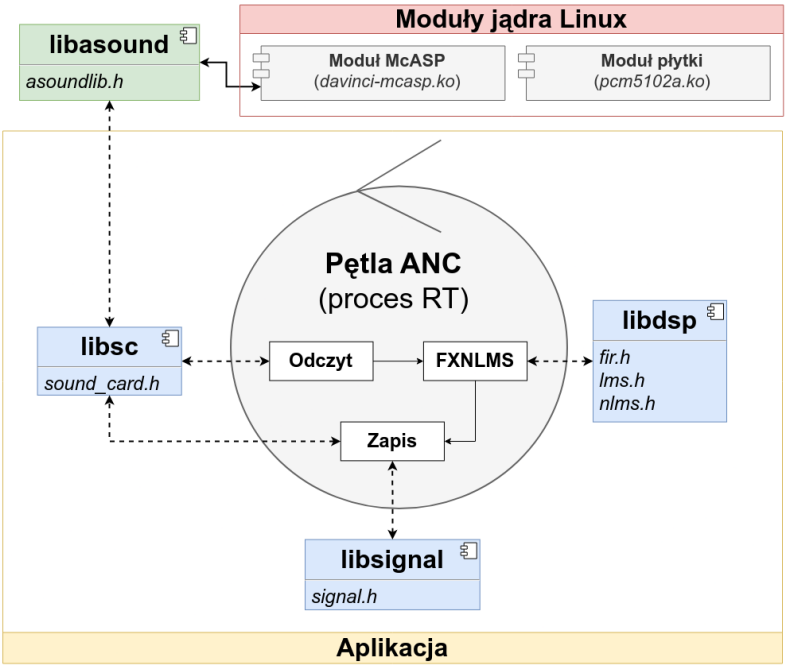
5.2.6 System plików

W celu generacji systemu plików wykorzystano narzędzie *debootstrap*, które umożliwia instalację systemu plików na bazie dystrybcji Debian we wskazanym katalogu. Narzędzie umożliwia zdefiniowanie architektury, na której docelowy system będzie działał oraz zestawu paczek, które zostaną pobrane i zainstalowane na końcu procesu instalacji.

Po wykonanym procesie instalacji, nowy powstały system można w wygodny sposób skonfigurować z wykorzystaniem narzędzia *chroot*, które umożliwia dynamiczną zmianę systemu, na którym aktualnie się pracuje. Dzięki temu, proces konfiguracji haseł, połączeń sieciowych oraz instalacja dodatkowych pakietów może odbyć się w prosty sposób z poziomu komputera personalnego, a nawet zostać zautomatyzowana skryptem powłoki.

5.3 Aplikacja

Diagram aplikacji przedstawiono na rysunku 5.8:



Rysunek 5.8: Diagram aplikacji

Kolorem niebieskim na diagramie 5.8 zaznaczono biblioteki wchodzące w bezpośredni skład aplikacji i są to moduły, które napisano samodzielnie. Można z nich wyróżnić:

- Bilbiotekę *libdsp* — tworzy ją implementacja filtracji cyfrowej opartej na modelu FIR (nagłówek *fir.h*) oraz algorytmów LMS (nagłówek *lms.h* i *nlms.h*).
- Bibliotekę *libsignal* — tworzy ją implementacja generatorów standardowych sygnałów wykorzystywanych do symulacji źródła hałasu, testowania i identyfikacji (sygnały periodyczne, szumy).
- Bibliotekę *libsc* — opakowuje funkcje zewnętrznej biblioteki *libasound* w celu uzyskania wyższego poziomu abstrakcji. Udostępnia funkcje pozwalające na interakcje z urządzeniami dźwiękowymi w systemie.

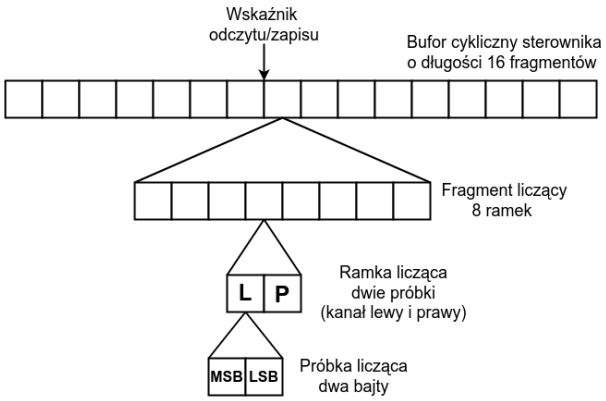
Kolorem zielonym zaznaczono moduł zewnętrzny, będącą w tym przypadku biblioteką *libasound*, udostępnianą przez środowisko ALSA, które jest wykorzystanym programowym systemem obsługi dźwięku. Na czerwono przedstawiono kontener z modułami jądra Linux, z którymi komunikuje się system ALSA.

5.4 System ALSA - biblioteka *libasound*

System ALSA (ang. *Advanced Linux Sound Architecture*) jest głównym środowiskiem kontrolującym urządzenia dźwiękowe w systemie operacyjnym GNU/Linux i jest bezpośrednim pośrednikiem pomiędzy aplikacjami działającymi w trybie użytkownika oraz

sterownikami działającymi w trybie jądra. Udostępnia zestaw narzędzi, pozwalających na interakcję z urządzeniami audio w systemie (przykładowo programy *aplay, arecord, alsamixer*) oraz bibliotekę *libasound*, która pozwala na wywoływanie funkcji systemu ALSA w aplikacji użytkownika. Z funkcji udostępnianych przez bibliotekę można wyróżnić kompleksowy zestaw akcji służycych konfiguracji danego urządzenia audio wraz z metodami pozwalającymi na odczyt i zapis, skutkującymi odpowiednio nagraniem oraz odtworzeniem danych z kanałów obsługiwanej karty.

Cyfrową reprezentacją dźwięku wykorzystywaną przez opisywany system jest modulacja pulsowo-kodowa (ang. *Pulse Code Modulation*, PCM), a próbki kodowane w ten sposób alokowane są w tworzonych na poziomie modułów jądra buforach cyklicznych, których długość można modyfikować korzystając z funkcji udostępnianych przez bibliotekę. Jeżeli bufor jest zapełniony, sterownik generuje przerwanie, którego obsługa oddelegowana jest do modułu DMA (ang. *Direct Memory Access*), przeprowadzającego nieblokujący transfer z bufora modułu do bufora aplikacji. Próbki w buforze sterownika szeregowane są w określony sposób, który przedstawiono na rysunku 5.9:



Rysunek 5.9: Podział bufora sterownika

Bufor składa się fragmentów (lub okresów, ang. *periods*), składających się z określonej ilości ramek. Ramki z kolei składają się próbek przypisanych określonym kanałom karty, których długość zdefiniowana jest przez przyjęty format. W trakcie działania programu w trybie użytkownika, próbki transportowane są z/do bufora sterownika do/z bufora aplikacji. W przypadku odczytywania danych może dojść do sytuacji, kiedy nie są one czytane z odpowiednią prędkością przez aplikację w trybie użytkownika, co skutkuje nadpisaniem najstarszych próbek w buforze cyklicznym sterownika. W przypadku zapisywania danych, może dojść do sytuacji odwrotnej, czyli bufor cykliczny całkowicie się opróżni, kiedy dane nie są odpowiednio szybko wysyłane. Opisywane sytuacje odpowiednio przyjęło się nazywać *buffer overrun* i *buffer underrun* i ich pojawienie się zazwyczaj skutkuje przerwami w nagraniach/odtworzeniach.

Bardzo znaczącą kwestią związaną z przetwarzaniem sygnałów audio jest latencja, która w omawianym kontekście informuje o czasie, który jest potrzebny na przetworzenie

danych w systemie od momentu ich wprowadzenia do momentu ich wypuszczenia. W nomenklaturze systemu ALSA, zdefiniowana jest zależnością:

$$T_L = \frac{n_f L_{fb}}{f_s L_{sb}}, \tag{5.1}$$

gdzie n_f to ilość fragmentów, L_{fb} to rozmiar fragmentu w bajtach, f_s to częstotliwość próbkowania, a L_{sb} to rozmiar próbki w bajtach. Jak można zauważyć, licznik równiania 5.1 odpowiada rozmiarowi bufora bajtach, czyli całe wyrażnienie odpowiada czasowi potrzebnemu na zbuforowanie danych. Przykładowo, chcąc odtworzyć dźwięk z szesnastobitowego systemu stereo (dwa kanały wyjściowe) z typową częstotliwością audio $f_s = 44.1$ kHz, ramka ma rozmiar czterech bajtów (dwa bajty na dwa kanały, $L_{sb} = 2$). Deklarując rozmiar fragmentu na 2048 ramek uzyskuje się rozmiar fragmentu $L_{fb} = 8192$ bajtów. Deklarując rozmiar bufora na dwa fragmenty $n_f = 2$, wynikowo otrzymuję się latencję $T_L = 93$ ms. Latencja dla stałego rozmiaru próbki jest więc odwrotnie proporcjonalna do częstotliwości próbkowania i proporcjonalna do rozmiaru bufora.

Ze zmniejszaniem latencji wiąże się szereg problemów. Deklarując coraz to mniejsze rozmiary fragmentów i większe częstotliwości próbkowania, zwiększa się ilość przerwań na transfery próbek, co powoduje zwiększenie zużycia procesora, zmniejszając responsywność pozostałych aplikacji działających w systemie. Dodatkowo, okno czasowe pozwalające na wykonanie obliczeń w pętli przetwarzania skraca się, ponieważ czas potrzebny na zbuforowanie fragmentu jest krótszy. Minimalny rozmiar fragmentu zdefiniowany jest w sterownikach jądra obsługujących interfejsy cyfrowe komunikujace się ze wspierającymi je kartami dźwiekowymi i dla McASP wynosi on **32 ramek**. Analogicznie zdefiniowane są dostępne częstotliwości próbkowania, które dla wykorzystywanego protokołu wynoszą **minimalnie 8 kHz i maksymalnie 192 kHz**. Projektując system ANC wymagana jest minimalizacja opóźnień w torach przetwarzania, dlatego w tym celu porządane jest, aby rozmiaru fragmentu i częstotliwości próbkowania dobrać w sposób zapewniający jak najniższą latencję, ale również pozwalający na zdążenie z wykonaniem przetwarzania.

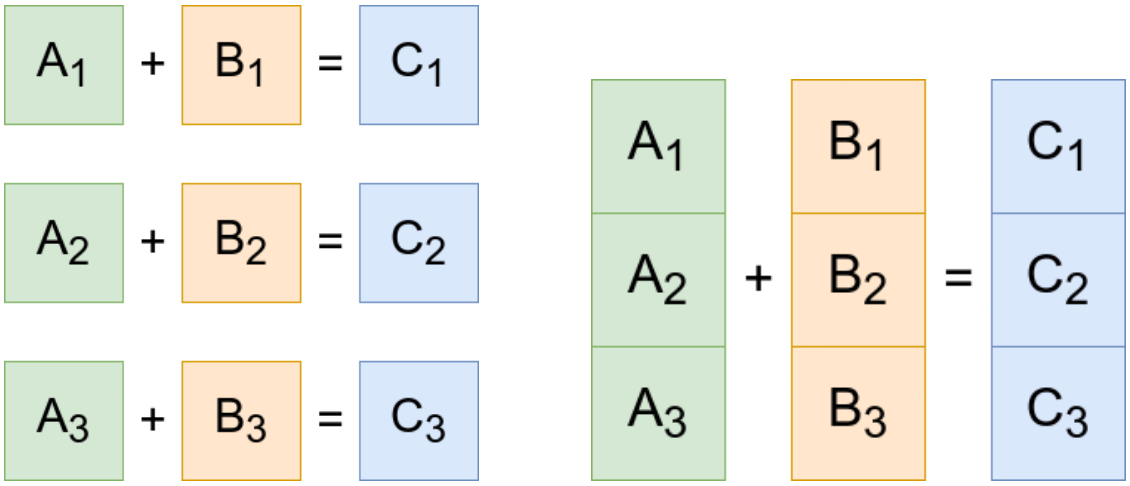
5.5 Biblioteki użytkownika

5.5.1 Biblioteka *libdsp*

Implementacja algorytmów cyfrowego przetwarzania sygnałów została wykonana w sposób pozwalający na wykorzystanie dostępnych na module Sitara AM3358 układów umożliwiajacych sprzętowe wykonanie operacji zmiennoprzecinkowych (*Floating Point Unit*, FPU), oraz przetwarzanie strumieniowe (*Single Instruction, Multiple Data*, SIMD). Wykorzystanie tych modułów wiąże się z deklaracją dodatkowych flag przekazywanych kompulatorowi i w przypadku pakietu GCC są to:

- *-ffast-math, -ftree-vectorize* — dla obliczeń wektorowych (wykorzystanie SIMD)
- *-mfpu=neon, -mhard-float* — dla sprzętowych obliczeń zmiennoprzecinkowych (FPU)

Poza flagami kompilacji, w przypadku wykorzystania SIMD ważne jest, aby program napisany był w sposób pozwalający kompilatorowi na jak najczęstsze wykorzystanie operacji wektorowych. Przykładowe porównanie operacji dodawania bez i z wykorzystaniem koprocesora SIMD przedstawiono na rysunkach 5.10:



(a) Operacja wektorowa bez SIMD (b) Operacja wektorowa z SIMD

Rysunek 5.10: Porównanie operacji wektorowych

Należy zwrócić uwagę, że wektorowo można wykonać jedynie niektóre proste operacje arytmetyczne i wymaganiem jest, aby operandy były umiejscowione w ciągłym obszarze pamięci. Możliwe do urównoleglenia pętle w procesie tłumaczenia kodu programu na kod maszynowy zostaną zastąpione pojedynczymi instrukcjami koprocesora, więc pisząc kod należy pamiętać, żeby projektowane pętle nie posiadały dodatkowej logiki, a w szczególności tej operującej na poszczególnych indeksach wykorzystanych tablic (wektorów). Chcąc zweryfikować, czy kompilator poprawnie przewidział zamiary programisty i wykorzystał instrukcje SIMD, należy prześledzić wygenerowany kod asemblera w poszukiwaniu tłumaczeń pętli i zweryfikować, czy faktycznie zostały zastąpione instrukacji SIMD, które w przypadku architektury NEON™ objawią się przykładowo jako rozkazy *vadd* dla dodawania i *vmult* dla mnożenia.

Napisana biblioteka została zweryfikowana pod kątem wykorzystania SIMD i rezultat przykładowego testu filtracji przedstawiono w tabeli 5.2:

Długość filtra	**Ilość iteracji**	**Czas, bez SIMD**	**Czas, z SIMD**
128	10e6	27.389 s	4.460 s

Tablica 5.2: Test wykorzystania SIMD

Jak można zauważyć bezpośrednio z 5.2, wykorzystanie SIMD w tym przypadku zwiększa prędkość obliczeń około sześciokrotnie.

5.5.2 Biblioteka *libsignal*

Poza standardowymi sygnałami okresowymi wykorzystywanymi do testów i symulacji, kwestią wartą poruszenia jest generator wykorzystywany w procesie identyfikacji toru wtórnego. Jako sygnał identyfikujący wybrano losowy szum biały, którego podstawowa implementacja może opierać się na wykorzystaniu funkcji *rand()* z biblioteki standardowej języka C. Funkcja *rand()* wykorzystywana jest do generacji liczby pseudolosowej z przedziału 0 do wartości RAND_MAX, która zależy od implementacji bilblioteki na daną architekturę [5].

Funkcja *rand()*, będąca implementacją generatora liczb pseudolosowych napisana jest w oparciu o liniowy generator kongruencjalny (ang. *Linear congruential generator*, LCG), który nie słynie z dobrej jakości w porównaniu z innymi generatorami liczb pseudolosowych [12]. Z głównych mankamentów generatora *rand()* można wyróżnić:

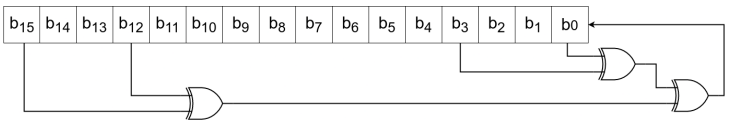
- **Przedział generacji jest zdefiniowany przez bibliotekę i tylko w tym przedziale liczby generowane są równomiernie** — w przypadku, kiedy RAND_MAX zdefiniowany jest jako maksymalny zakres 32 bitowy, a użytkownik chce wygenerować losową liczbę 16 bitową, to wygenerowana liczba pochodzi z rozkładu nierównomiernego [12].[*nie rozumiem tego, dokumentacja rand() od zawsze mówi jasno, że losuje wartość z 0 do RAND_MAX, i powszechnie znane jest, że jak chce się dostać inny zakres to trzeba podzielić i pomnożyć, i nie ma tego problemu, z dokładnością do nieliniwości tego przekształcenia*]

- **Stan generatora inicjalizowany jest globanie** — niemożliwym jest wykorzystanie kilku instancji silnika w jednej aplikacji, co znacząco komplikuje pracę, kiedy programista chce urównoleglić niektóre operacje [12][*od dawna jest też rand_r()*].

- **Jakość implementacji jest wątpliwa** — generatory LCG słyną z niskiej losowości bitów mniej znaczących w porównaniu z bardziej znaczącymi. Liczby generowane za pomocą tych generatorów mogą tworzyć widoczne struktury kratowe lub płaskie, co bardzo dobrze widać w przypadku generatora RANDU [12]. Dodatkowo sprawdzono, że typowe implementacji LCG zazwyczaj wypadają bardzo źle w testach losowości statystycznej [34] [*implementacja LCG zwykle jest przyzwoita, przynajmniej w zakresie założeń, no i w sensownych bibliotekach standardowych. A LCG nie są takie złe, tylko w przypadku rand() jest bardzo źle, bo te założenia przyjęto źle, dano tam bardzo mały stan. Ale też to były inne czasy, mało RAMu, a ten stan ma każdy proces w systemie. Tzn. tutaj nie problem implementacji, a interfejsu, że np. srand() przyjmuje seed jako unsigned int.*]

Mając na uwadze problemy wyszczególnione wyżej, w celu generacji sygnału identyfikującego posłużono się klasą generatorów będącą podzbiorem generatorów działających

na bazie rejestru przesuwnego z liniowym sprzężeniem zwrotnym (ang. *Linear Feedback Shift Register*, LFSR). W literaturze [11] [4] można spotkać się z sygnałami PRBS (ang. *Pseudorandom binary sequence*), które generowane są właśnie z ich wykorzystaniem. Przykładowy, ugólniony sposób działania generatorów tej klasy przedstawiono na rysunku 5.11:



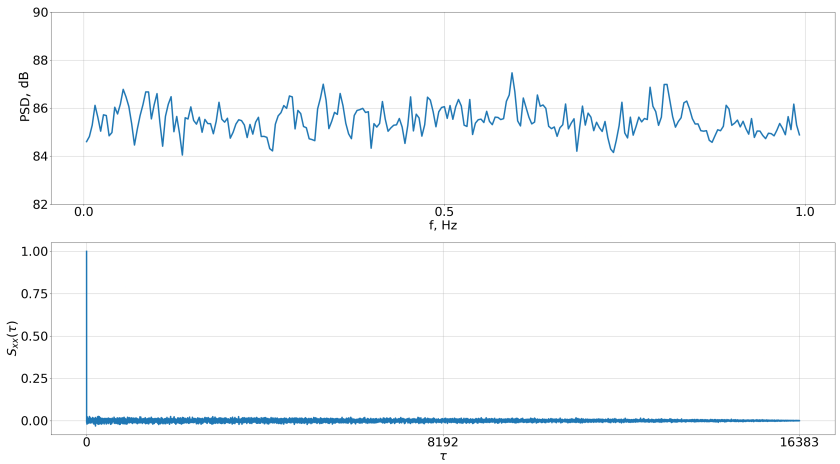
Rysunek 5.11: Przykładowy LFSR

Na jakość generatora działającego w oparciu LFSR wpływ mają numery bitów wykorzystywane do obliczenia nowego stanu, rodzaj i ilość operacji na nich przeprowadzanych oraz stan początkowy (ang. *seed*). Jak można zauważyć z 5.11, dla stanu początkowego 0 generator nie będzie działał poprawnie. Każdy LFSR jest związany z określonym wielomianem i maksymalny okres, w którym generowane liczby się nie duplikują, ogarniczony jest przez jego stopień i zdefiniowany jest przez liczbę bitów M:

$$T = 2^M - 1 \tag{5.2}$$

Istnieje wiele podzbiorów generatorów implementowanych na bazie LFSR. Jednym z nich są generatory oparte na kombinacji alternatywy rozłącznej, przesunięcia i/lub rotacji aktualnego stanu rejestru do obliczenia kolejnego. Omawiany podzbiór, ze względu na wykorzystywane operacje, przyjęło się nazywać generatorami *xorshift* i cieszą się one wysokim uznaniem w literaturze [34] ze względu na niskie zużycie zasobów do wykonania obliczeń oraz dużą losowość, weryfikowaną na podstawie testów statystycznych z biblioteki TestU01.

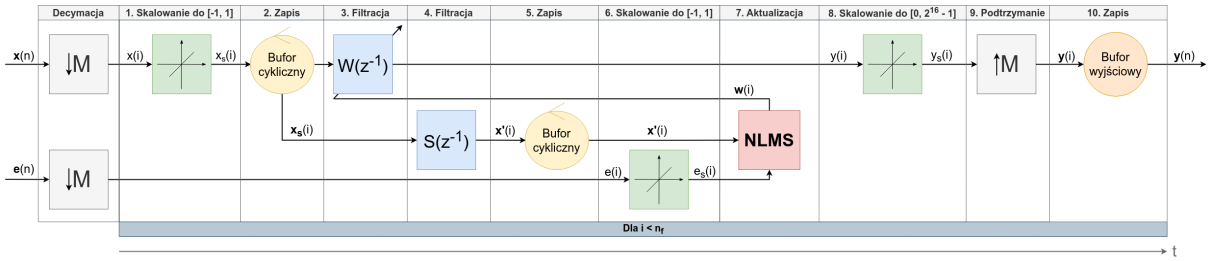
W bibliotece *libsignal* zdecydowano się na implementację 16 bitowego generatora *xoroshiro16++*, działającego z wykorzystaniem kombinacji alternatywy rozłącznej, rotacji, przesunięcie bitowego i rotacji, gdzie wyjście jest dodatkowo poddawane transformacji nieliniowej z wykorzystaniem dodawania. Generator przechodzi wszystkie testy statystycznej losowości ze zbioru TestU01, a jego znormalizowaną funkcję autokorelacji i gęstość widmową mocy obliczoną z wykorzystaniem metody Welch'a dla $N = 16382$ próbek przedstawiono na rysunku 5.12:



Rysunek 5.12: Funkcja autokorelacji i GWM sygnału z generatora *xoroshiro16++*

5.6 Główna pętla programowa

Przebieg operacji w głównej pętli programowej przedstawiono na schemacie 5.13:



Rysunek 5.13: Przebieg operacji - pętla ANC

Częstotliwość próbkowania układu, jak opisywano w sekcji dotyczącej systemu ALSA, jest ograniczona przez sterowniki jądra i zamyka się w przedziale od 8 do 192 kHz. Korzystając z założenia, że układ pracuje na hałasach do częstotliwości 500 Hz, będącej również częstotliwością graniczną filtrów sekcji analogowej, wejściowe dane można poddawać procesowi decymacji, otrzymując w ten sposób $f'_s = \frac{f_s}{M}$. Należy zwrócić uwagę, że częstotliwość próbkowania f'_s istnieje tylko w pętli przetwarzającej poszczególne próbki, a przetworniki pracują w dalszym ciągu na częstotliwości f_s. Decymacja pozwala na M krotne zmniejszenie opóźnienia w torze cyfrowym, wynikającego z buforowania fragmentów, więc przykładowo, dla najniższego możliwego rozmiaru fragmentu $n_{f,min} = 8$ ramek i dzielnika $M = 4$ otrzymuje się opóźnienie 2 ramek, czyli 64 bajtów w przypadku dwóch kanałów i próbek liczących 16 bitów.

Proces przetwarzania, przedstawiony na rysunku 5.13 można opisać następująco:

1. Skalowanie (normalizacja) próbki $x(i)$ do zakresu $[-1, 1]$.

2. Wpisanie próbki $x(i)$ do bufora cyklicznego.

3. Filtracja danych w buforze cyklicznym przez adaptacyjny filtr $W(z^{-1})$, otrzymując wynikowo sygnał $y(i)$, będacy sygnałem wyjściowym trafiający na głośnik.
4. Filtracja danych w buforze cyklicznym przez filtr $S(z^{-1})$ będący zidentyfikowaną odpowiedzią impulsową toru wtórnego i otrzymanie wynikowo próbki $x'(i)$.
5. Wpisanie próbki $x'(i)$ do bufora cyklicznego.
6. Skalowanie (normalizacja) próbki $e(i)$ do zakresu $[-1, 1]$.
7. Aktualizacja wag filtra $W(z^{-1})$ zgodnie z algorytmem NLMS.
8. Skalowanie próbki wyjściowej $y(i)$ do zakresu 16 bitowego.
9. M krotne podtrzymanie obliczonej próbki, w celu powrócenia do częstotliwości pracy przetworników.
10. Zapisanie M próbek $\mathbf{y}(i)$ do bufora wyjściowego, będacego obszarem w pamięci przekazywanym do sterownika jądra.

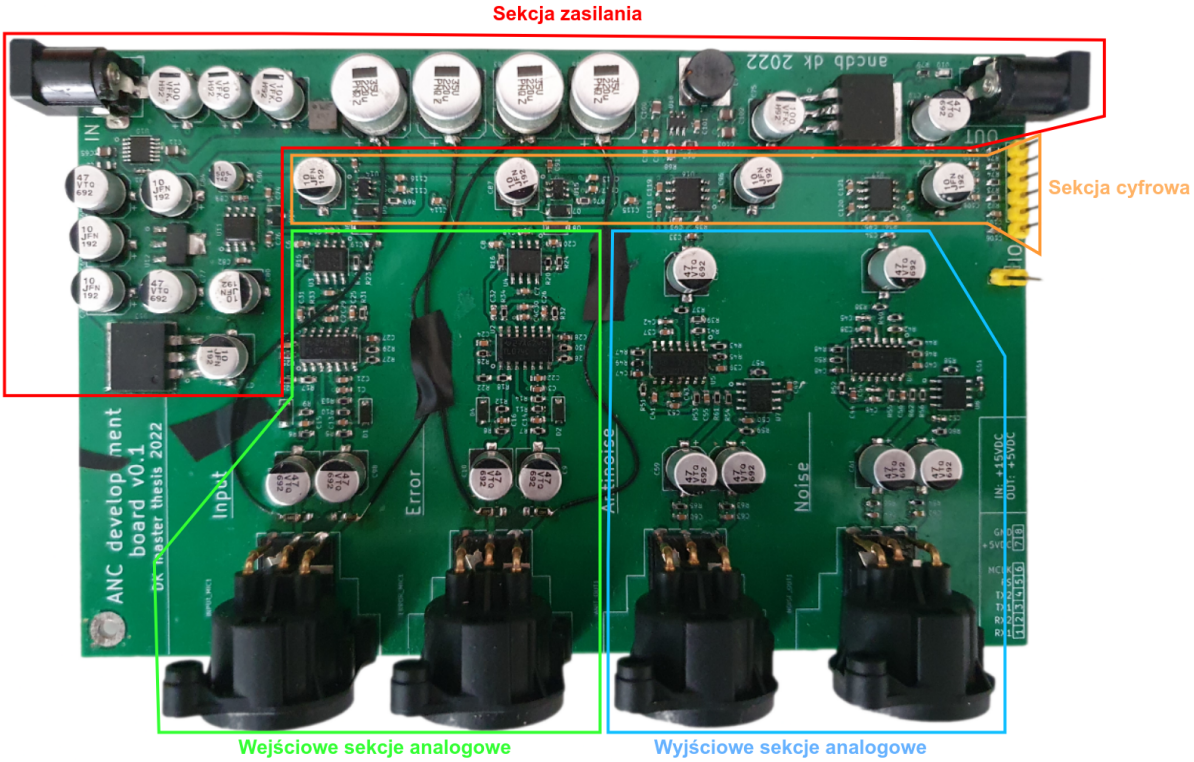
Schemat 5.13 został wykonany w sposób mający na celu wizualizację przebiegu wykonywania obliczeń, które następują po sobie w odpowiedniej kolejności. Na czas przetwarzania nałożone jest istotne ograniczenie wynikające ze sposobu realizacji transferów. W celu minimalizacji latencji, **długość bufora została określona na dwa fragmenty**, więc bezpośrednio po odczycie pierwszego (generacji przerwania przez sterownik DMA, informującego o odczytanym fragmencie), wszystkie obliczenia muszą zamknąć się w czasie potrzebnym do zbuforowania kolejnego. Jeżeli czas zostanie przekroczony, pętla przetwarzania zostanie przerwana stanem *buffer underrun*, informującym o zbyt późnym zapisie, ponieważ karty są ze sobą zsynchronizowane (operacje odczytu/zapisu buforów następują po sobie). Ograniczenia na czas przetwarzania implikują ograniczenia na złożoność obliczeń w pętli, przykładowo długości wykorzystanych filtrów. Przy odpowiednio długich filtrach może nastąpić sytuacja, w której obliczenia nie zamykają się w odpowiednim czasie. W takiej sytuacji, aby zapewnić ciągłość transferów, należy albo zmniejszyć długość filtrów, albo zwiększyć rozmiar fragmentu. Zwiekszenie rozmiaru fragmentu powoduje więcej czasu na przetwarzanie (dłuższy okres buforowania), ale również większe opóźnienia. Wpływ częstotliwości próbkowania, długości fragmentów oraz filtrów zostanie szerzej zbadany w sekcji badań, dlatego na tym etapie można stwierdzić, że parametrami konfiguracyjnymi deklarowanymi przez użytkownika oprogramowania jest częstotliwość próbkowania f_s, krotność decymacji M (należy pamietać, żeby była adekwatna do f_s) długość filtrów $W(z^{-1})$ i $S(z^{-1})$, długość fragmentu n_f oraz krok algorytmu NLMS α. Poza parametrami numerycznymi, konfiguracji również podlega poziom priorytetu oraz wytyczna planisty procesu.

Rozdział 6

Testy i badania

6.1 Weryfikacja części sprzętowej

Sporządzoną płytkę drukowaną przedstawiono na rysunku 6.1:



Rysunek 6.1: Sporządzona płytka drukowana

Widoczne na płyce przewody montażowe (kynary) są wynikiem błędu sprzętowego związanego z zasilaniem phantom, dla którego bez modyfikacji poprowadzono +12 V. Z racji, że wykorzystane mikrofony są wyspecyfikowane na zakres +15 do +48 V, zasilanie +15 V dostarczono przez przewody z dużych kondensatorów sekcji filtrującej główne zasilanie. Później zweryfikowano, że wybrane mikrofony również działają dobrze dla zasi-

lania +12 V, pomiomo, że nie są na niego wyspecyfikowane, więc finalnie przewody można usunąć i zostawić początkowe połączenia.

6.1.1 Sekcja zasilania

Pomiary sekcji zasilania przedstawiono w tabeli 6.1:

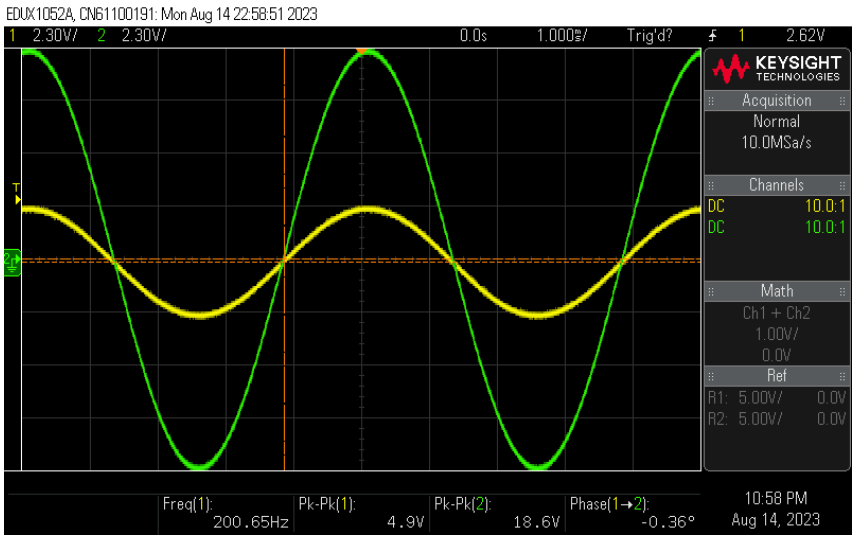
Nazwa	**Spodziewany wynik** [V]	**Pomiar** [V]
+15V	15 ± 0.75 (5%)	15.255
-15V	-15 ± 0.75 (5%)	**-13.784**
+12VA	12 ± 0.24 (2%)	11.993
-12VA	-12 ± 0.36 (3%)	-11.971
+5VD	5 ± 0.1 (2%)	5.037
+5VA	5 ± 0.002	4.998
+1.25_OFFSET	1.25 ± 0.0125 (1%)	1.252

Tablica 6.1: Pomiary sekcji zasilania

Jak można zauważyć, zasilanie −15 V jest poza spodziewaną skalą. Jest to problem, który nie wpływa drastycznie na pracę pozostałych sekcji, ponieważ zasilanie później jest filtrowane i stabilizowane przez liniowy regulator napięcia niskiego spadku. Pozostałe wartości mieszczą się w wyspecyfikowanych zakresach.

6.1.2 Wyjściowe sekcje analogowe i przetworniki

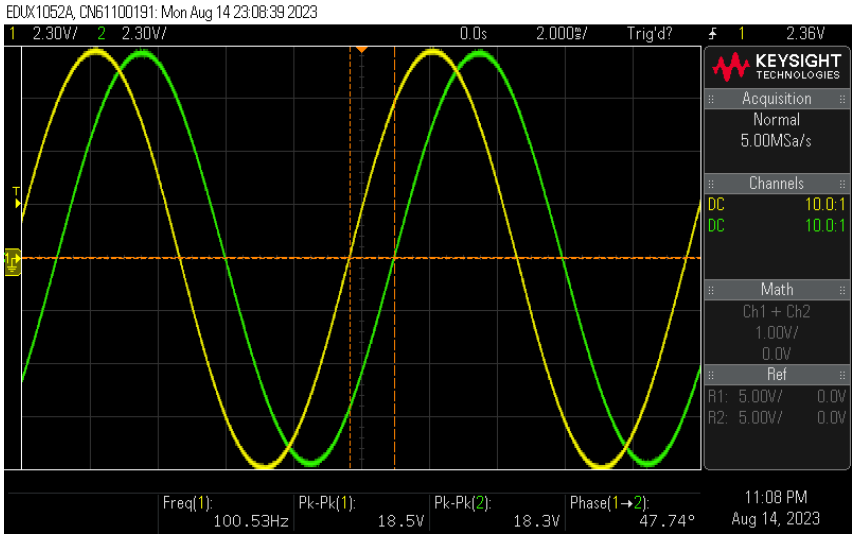
Pomiary wejść i wyjść wzmacniacza 4.21 dla wejściowego sinusa o częstotliwości 200 Hz i ampitudzie równej połowie zakresu przetwornika przedstawiono na rysunku 6.2:



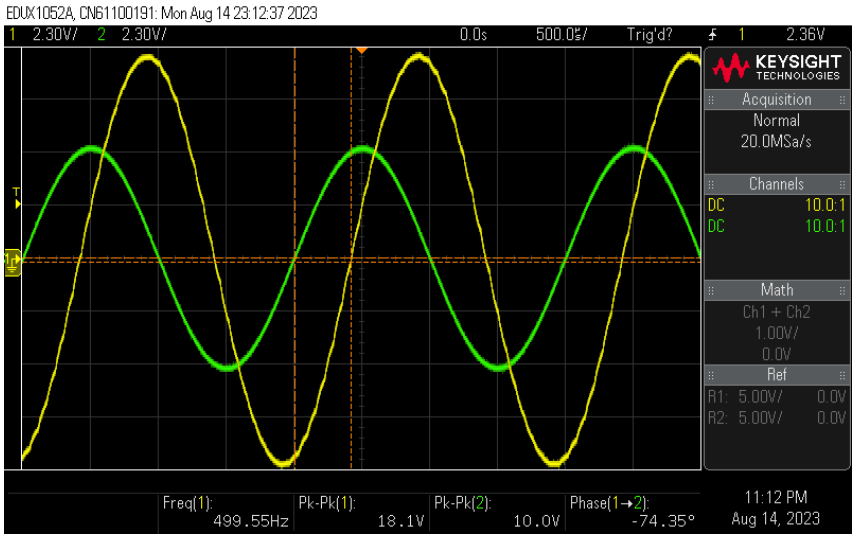
Rysunek 6.2: Pomiary wzmacniacza wyjściowej sekcji analogowej

Przebieg zielony przedstawia wyjście, a żółty wejście wzmacniacza 4.21. Jak można zauważyć, wzmocnienie dość dobrze pokrywa się z obliczoną teoretycznie wartością $k \approx 4$, nie powoduje saturacji i pozostawia bezpieczny zakres 2 V.

Pomiary wejść i wyjść filtra 4.15 dla wejściowych sinusów o częstotliwościach 100 Hz i 500 Hz oraz amplitudach równych połowie zakresu przetwornika przedstawiono na rysunkach 6.3:



(a) Przebiegi dla częstotliwości 100 Hz

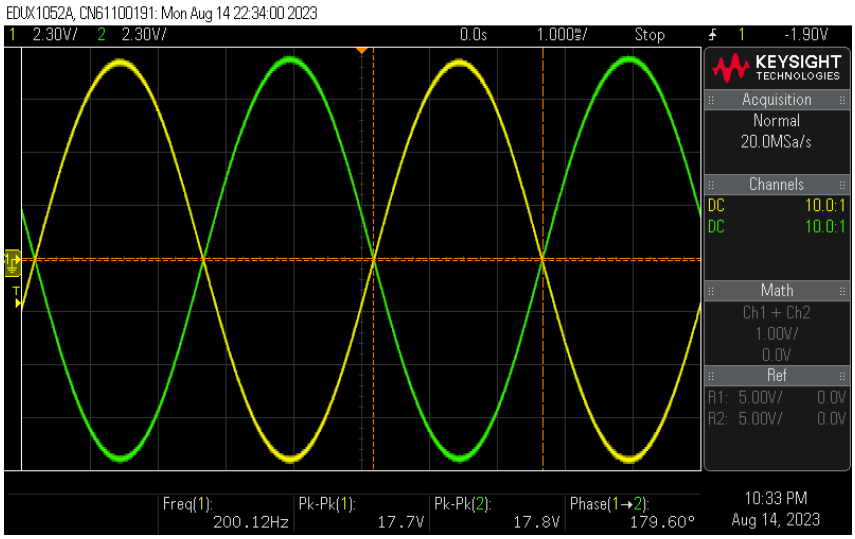


(b) Przebiegi dla częstotliwości 500 Hz

Rysunek 6.3: Pomiary filtra wyjściowej sekcji analogowej

Przebiegi żółte są wyjściami wzmacniacza 4.21, a przebiegi zielone wyjściami filtra 4.15. Dla sygnału o częstotliwości 500 Hz, która jest częstotliwością graniczną filtru 4.15 można zaobserwować tłumienie w okolicach 5 dB. Oznacza to, że pasmo zaporowe zaczyna się wcześniej niż obliczono, co może być spowodowane rzeczywistymi wartościami elementów wraz z ich niedokładnościami. Bliższe pasmo zaporowe filtrów sekcji wyjściowych nie powoduje problemów, ponieważ sygnały do 500 Hz w dalszym ciągu są użyteczne.

Wyjście różnicowe sekcji analogowej dla sygnału sinusoidalnego o częsotliwości 200 Hz i pełnego zakresu przetwornika cyfrowo-analogowego przedstawiono na rysunku 6.4:

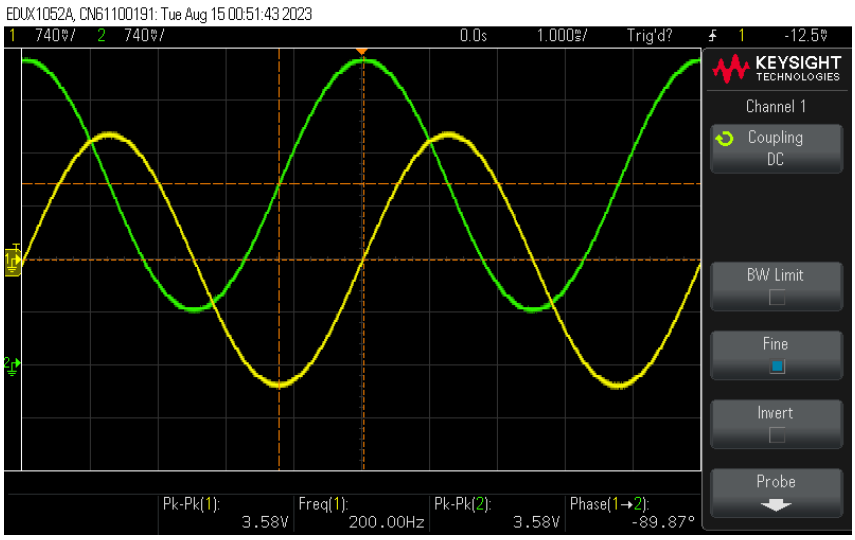


Rysunek 6.4: Wyjście różnicowe wyjściowej sekcji analogowej

Sygnały są względem siebie dokładnie odwrócone, a ich amplituda jest jednakowa, więc sekcja spełnia swoje zadanie tak jak w teorii.

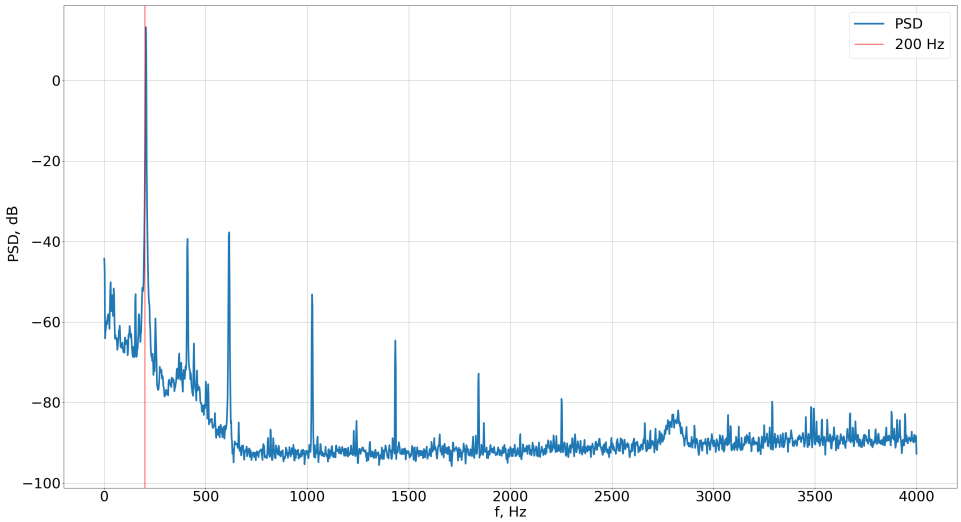
6.1.3 Wejściowe sekcje analogowe i przetworniki

Pomiary wejść i wyjść pełnego toru dla wejściowego sinusa o częstotliwości 200 Hz i natężeniu około 85 dB SPL przedstawiono na rysunku 6.5. Dźwięk odtworzono z głośnika wykorzystywanego w systemie, a mikrofon ustawiono bezpośrednio przy membranie w cichym pokoju.



Rysunek 6.5: Pomiary wejściowego toru analogowego dla $f_s = 200$ Hz

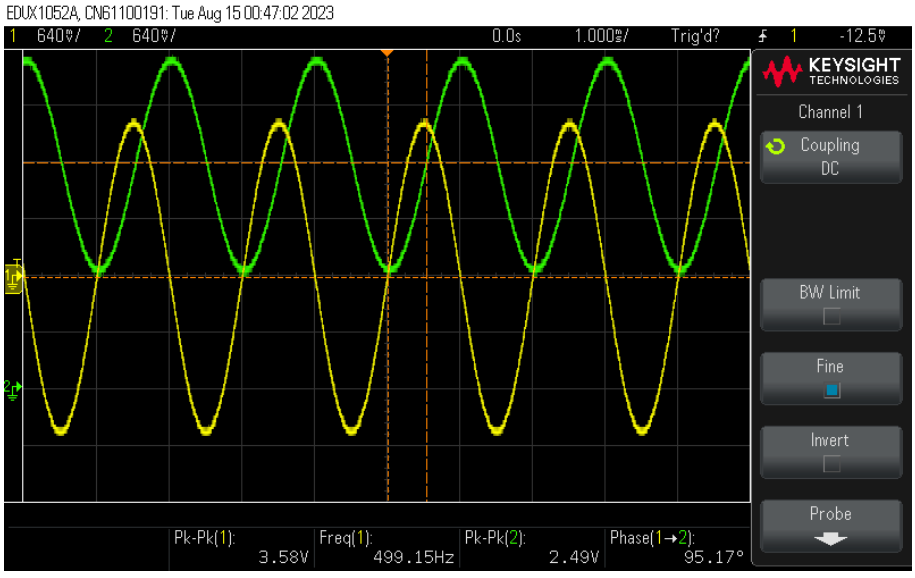
Przebieg zielony przedstawia wejście przetwornika analogowo-cyfrowego, a żółty wyjście wzmacniacza 4.11. Sygnał na wejściu przetwornika jest odpowiednio przesunięty o dodaną składową stałą, a amplitudy obu sygnałów są takie same. Gęstość widmową mocy obliczoną metodą Welcha z oknem Hanna dla $f_s = 8$ kHz przedstawiono na rysunku 6.6:



Rysunek 6.6: Gęstość widmowa mocy pomiaru dla $f_s = 200$ Hz

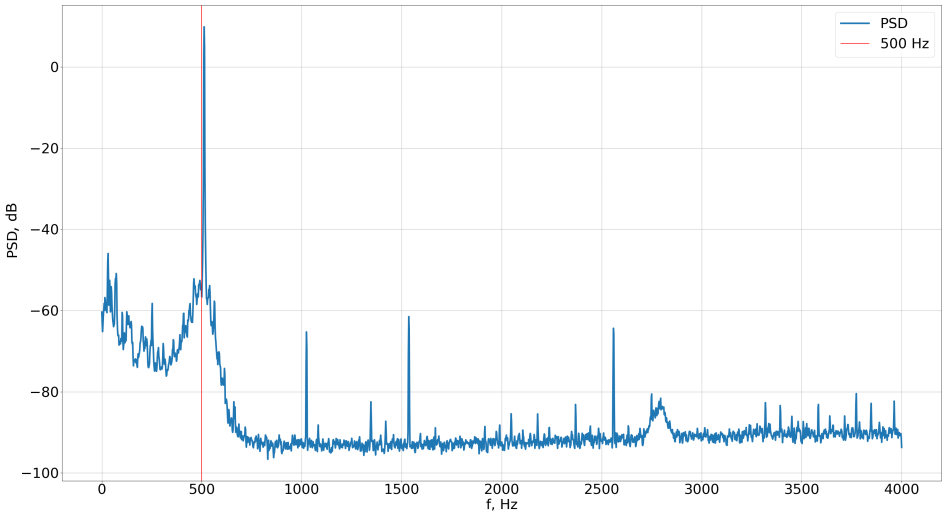
Na przebiegu 6.6 dobrze widać wpływ filtra 4.15, który mocno wytłumia zakres powyżej 500 Hz. Pik nagranego sinusa jest dużej mocy, widać jego harmoniczne a tło szumu akceptowalne.

Pomiary wejść i wyjść pełnego toru dla wejściowego sinusa o częstotliwości 500 Hz i natężeniu około 85 dB SPL przedstawiono na rysunku 6.7. Dźwięk odtworzono analogicznie jak w poprzednim pomiarze.



Rysunek 6.7: Pomiary wejściowego toru analogowego dla $f_s = 500$ Hz

Przebieg zielony przedstawia wejście przetwornika analogowo-cyfrowego, a żółty wyjście wzmacniacza 4.11. Poza obserwacjami analogicznymi jak w punkcie wyżej, można zaobserwować, że filtr 4.15 w tej sekcji zachowuje się zgodniej z obliczeniami. Tłumienie dla sygnału o częstotliwości granicznej wynosi około 3.22 dB. Gęstość widmową mocy obliczoną metodą Welcha z oknem Hanna dla $f_s = 8$ kHz przedstawiono na rysunku 6.8:



Rysunek 6.8: Gęstość widmowa mocy pomiaru dla $f_s = 500$ Hz

Nagrany sinus jest mniejszej mocy niż na 6.8, a jego najbliższe harmoniczne mniej widoczne, co może być spodowowane kombinacją filtracji i jakością nagrania.

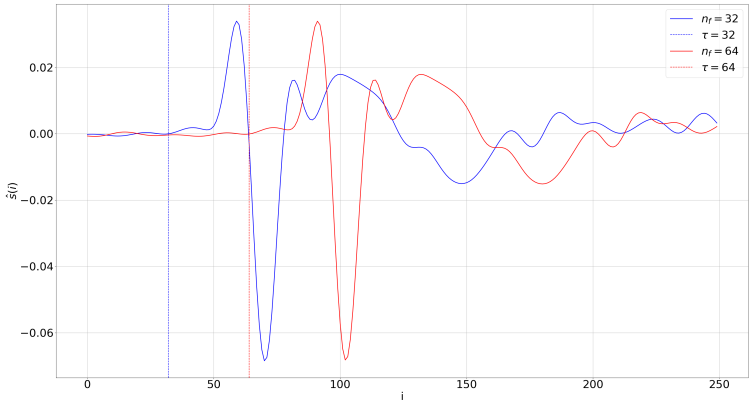
6.2 Weryfikacja części programowej

Sporządzony układ, stanowiący wykorzystane do badań stanowisko pomiarowe przedstawiono na rysunku 6.9:

Rysunek 6.9: Stanowisko pomiarowe

6.2.1 Identyfikacja toru wtórnego

Aby zweryfikować wpływ rozmiaru fragmentów na długość opóźnienia, identyfikację przeprowadzono dla $n_f = 32$ i $n_f = 64$. Sygnały wejściowe wygenerowano za pomocą generatora *xoroshiro16++*, a eksperymenty przeprowadzano za pomocą metody korelacyjnej w pięciosekundowym oknie. Zidentyfikowane odpowiedzi impulsowe ograniczone do dwustu wartości przedstawiono na rysunku 6.10:



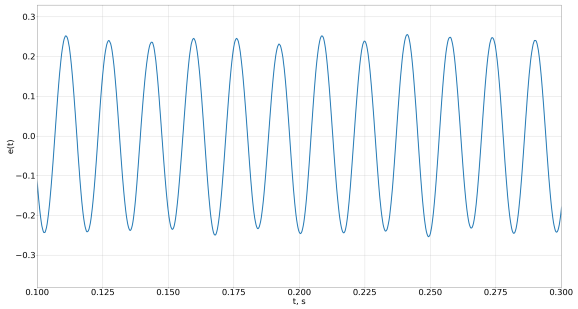
Rysunek 6.10: Indentyfikacja toru wtórnego — wpływ n_f

Jak można zauważyć z 6.10, buforowanie wpływa na zidentyfikowaną odpowiedź impulsową, opóźniając ją o ilość próbek równą długości fragmentu. Poza opóźnieniem wynikającym z buforowania, na przebiegu odpowiedzi impulsowej można zaobserwować wpływ torów analogowych, objawiający się dodatkowym opóźnieniem, wynikającym głównie z filtracji.

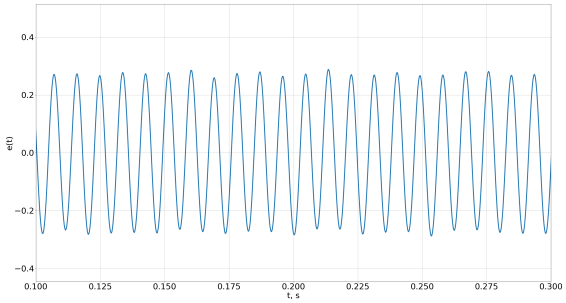
6.2.2 Badania procesu ANC - wpływ parametrów

Pomiary przeprowadzono na trzech sygnałach wejściowych zdefiniowanych poniżej:

1. Sygnały sinusoidalne o częstotliwościach $f = 120$ Hz i $f = 220$ Hz, których nagrania mikrofonem błędu przedstawiono na rysunkach 6.11:



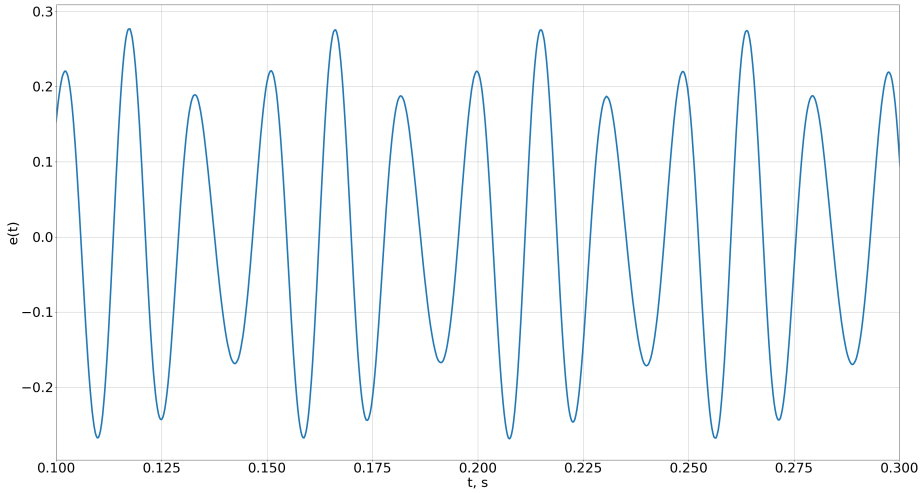
(a) Sinusoida o $f = 120$ Hz



(b) Sinusoida o $f = 220$ Hz

Rysunek 6.11: Przebiegi referencyjne

2. Sumy dwóch sinusoid o częstotliwościach $f_0 = 120$ Hz i $f_1 = 160$ Hz, których nagranie mikrofonem błędu przedstawiono na rysunku 6.12:



Rysunek 6.12: Suma sinusoid o $f_0 = 120$ Hz i $f_1 = 160$ Hz

Dla każdego sygnału starano utrzymać natężenie w okolicach 80 dB SPL, badania przeprowadzano w oknie czasowym o długości pięciu sekund, a wyniki poddawno skalowaniu do zakresu -1 do 1. Z parametrów poddawanych ocenie wyróżniono:

1. Wielkość kroku adaptacji α algorytmu NLMS
2. Długość filtra $W(z^{-1})$
3. Długość fragmentu n_f
4. Częstotliwość próbkowania f'_s
5. Poziom priorytetu P i wytyczna planisty

Oceny wpływów poszczególnych parametrów realizowano dla stałych wartości pozostałych. Zestaw parametrów stanowiących punkt referencjny do reszty badań przedstawiono w tabeli 6.2

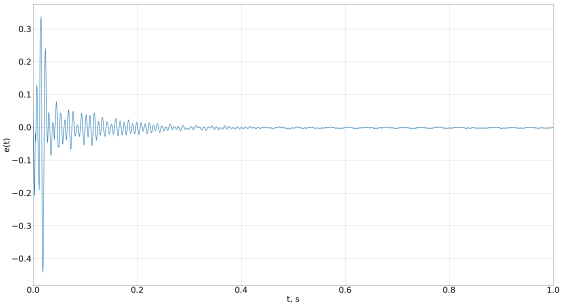
f'_s, Hz	n_f, ramki	α	L	P	Wytyczna planisty
2000	32	0.1	128	99	SCHED_RR

Tablica 6.2: Wybrany zestaw parametrów referencyjnych

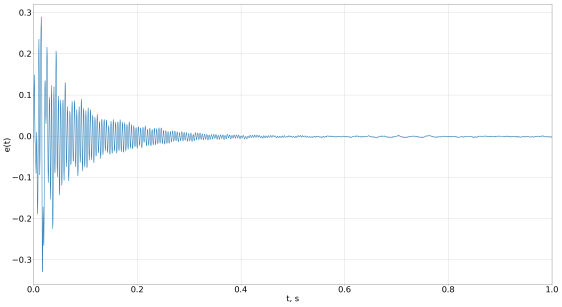
Częstotliwość f'_s w tabeli 6.2 jest częstotliwością uzyskiwaną po przeprowadzeniu decymacji z $M = 4$ i tak określony dzielnik pozostał niezmienny w trakcie badań. W celu dobrania odpowiedniego kroku, wstępnie przeprowadzono serię kilku pomiarów, mających na celu wyłonienie wartości α, dla której układ zachowuje się najzgodniej z oczekiwaniami. W

ich trakcie zauważono, że dla kroku $\alpha > 0.1$ system zaczyna działać niepoprawnie, co objawia się kompletnym brakiem tłumienia i generacją sygnału wyjściowego, powodującego sumarycznie większy hałas, niż w przypadku wyłączonego układu. W związku z tym, krok $\alpha = 0.1$ przyjęto jako granicę, stanowiącą parametr referencyjny oraz ogaraniczenie dla reszty badań.

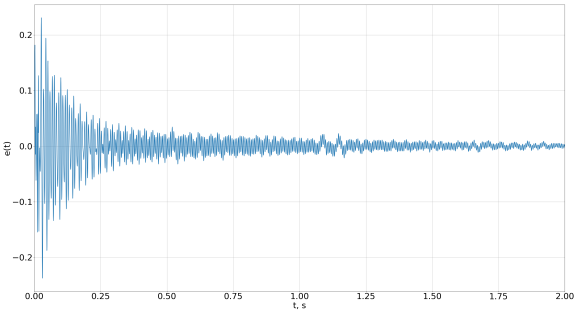
Aby uniknąć duplikacji wykresów, dla parametrów zdefiniowanych w 6.2 uzyskane przebiegi błędu $e(n)$ przedstawiono na rysunkach 6.13.



(a) Przebieg $e(n)$ dla 1 sygnału testowego



(b) Przebieg $e(n)$ dla 2 sygnału testowego



(c) Przebieg $e(n)$ dla 3 sygnału testowego

Rysunek 6.13: Przebiegi $e(n)$ dla sygnałów testowych

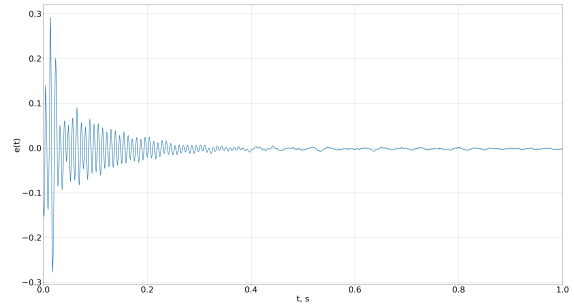
Jak można wywnioskować z rysunków 6.13, sinusoidy są idealnie tłumione, a czas zbieżności można oszacować na około 0.4 s w obu przypadkach. Dlu sumy sinusów algorytm zbiega się zdecydowanie dłużej i czas, do którego błąd osiąga wartości bardzo bliskie zeru, można oszacować na około 2 s, a tłumienie, realizowane przez system jest wyczuwalne przez słuchacza, nawet kiedy nie stoi bezpośrednio przy mikrofonie błędu.

Wpływ wielkości kroku

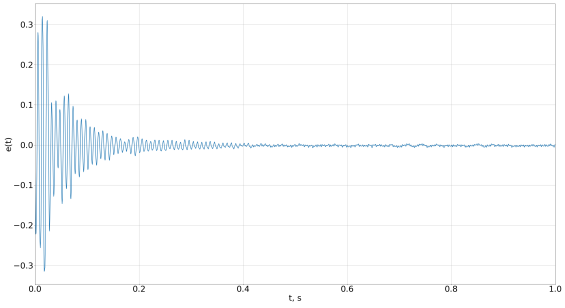
Do badania wpływu wielkości kroku α przyjęto zestaw parametrów referencyjnych zgodnie z tabelą 6.2, a krok algorytmu NLMS α dobierano zgodnie z zależnością:

$$\alpha(i) = \frac{i}{50}, \qquad \text{dla } i = 1, 2, 3, 4. \tag{6.1}$$

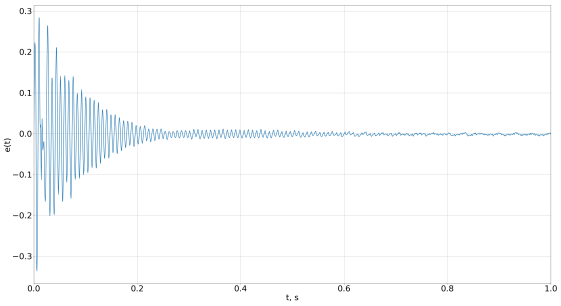
Rezultaty dla pierwszego sygnału testowego przedstawiono na rysunku 6.14:



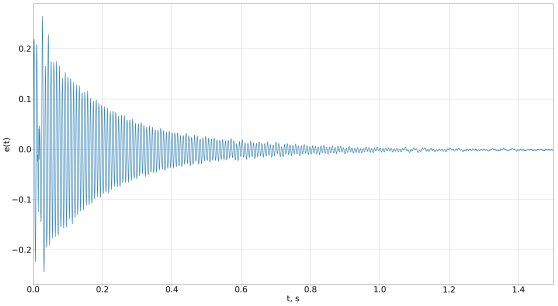
(a) $\alpha = 0.08$



(b) $\alpha = 0.06$



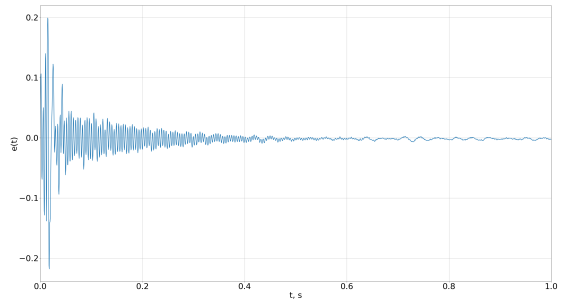
(c) $\alpha = 0.04$



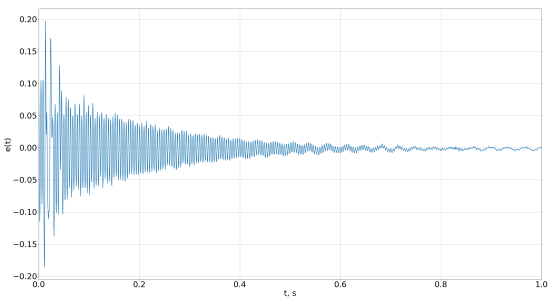
(d) $\alpha = 0.02$

Rysunek 6.14: Badania wpływu rozmiaru kroku α — sygnał 1

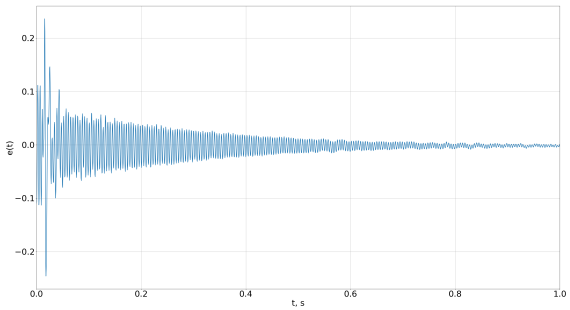
Rezultaty dla drugiego sygnału testowego przedstawiono na rysunku 6.15:



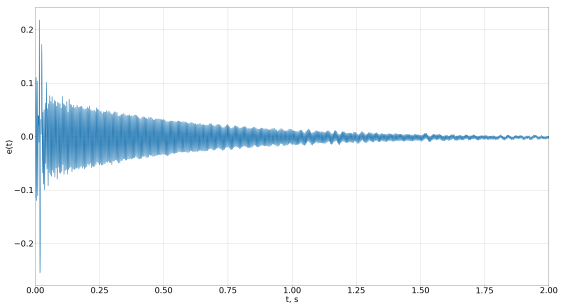
(a) $\alpha = 0.08$



(b) $\alpha = 0.06$



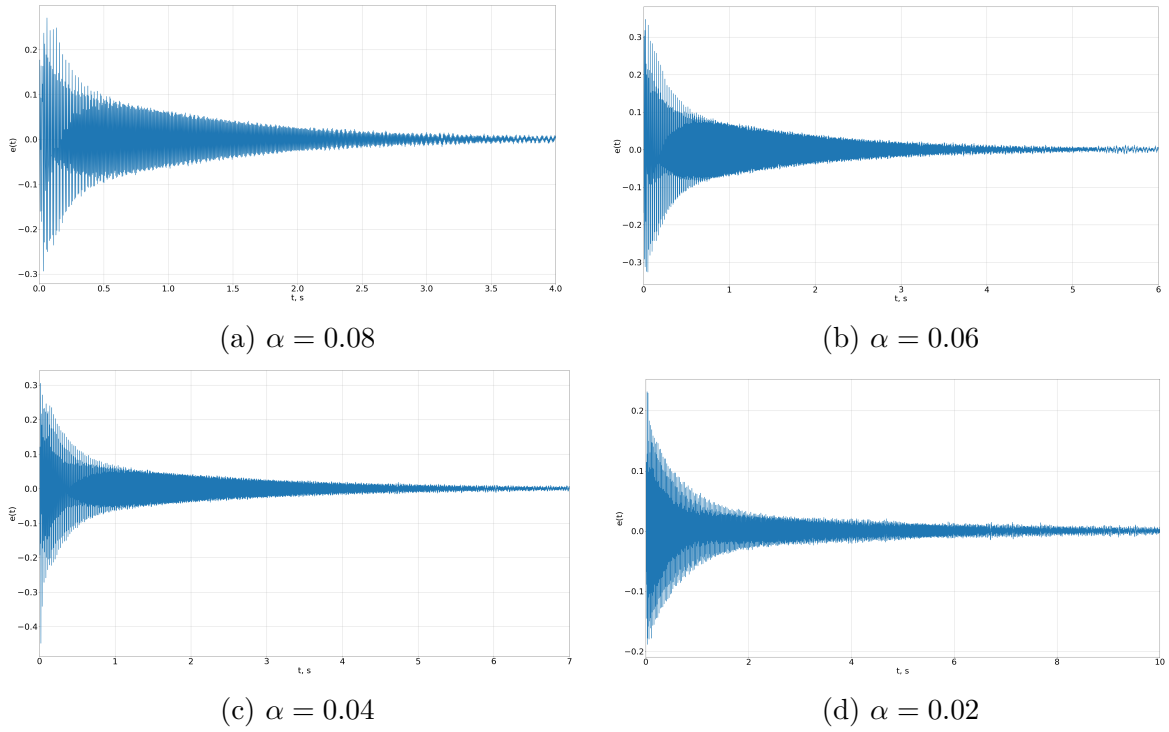
(c) $\alpha = 0.04$



(d) $\alpha = 0.02$

Rysunek 6.15: Badania wpływu rozmiaru kroku α — sygnał 2

Rezultaty dla trzeciego sygnału testowego przedstawiono na rysunku 6.16:



(a) $\alpha = 0.08$ (b) $\alpha = 0.06$

(c) $\alpha = 0.04$ (d) $\alpha = 0.02$

Rysunek 6.16: Badania wpływu rozmiaru kroku α - sygnał 3

Jak można zauważyć z rysunków 6.14, 6.15 i 6.16, krok α algorytmu NLMS wpływa na szybkość zbieżności, co pokrywa się z teorią. Szacowane czasy zbieżności dla poszczególnych wartości kroku i sygnałów testowych przedstawiono w tabeli 6.3:

Sygnał	t_z dla $\alpha = 0.1$	t_z dla $\alpha = 0.08$	t_z dla $\alpha = 0.06$	t_z dla $\alpha = 0.04$	t_z dla $\alpha = 0.02$
Sinusoida o $f = 120$ Hz	0.36	0.42	0.56	0.67	1.11
Sinusoida o $f = 120$ Hz	0.46	0.55	0.80	0.92	1.87
Suma sinusoid	1.8	3.6	4.54	6.16	> 10

Tablica 6.3: Wybrany zestaw parametrów referencyjnych

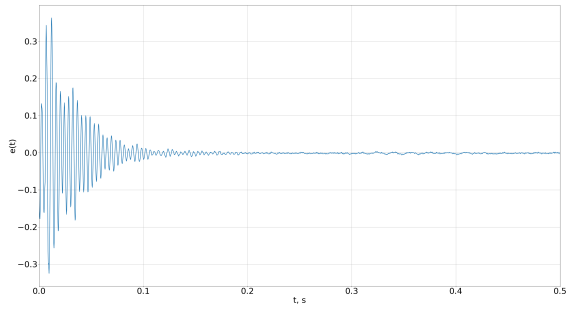
Zmniejszenie kroku powoduje wolniejsze zbieganie się algorytmu, a największą różnicę widać w momencie zmiany kroku z 0.04 na 0.02. Jak można zauważyć z 6.2, z sygnałów testowych najwolniej zbiega się suma sinusoid i dla tego sygnału zaobserwowano również największe różnice pomiędzy kolejnymi wartościami kroku.

Wpływ długości filtrów

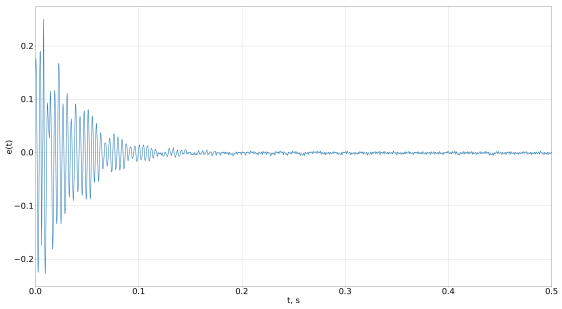
Do badania wpływu długości L filtra $W(z^{-1})$ przyjęto zestaw parametrów referencyjnych zgodnie z tabelą 6.2, a wartość L dobierano zgodnie z zależnością:

$$L(i) = 2^i \qquad \text{dla } i = 8, 9, 10. \tag{6.2}$$

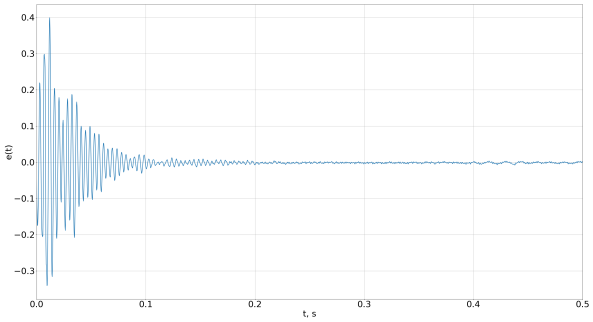
Rezultaty dla pierwszego sygnału testowego przedstawiono na rysunku 6.17:



(a) $L = 256$



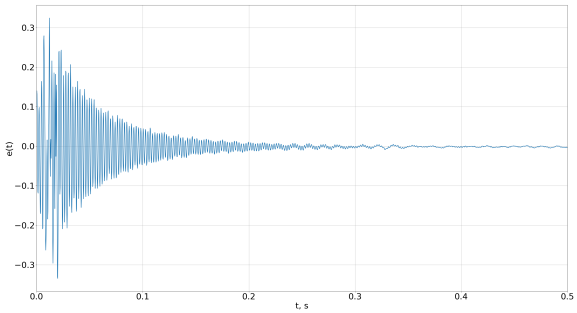
(b) $L = 512$



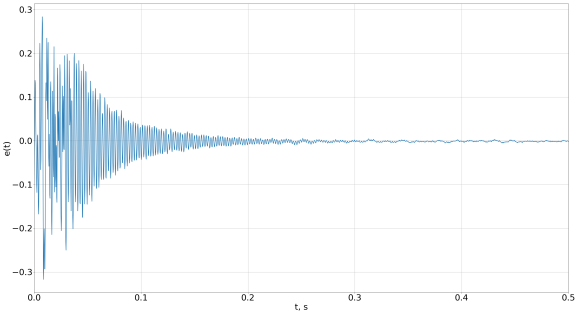
(c) $L = 1024$

Rysunek 6.17: Badania wpływu długości L filtra $W(z^{-1})$ — sygnał 1

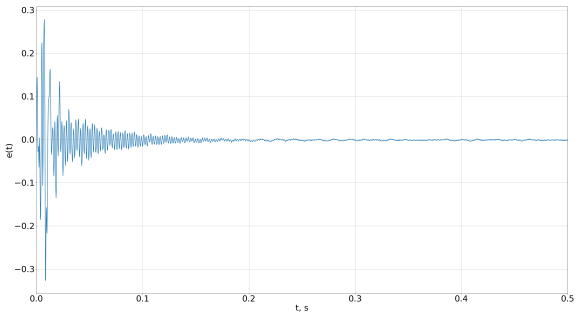
Rezultaty dla drugiego sygnału testowego przedstawiono na rysunku 6.18:



(a) $L = 256$



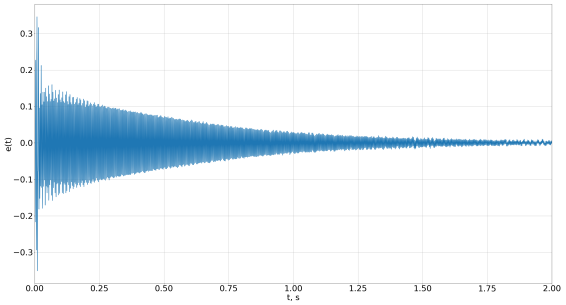
(b) $L = 512$



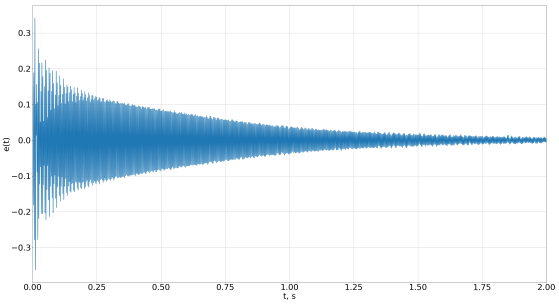
(c) $L = 1024$

Rysunek 6.18: Badania wpływu długości L filtra $W(z^{-1})$ — sygnał 2

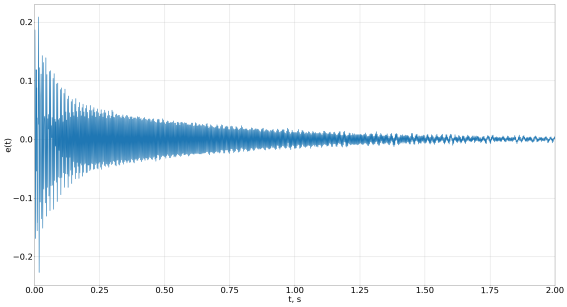
Rezultaty dla trzeciego sygnału testowego przedstawiono na rysunku 6.19:



(a) $L = 256$



(b) $L = 512$



(c) $L = 1024$

Rysunek 6.19: Badania wpływu długości L filtra $W(z^{-1})$ — sygnał 3

Jak można zauważyć z rysunków 6.17, 6.18 i 6.19, długość filtra wpływa nieznacznie na zebrane przebiegi. Czas zbieżności jest w przybliżeniu taki sam w przypadku różnych długości L poszczególnych sygnałów.

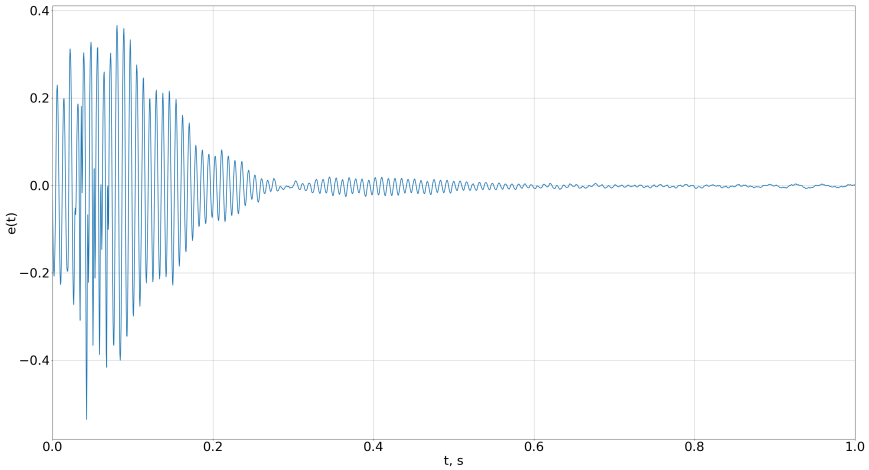
Kwestią znaczącą przy doborze długości L jest natomiast czas potrzebny na przetworzenie danych. W czasie pomiarów zaobserwowano, że dla $L > 1024$ obliczenia zaczynają trwać zbyt długo, co skutkuje przerwami w transmisji, spowodowanymi stanami *xrun_recovery* systemu ALSA (próbki nie są odpowiednio szybko czytane/zapisywane). Takie przerwy wpływają destrukcyjnie na działanie systemu i objawiają się destabilizacją pętli przetwarzania, co skutkuje generacją dodatkowego hałasu przez głośnik wyjściowy.

Wpływ częstotliwości próbkowania

Do badania wpływu częstotliwości próbkowania f_s przyjęto zestaw parametrów referencyjnych zgodnie z tabelą 6.2, a jako nową wartość f_s przyjęto:

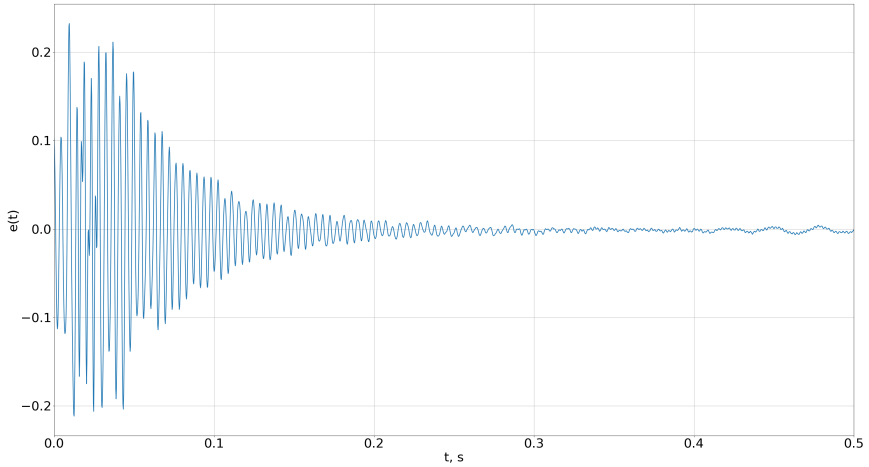
$$f_s = 2f_{s,ref} = 16000 \text{ Hz}. \tag{6.3}$$

Rezultaty dla pierwszego sygnału testowego przedstawiono na rysunku 6.20:



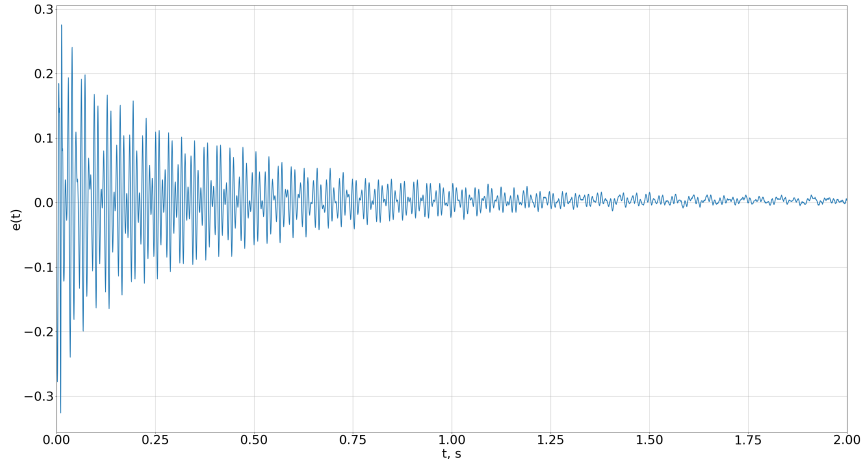
Rysunek 6.20: Badanie wpływu częstotliwości próbkowania — sygnał 1

Rezultaty dla drugiego sygnału testowego przedstawiono na rysunku 6.21:



Rysunek 6.21: Badanie wpływu częstotliwości próbkowania — sygnał 2

Rezultaty dla trzeciego sygnału testowego przedstawiono na rysunku 6.22:



Rysunek 6.22: Badanie wpływu częstotliwości próbkowania — sygnał 3

Analizując przebiegi 6.20, 6.21 i 6.22 można stwierdzić, że zwiększona częstotliwość próbkowania nie spowodowała drastycznych zmian w porównaniu z poprzednią (mniejszą). Czas zbieżności dla poszczególnych sygnałów testowych jest zbliżony z rezultatami dla $f_s' = 2000$ Hz.

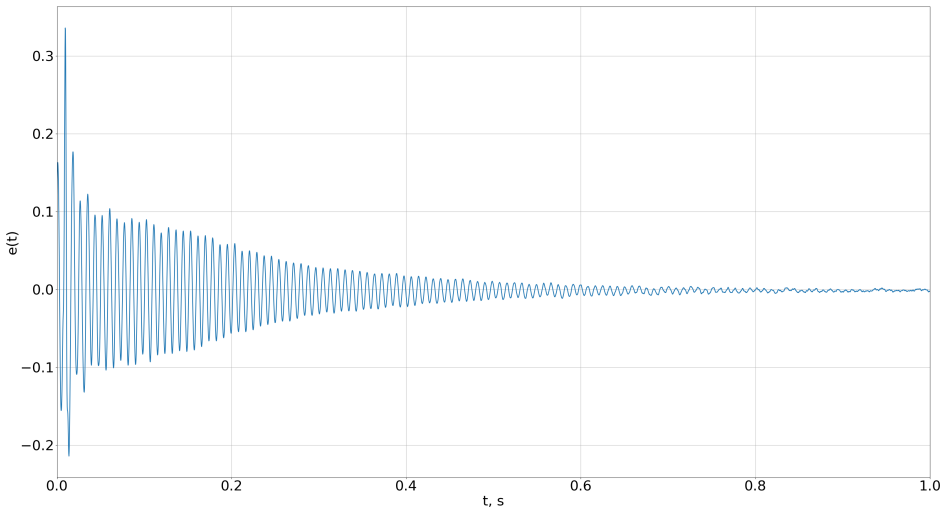
Pomimo, że w zaprojektowanym układzie częstotliwość próbkowania nie wpływa znacząco na przebiegi, to jej wzrost objawia się mniejszym czasem na wykonanie obliczeń. Dla częstotliwości $f_s > 4000$ system zaczyna nie nadążać z ich wykonaniem, co skutkuje, analogicznie jak w przypadku zbyt długich filtrów, przerwaniem transmisji i destabilizacją pętli przetwarzania.

Wpływ rozmiaru fragmentu

Do badania wpływu rozmiaru fragmentu n_f przyjęto zestaw parametrów referencyjnych zgodnie z tabelą 6.2, a jako nową wartość przyjęto

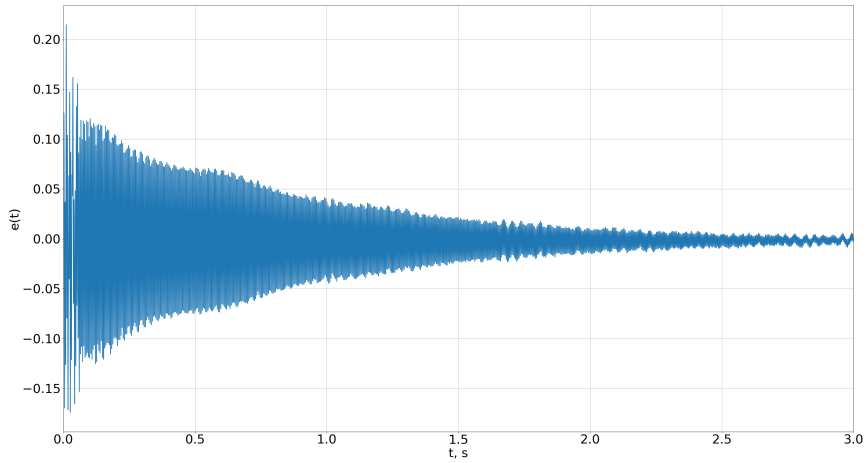
$$n_f = 2n_{f,ref} = 128 \text{ ramek} \tag{6.4}$$

Rezultaty dla pierwszego sygnału testowego przedstawiono na rysunku 6.20:



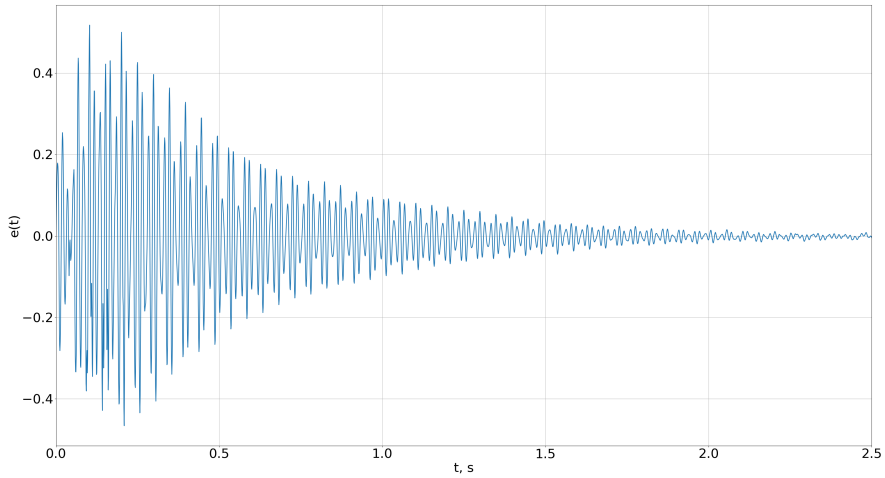
Rysunek 6.23: Badanie wpływu rozmiaru fragmentu — sygnał 1

Rezultaty dla drugiego sygnału testowego przedstawiono na rysunku 6.21:



Rysunek 6.24: Badanie wpływu rozmiaru fragmentu — sygnał 2

Rezultaty dla trzeciego sygnału testowego przedstawiono na rysunku 6.22:



Rysunek 6.25: Badanie wpływu rozmiaru fragmentu — sygnał 3

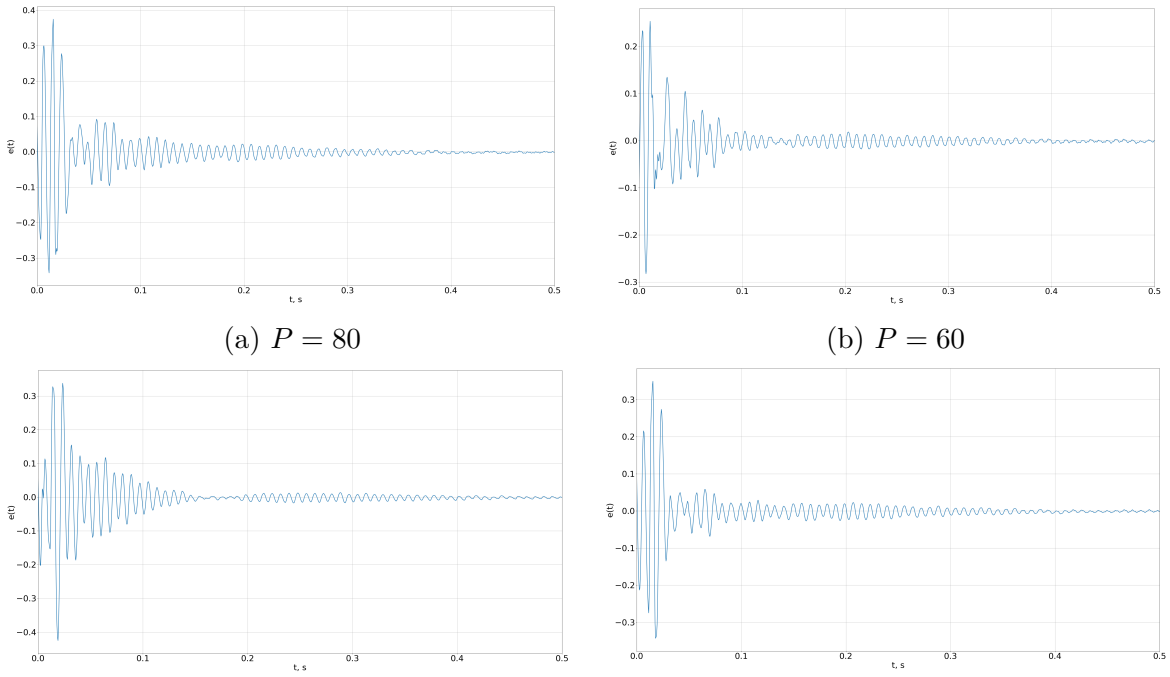
Wnioskując z 6.23, 6.24 i 6.25 można stwierdzić, że zwiększony rozmiar fragmentu znacząco pogorszył jakość tłumienia. Algorytmy zbiegają się dłużej, co jest najbardziej widoczne w przypadku sygnału drugiego, dla którego algorytm zbiega się w przybliżeniu trzy razy wolniej, niż algorytm z parametrami referencyjnymi. Jak pokazano na rysunku 6.10, zwiększony rozmiar fragmentu powoduje większe opóźnienie, co może być głównym powodem zaobserwowanego w tej serii badań zachowania.

Wpływ poziomu priorytetu

Do badania wpływu poziomu priorytetu P procesu przyjęto zestaw parametrów referencyjnych zgodnie z tabelą 6.2, a poziom P dobierano zgodnie z zależnością:

$$P(i) = 100\frac{i}{5} \qquad \text{dla } i = 1, 2, 3, 4 \tag{6.5}$$

Rezultaty dla pierwszego sygnału testowego przedstawiono na rysunku 6.26:

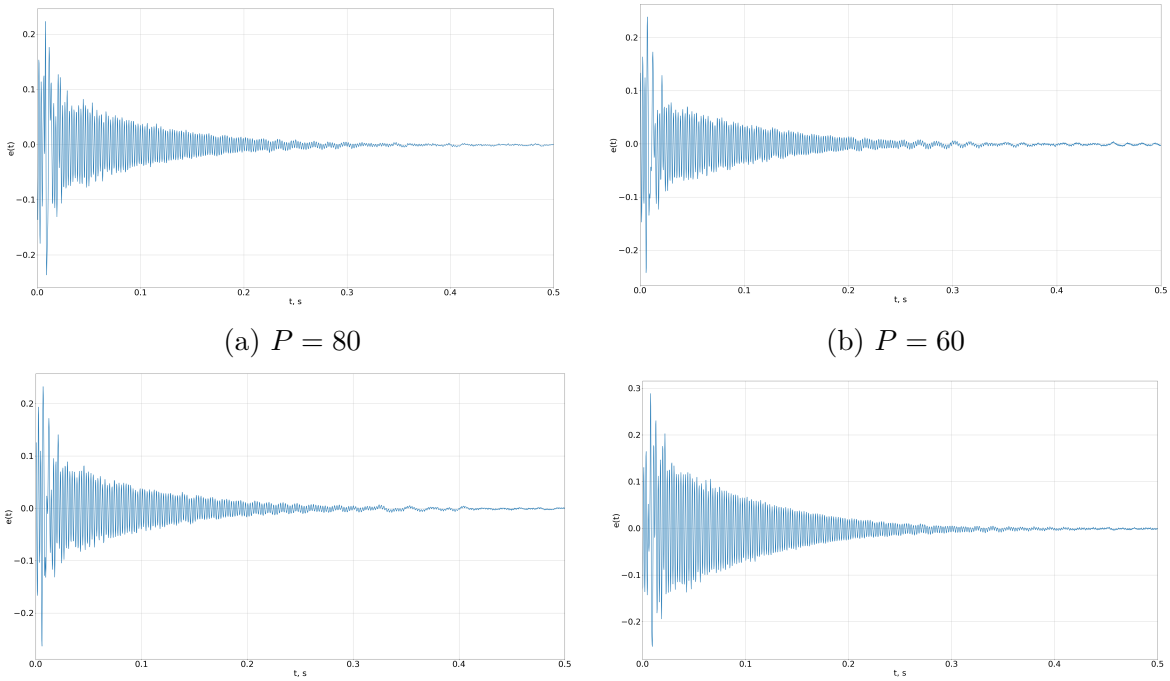


(a) $P = 80$ (b) $P = 60$

(c) $P = 40$ (d) $P = 20$

Rysunek 6.26: Badania wpływu poziomu priorytetu P — sygnał 1

Rezultaty dla drugiego sygnału testowego przedstawiono na rysunku 6.26:

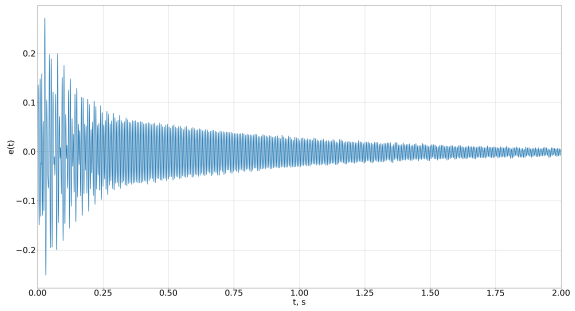


(a) $P = 80$ (b) $P = 60$

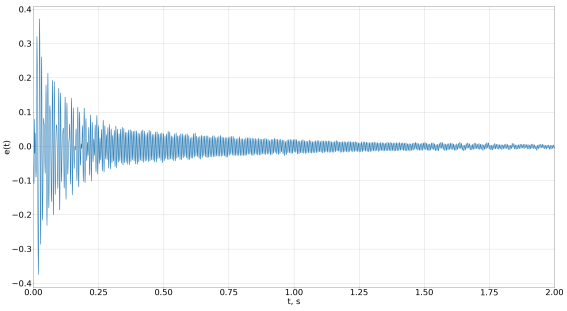
(c) $P = 40$ (d) $P = 20$

Rysunek 6.27: Badania wpływu poziomu priorytetu P — sygnał 2

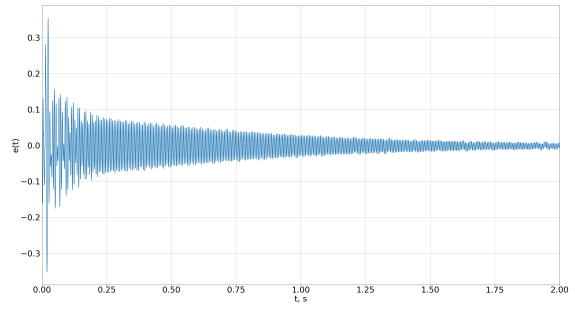
Rezultaty dla trzeciego sygnału testowego przedstawiono na rysunku 6.28:



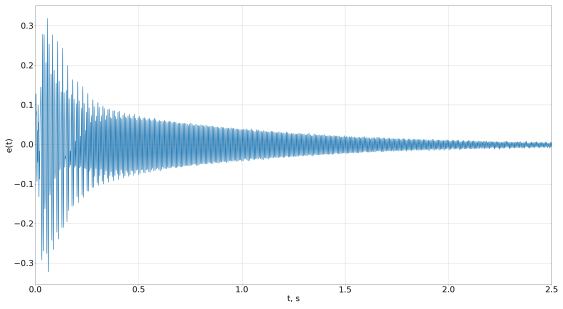
(a) $P = 80$



(b) $P = 60$



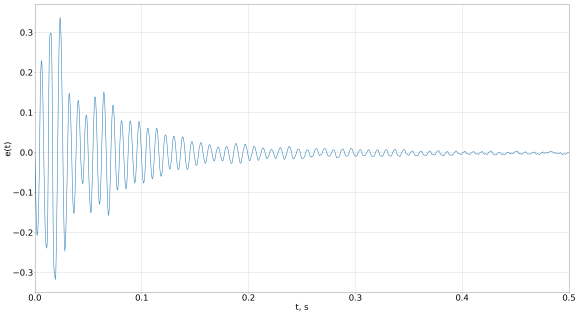
(c) $P = 40$



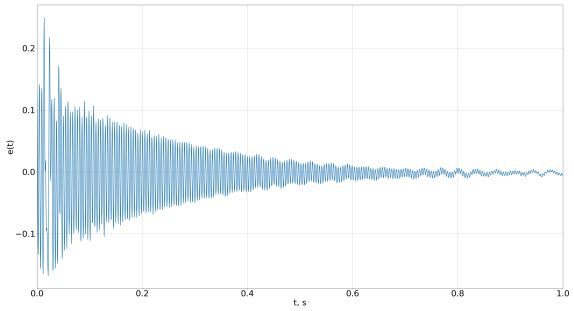
(d) $P = 20$

Rysunek 6.28: Badania wpływu poziomu priorytetu P — sygnał 3

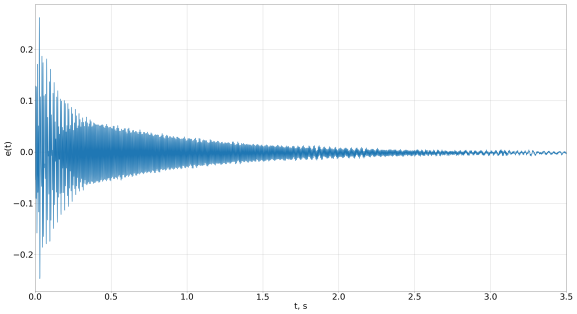
Na rysunkach 6.29 przedstawiono przebiegi uzyskane dla zmiany wytycznej planisty z SCHED_RR na SCHED_FIFO:



(a) Pierwszy sygnał testowy



(b) Drugi sygnał testowy



(c) Trzeci sygnał testowy

Rysunek 6.29: Badania rozszerzeń czasu rzeczywistego — zmiana wytycznej planisty

Jak pokazano na 6.26, 6.27 i 6.28, zmniejszanie poziomu priorytetu w analizowanym

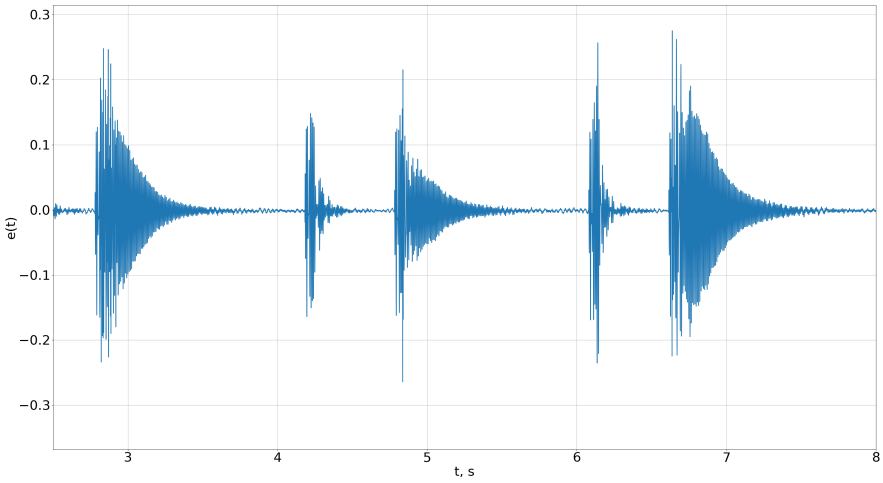
zakresie kompletnie nie wpływa jakość wynikowego tłumienia. Czasy zbieżności w każdym przypadku są prawie identyczny, w porównaniu z przebiegami uzyskanymi dla maksymalnego priorytetu. Powodem, dla którego tak się dzieje, jest z pewnością rozkład procesów pracujących w systemie. Analizując rozkład procesów podczas działania pętli ANC z wykorzystaniem polecenia *top* zaobserwowano, że prócz procesu o nazwie *migration/0*, odpowiadającego za dystrybucję czasu pracy pomiędzy rdzenie procesora oraz tych, odpowiadających za obsługę przerwań (procesy *irq/**), żadne inne nie posiadają priorytetu większego niż $P = 20$.

Zmiany wytycznej planisty lekko pogorszyła zbieżność algorytmu, co przedstawiono na rysunkach 6.29. Tłumienie drugiego i trzeciego sygnału testowego odbywa się nieznacznie dłużej niż w przypadku wykorzystania SCHED_RR. Poza dłuższym czasem zbieżności, nie zaobserwowano dodatkowych zmian. [*A ma Pan jakiś pomysł dlaczego w ogóle widać jakieś zmiany? Bo teoretycznie jeśli nic innego nie ma tego samego priorytetu co Pan program, to różnic nie powinno być.*]

6.2.3 Dodatkowe eksperymenty i podsumowanie badań

Ekperyment — losowe pobudzenia sinusoidalne

Eksperyment opiera się na wykorzystaniu sygnału testowego, który składa się z pobudzeń sinusoidalnych z zakresu od 100 Hz do 400 Hz generowanych w pewnych odstępach czasu. Pomiędzy pobudzeniami stosowano arbitralne okresy ciszy, manualnie włączając i wyłączając głośnik, stanowiący symulowane źródło hałasu w systemie. Przebieg błędu $e(n)$ dla tak opisanego sygnału wejściowego przedstawiono na rysunku 6.30:



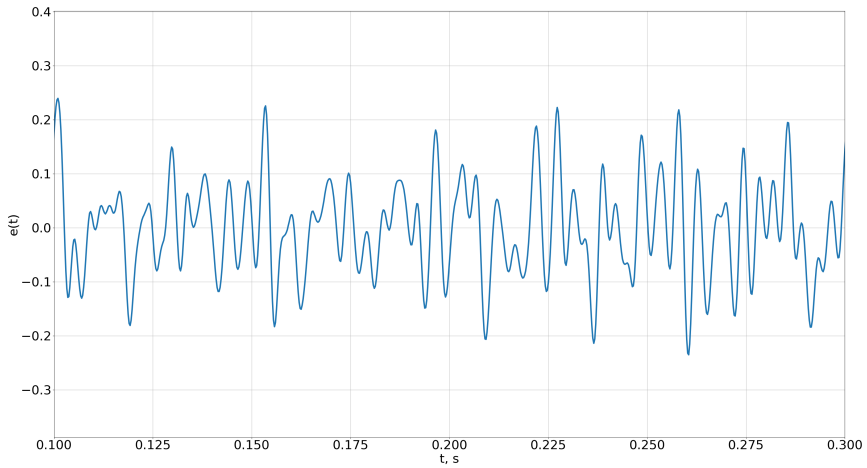
Rysunek 6.30: Eksperyment — pseudolosowe pobudzenia sinusoidalne

Jak można zauważyć z 6.30, każdy nowo pojawiający się hałas tonalny zostaje stłumiony idealnie, a czas zbieżności nie przekracza jednej sekundy. Okresy ciszy (hałas nie jest emitowany) nie powodują niechcianych pobudzeń systemu.

Analizując układ dla hałasów totalnych o innych częstotliwościach niż te, które wykorzystano jako sygnały testowe do badania wpływu algorytmów, zauważono, że system jest w stanie w dobrym stopniu tłumić sinusoidy do założonego pasma. Poza nim, system nie działa prawidłowo i zaczyna generować sygnały powodujące większy hałas niż wejściowy.

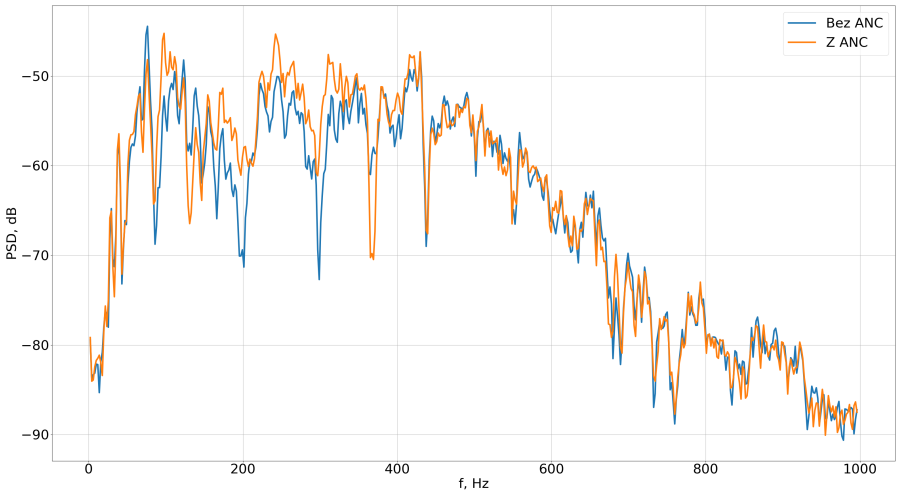
Eksperyment — hałas losowy

W przeprowadzonym eksperymencie, korzystając z zestawu parametrów referencyjnych określonych w tabeli 6.2, jako hałas wykorzystano sygnał losowy wygenerowany generatorem *xoroshiro16++*, którego nagranie mikrofonem błędu sygnału przedstawiono na rysunku 6.31:



Rysunek 6.31: Sygnał testowy — szum losowy (po przybliżeniu i normalizacji)

Porównanie gęstości widmowej mocy sygnału $e(n)$ w przypadku włączonego i wyłączonego ANC dla wejściowego hałasu losowego obliczoną metodą Welcha z oknem Hanna dla $f_s' = 2$ kHz przedstawiono na rysunku 6.32:

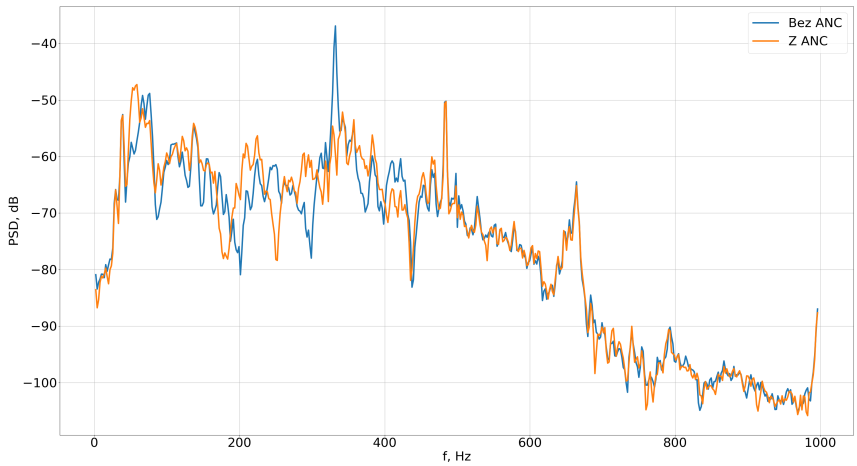


Rysunek 6.32: Sygnał losowy — pierwszy eksperyment

Jak można zauważyć z 6.32, system nie działa poprawnie dla hałasów losowych. Takie zachowanie systemu można wywnioskować na podstawie przeprowadzonych wcześniej badań, z których wynika, że algorytm zbiega się zdecydowanie zbyt wolno, żeby poprawnie reagować na szybkozmienny, pseudolosowy sygnał wejściowy. W przypadku włączonego systemu, pasmo do 500 Hz jest w wielu miejscach o większej mocy, niż w przypadku wyłączonego, co świadczy o tym, że włączone ANC w tym przypadku tylko pogarsza sprawę.

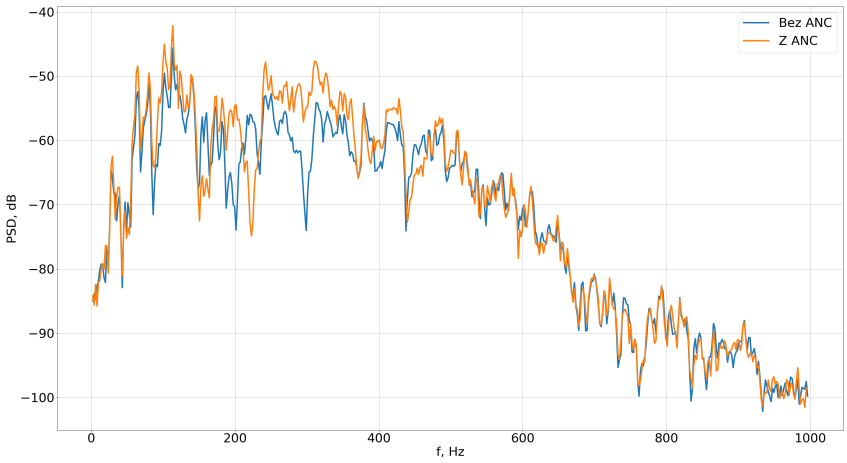
Eksperyment — hałasy rzeczywiste

W przeprowadzonym eksperymencie sprawdzono, jak układ zachowuje się dla rzeczywistych sygnałów wejściowych pochodzących z elektrowni. Działanie układu dla pierwszego sygnału, będącego nagraniem pracujących młynów kulowych, przedstawiono w postaci przebiegów gęstości widmowej mocy obliczonej metodą Welcha z oknem Hanna dla $f_s' = 2$ kHz dla włączonego i wyłączonego systemu na rysunku 6.33:



Rysunek 6.33: Eksperyment — hałas generowany przez pracujące młyny kulowe

Działanie układu dla drugiego sygnału, będącego nagraniem pracujących pomp wody w analogiczny sposób przedstawiono na rysunku 6.34:



Rysunek 6.34: Eksperyment — hałas generowany przez pracujące pompy wody

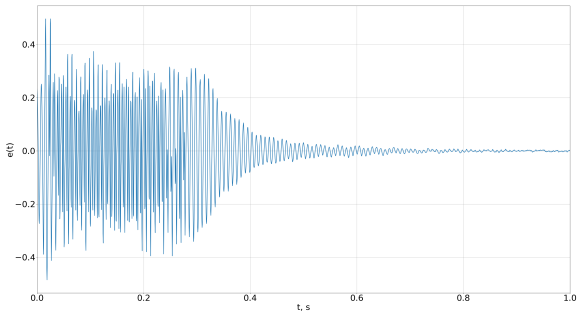
Jak można zauważyć z rysunku 6.33, układ był w stanie wytłumić mocno odznaczający się pik w okolicach 400 Hz o około 12 dB. Poza osłabieniem odznaczającej się składowej, moc reszty przy włączonym tłumieniu jest na podobnym poziomie co przy wyłączonym systemie.

Dla drugiego eksperymentu, wprost z rysunku 6.34 można zauważyć, że dla tego typu sygnału układ nie działa w sposób przewidywalny. Włączony system podwyższa moc wielu składowych pasma do 500 Hz, gdzie najbardziej odznaczającymi są te z przedziału od około 250 Hz do 300 Hz.

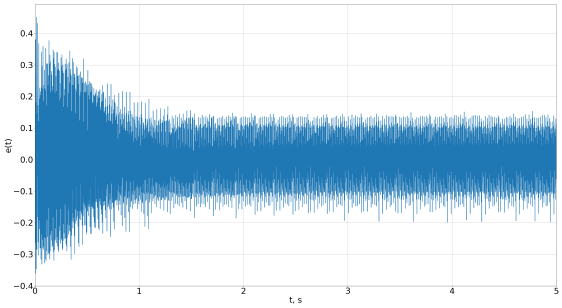
Eksperyment — działanie algorytmu na jądrze bez łatki PREEMPT_RT

Aby dodatkowo zweryfikować wpływ rozszerzeń czasu rzeczywistego, jądro systemu operacyjnego na wykorzystywanej platformie obliczeniowej podmieniono na takie bez wprowadzonej łatki PREEMPT_RT. Uruchamiając program, priorytet procesu pozostawiono jako domyślny oraz nie zmienianio wytycznej planisty.

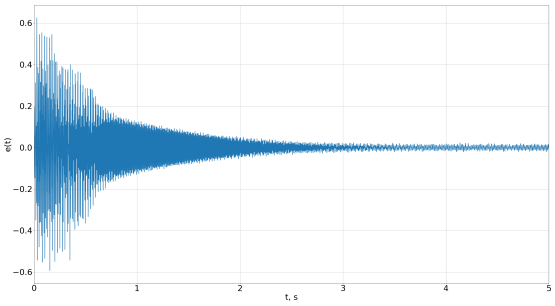
Realizując badania dla wykorzystanych wcześniej sygnałów testowych od razu zauważono, że system zachowuje się mniej deterministycznie niż wcześniej, co uchwycono na rysunkach 6.35:



(a) Pierwszy sygnał testowy



(b) Drugi sygnał testowy



(c) Trzeci sygnał testowy

Rysunek 6.35: Eksperyment - jądro bez łatek PREEMPT_RT

Jednym z najbardziej znaczących powodów, dla którego system mógł zacząć zachowywać się znacząco gorzej, jest domyślny priorytet, który jest nadawany procesowi, wynoszący $P = 0$. Proces z takim priorytetem jest łatwo wywłaszczalny przez inne (z większym), co z pewnością powoduje problemy z synchronizacją w trakcie działania algorytmu.

Podsumowanie badań

Wpływy poszczególnych parametrów na działanie systemu przedstawiono poniżej:

1. Wielkość kroku adaptacji α algorytmu NLMS — decyduje o szybkości zbiegania się algorytmu. System najszybciej zbiega się dla:

$$\alpha_{ref} = 0.1, \tag{6.6}$$

   staje się niestabilny dla większego i zmniejsza szybkość zbieżności wraz z jego zmniejszaniem.

2. Długość filtra $W(z^{-1})$ — nie wpływa znacząco na jakość tłumienia, ale ma duży wpływ na szybkość wykonywania obliczeń. Dla $L > 1024$ obliczenia zaczynają trwać zbyt długo, co skutkuje przerwami w transmisji.

3. Częstotliwość próbkowania f'_s — podobnie jak długość L filtra $W(z^{-1})$, nie wpływa znacząco na jakość tłumienia, ale im jest większa, tym okno czasowe na wykonanie

obliczeń mniejsze. Zweryfikowano, że zaprojektowany układ zaczyna nie nadążać z obliczeniami dla $f'_s > 4000$ Hz.

4. Długość fragmentu n_f — im większa, tym system działa gorzej. Zwiększenie długości powoduje zwiększenie opóźnienia w torze przetwarzania, co ma mocno negatywny wpływ na jakość tłumienia.

5. Poziom priorytetu P — do momentu, jeżeli nie przekracza się poziomu, w którym pracują inne procesy w systemie, nie powoduje znaczących zmian. Poza tym poziomem, który dla przeprowadzonych pomiarów zdefiniowany był na $P = 20$, istnieje szansa, że algorytm się rozsynchronizuje.

6. Wytyczna planisty — w wykonanych badaniach nie zaobserwowano decydującego wpływu, jednak jak można zaobserwować, SCHED_RR działa nieznacznie lepiej.

W dodatkowych eksperymentach zweryfikowano, że:

1. **System jest w stanie tłumić hałasy tonalne w postaci pojedynczych sinusoid i ich sum dla założonego pasma (do 500 Hz)** — poza tym pasmem, układ zaczyna generować sygnały nie powodujące tłumienia. Poprawnie tłumione są również sygnały składające się z sum sinusoid, jednak warunkiem jest, aby hałas nie zmieniał swojej częstotliwości szybciej niż około 1 s.

2. **System nie jest w stanie tłumić hałasów losowych** — zbieżność algorytmu dla takich sygnałów jest zbyt wolna.

3. **System jest w stanie wytłumić mocno odznaczające się składowe częstotliwościowe hałasów rzeczywistych o około 12 dB** — w przypadku nagrania młynów kulowych, odznaczający się pik w okolicach 400 Hz został wytłumiony o około 12 dB.

Rozdział 7

Podsumowanie

Okres projektowania i konstruowania wykonanego w ramach projektu systemu był dla autora niewątplie czasem niezwykle kształcącym i rozwijającym. System, konstruowany i projektowany od fundamentów, pozwolił autorowi na dokładną obserwację wpływu poszczególnej decyzji projektowej na finalny sposób działania układu. Aby system mógł wykonać swoje zadanie w najprostszej formie, każdy element musiał idealnie ze sobą współgrać, więc nawet najmniejsza, błędna decyzja projektowa wykonana w fazach początkowych, mogła finalnie spowodować wiele problemów w fazach finalnych, powodując nawet zakończenie projektu fiaskiem. Poprawne działanie systemu z ograniczeniami, które zaobserowowano badając działanie dla poszczególnych sygnałów symulujących hałas, jest z pewnością wielkim sukcesem i zarazem potwierdzeniem, że nad tego typu układami można pracować (z różnymi rezultatami) korzystając z amatorskich, ogólnodostępnych i tanich (lub darmowych) rozwiązań.

Analizując rezultaty przeprowadzonych badań od razu można stwierdzić, dlaczego wykorzystanie systemu operacyjnego, który nie jest systemem czasu rzeczywistego, w przypadku projektowania tego typu układów nie jest dobrą i popularną decyzją. Opóźnienia wprowadzane przez taki system i trudności związane z synchronizacją peryferiów objawiają się mocno negatywnym wpływem na jakość działania finalnego układu. Jak pokazano w sekcjach badającej wpływ wykorzystanej łatki PREEMPT_RT, polepsza ona działanie układu w porównaniu z klasycznym jądrem. Wykorzystanie takich rozszerzeń nie powoduje jednak, że system bazujący na jądrze Linux staje się automatycznie systemem czasu rzeczywistego. Opóźnienia, pomimo że zmniejszone i bardziej deterministyczne, dalej występują i ich wpływ dyskredytuje działanie systemu dla sygnałów losowych i takich, które szybko zmieniają swoją częstotliwość. Adaptacja w przypadku takich sygnałów reaguje zbyt wolno i hałasy nie są tłumione poprawnie. Dodatkowo, system ALSA, jako że nie jest przystosowany do realizacji krytycznych czasowo operacji wejścia/wyjścia, z racji swojej elastyczności nie jest idealnym wyborem przy realizacji systemów ANC. Jego głównym zadaniem jest zapewnienie warstwy abstrakcji wykorzystywanej do komunikacji z peryferiami audio, umożliwiającej konfigurowanie urządzeń z poziomu przestrzeni

użytkownika, więc tworzenie pętli regulacji z jego wykorzystaniem nie jest z pewnością wyborem optymalnym.

Pomimo problemów opisanych wyżej, zdaniem autora istnieje kilka sposobów, których wykorzystanie może polepszyć działanie systemu. Sterowniki jądra odpowiadające za transfery DMA oraz obsługę określonych urządzeń audio można zmodyfikować, zmieniając rozmiar fragmentu oraz częstotliwość próbkowania na mniejsze. Docelowo, jeżeli udałoby się uzyskać $n_f = 1$ i $f_s = 1$ kHz, pętla przetwarzania mogłaby operować na jednej próbce, jak to ma miejsce w typowych implementacjach na procesory sygnałowe i mikrokontrolery. Rozmiar fragmentu o wartości jednej ramki spowodowałby opóźnienie o jedną ramkę, które jest krytyczne dla jakości działania algorytmu. Z częstotliwością próbkowania również można iść w drugą stronę, zwiększając ją wraz z krotnością decymacji w pętli przetwarzania. Takie działanie, poza zmniejszeniem opóźnień, może również pozwolić na implementację prostszej części analogowej, zmniejszając wymagania na projektowane filtry.

Poza modyfikacjami systemowymi, zaimplementowane rozwiązanie można ulepszyć poprzez urównoleglenie operacji przedstawionych na rysunku 5.13. Równoczesne przetwarzanie sygnałów $x(n)$ i $e(n)$ z pewnością spowoduje szybsze działanie pętli przetwarzania[*BBB ma tylko 1 procesor, a i też tak jak ostatnio mówiliśmy, tutaj nie ma za dużo przetwarzania związanego z $e(n)$*], a to można wykorzystać do zwiększenia próbkowania lub długości filtrów, jeżeli będzie taka potrzeba. Poza tym, program można rozszerzyć o wspomnianą w sekcji analizy tematu metodę niwelowania dodatkowej pętli sprzężenia, jeżeli wymagane jest, aby układ działał na hałasach o wyższym natężeniu i z ułożeniem sprzętu, powodującym wpływanie sygnału $y(n)$ na $x(n)$. W przypadku zaprojektowanej elektorniki, patrząc z perspektywy czasu autor uważa, że wybrane przetworniki mogłyby być mniejszej dokładności (przykładowo 12 bitowe), a filtry o mniejszym stopniu, powodując mniejsze opóźnienia w torach analogowych[*z tym ostatnim to tak nie do końca. Jeśli chce Pan tłumić na określonym poziomie częstotliwości aliasingowe to musi mieć Pan jakieś tam opóźnienie, i tak jak kiedyś analizowałem, to przy filtrach analogowych optymalny stopień zależy od poziomu. I z tego co pamiętam to dla -45 dB optymalny był rząd 6-ty. Pan ma jakieś -36 dB, i może faktycznie 4-ty byłby lepszy. Ja to sprawdzałem dla filtrach optymalnych pod względem opóźnienia fazowego. Dla Butterwortha to może być trochę inaczej.*].

Bibliografia

[1] *2.7 V to 5.5 V, Serial-Input, Voltage-Output, 16-Bit DACs*. 2004. URL: https://www.analog.com/media/en/technical-documentation/data-sheets/AD5541_5542.pdf (term. wiz. 23. 07. 2023).

[2] *3 mW, 100 kSPS, 16-Bit ADC in 6-Lead SOT-23*. 2004. URL: https://www.analog.com/media/en/technical-documentation/data-sheets/AD7680.pdf (term. wiz. 23. 07. 2023).

[3] *BeagleBone Black Board*. URL: https://www.beagleboard.org/boards/beaglebone-black (term. wiz. 31. 07. 2023).

[4] Ewa Bielińska, Jarosław Figwer, Jerzy Kasprzyk, Tadeusz Legierski, Zbigniew Ogonowski i Marek Pawełczyk. *Identyfikacja procesów*. Gliwice: Wydawnictwo Politechniki Śląskiej, 2002. ISBN: 83-7335-077-2.

[5] *C/C++ documentation*. URL: https://en.cppreference.com/w/c/numeric/random (term. wiz. 03. 10. 2023).

[6] *Design of Active Noise Control Systems With the TMS320 Family*. 1996. URL: https://www.ti.com/lit/an/spra042/spra042.pdf (term. wiz. 23. 07. 2023).

[7] David Halliday, Robert Resnick i Jearl Walker. *Podstawy fizyki*. Warszawa: Wydawnictwo naukowe PWN, 2015. ISBN: 9788301181147.

[8] *High Voltage, Low Quiescent Current Inverting Charge Pump*. 2012. URL: https://www.analog.com/media/en/technical-documentation/data-sheets/3261fb.pdf (term. wiz. 23. 07. 2023).

[9] Jui-Kuo Juan, J.G. Harris i J.C. Principe. *Analog hardware implementation of adaptive filter structures*. 1997. DOI: 10.1109/ICNN.1997.616147.

[10] *KRK Classic 8 Professional Studio Monitor*. URL: https://www.krkmusic.com/Studio-Monitors/CLASSIC-8 (term. wiz. 23. 07. 2023).

[11] Sen Maw Kuo i Dennis R. Morgan. *Active Noise Control Systems: Algorithms and DSP Implementations*. USA: John Wiley & Sons, Inc., 1995. ISBN: 0471134244.

[12] Stephan T. Lavavej. *rand() Considered Harmful*. 2013.

[13] Jared Lewis. *Understanding Microphone Sensitivity*. 2012. URL: https://www.analog.com/media/en/analog-dialogue/volume-46/number-2/articles/understanding_microphone_sensitivity.pdf (term. wiz. 23. 07. 2023).

[14] *LM1085 3-A Low Dropout Positive Regulators*. 1999. URL: https://www.ti.com/lit/ds/symlink/lm1085.pdf (term. wiz. 23. 07. 2023).

[15] *LM2940x 1-A Low Dropout Regulator*. 2000. URL: https://www.ti.com/lit/ds/symlink/lm2940-n.pdf (term. wiz. 23. 07. 2023).

[16] *LM2990 Negative Low Dropout Regulator*. 1999. URL: https://www.ti.com/lit/ds/symlink/lm2990.pdf (term. wiz. 23. 07. 2023).

[17] *Low-Cost, Low-Power, Low-Dropout, SOT23-3 Voltage References*. 1999. URL: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX6001-MAX6005.pdf (term. wiz. 23. 07. 2023).

[18] Richard G. Lyons. *Wprowadzenie do cyfrowego przetwarzania sygnałów*. Warszawa: Wydawnictwa Komunikacji i Łączności, 2010. ISBN: 978-83-206-1764-1.

[19] *Measurement Condenser Microphone ECM8000 Technical Specifications*. URL: https://mediadl.musictribe.com/media/PLM/data/docs/P0118/S_BE_P0118_ECM8000_A4_WW.pdf (term. wiz. 23. 07. 2023).

[20] Erdahl Michael. *LC filter design*. 2017. URL: https://www.ti.com/lit/an/sprac97/sprac97.pdf (term. wiz. 23. 07. 2023).

[21] *Moore's law*. URL: https://www.britannica.com/technology/Moores-law (term. wiz. 14. 08. 2023).

[22] Henry W. Ott. *Electromagnetic Compatibility Engineering*. USA: John Wiley & Sons, Inc., 2009. ISBN: 9780470189306.

[23] *Precision Micropower, Low Dropout Voltage References*. 1996. URL: https://www.analog.com/media/en/technical-documentation/data-sheets/REF19xSeries.pdf (term. wiz. 23. 07. 2023).

[24] *Raspberry Pi Documentation*. URL: https://www.raspberrypi.com/documentation/ (term. wiz. 30. 07. 2023).

[25] *Sitara^{TM} Linux ALSA DSP Microphone Array Voice Recognition*. 2016. URL: https://www.ti.com/lit/an/slaa701a/slaa701a.pdf (term. wiz. 23. 07. 2023).

[26] Torsten Söderström i Petre Stoica. *Identyfikacja systemów*. Warszawa: Wydawnictwo Naukowe PWN, 1997. ISBN: 83-01-12158-0.

[27] Ulrich Tietze i Christoph Schenk. *Sztuka elektroniki*. Warszawa: Wydawnictwa Naukowo-Techniczne, 1997. ISBN: 83-204-2123-3.

[28] Ulrich Tietze i Christoph Schenk. *Układy półprzewodnikowe*. Warszawa: Wydawnictwa Naukowo-Techniczne, 1997. ISBN: 83-204-2123-3.

[29] *TL07xx Low-Noise FET-Input Operational Amplifiers*. 2023. URL: https://www.ti.com/lit/ds/symlink/tl072h.pdf (term. wiz. 23. 07. 2023).

[30] *TPS56x209, 4.5V to 17V Input, 2-A, 3-A Synchronous Step-Down Voltage Regulator in 6 pin SOT-23*. 2014. URL: https://www.ti.com/lit/ds/symlink/tps562209.pdf (term. wiz. 23. 07. 2023).

[31] Bobby Tufino. *McASP Design Guide - Tips, Tricks and Practical Examples*. 2019. URL: https://www.ti.com/lit/an/sprack0/sprack0.pdf (term. wiz. 23. 07. 2023).

[32] *Understanding Linux real-time PREEMPT_RT training*. 2004. URL: https://bootlin.com/doc/training/preempt-rt (term. wiz. 23. 07. 2023).

[33] *VEC50 Series AC-DC Power Supplies*. 2021. URL: https://www.xppower.com/portals/0/pdfs/SF_VEC50.pdf (term. wiz. 23. 07. 2023).

[34] *xoshiro/xoroshiro generators and the PRNG shootout*. URL: https://prng.di.unimi.it/ (term. wiz. 03. 10. 2023).

Dodatki

Spis skrótów i symboli

SPL poziom natężenia dźwięku (ang. *sound pressure level*)

ANC aktywna redukcja hałasu (ang. *active noise control*)

ADC przetwornik analogowo-cyfrowy (ang. *analog to digital converter*)

DAC przetwornik cyfrowo-analogowy (ang. *digital to analog converter*)

SMPS zasilacz impulsowy (ang. *switching-mode power supply*)

PSU sekcja zasilania (ang. *power supply unit*)

AP zasilanie sekcji analogowych (ang. *analog power*)

DP zasilanie sekcji cyfrowych (ang. *digital power*)

REF zasilanie referencyjne (ang. *reference voltage*)

EMIF filtracja zakłóceń wysokiej częstotliwości (ang. *electromagnetic interference filtering*)

DSP przetwarzanie sygnałów cyfrowych lub procesor sygnałowy (ang. *digital signal processing*, ang. *digital signal processor*)

McASP wielokanałowy szeregowy interfejs audio (ang. *multi-channel audi serial port*)

EDA oprogramowanie wspierające projektowanie układów elektroniczych (ang. *electronic design automation*)

FIR filtr o skończonej odpowiedzi impulsowej (ang. *finite impulse response*)

LMS algorytm adaptacyjny oparty na gradientowej metodzie najmniejszego spadku (ang. *least mean squares*)

FXLMS algorytm adaptacyjny oparty na gradientowej metodzie najmniejszego spadku z filtracją wejścia(ang. *filtered x least mean squares*)

Pliki uzupełniające

Wraz z niniejszym dokumentem załączono co następuje:

- Kody źródłowe
- Projekt sprzętu

Spis rysunków

1.1 Wizualizacja fizycznej idei aktywnej redukcji hałasu 3
1.2 Typowy tor przetwarzania 4

2.1 Porównanie struktur systemów 7
2.2 Podział struktur systemów ANC 8
2.3 Wyprowadzenie schematu systemu ANC, 0 9
2.4 Wyprowadzenie schematu systemu ANC, 1 10
2.5 Wyprowadzenie schematu systemu ANC, 2 12
2.6 Wyprowadzenie schematu systemu ANC, 3 13
2.7 Wyprowadzenie schematu systemu ANC, 4 15
2.8 Schemat systemu ANC 16
2.9 Schemat systemu ANC z normalizacją danych wejściowych 18
2.10 Schemat eksperymentu identyfikacji metodami offline 20
2.11 Schemat systemu ANC z uwzględnieniem dodatkowego sprzężenia 21
2.12 Schemat systemu ANC z niwelacją sprzężenia 21
2.13 Identyfikacja sprzężenia 22

3.1 Architektura zaprojektowanego systemu 29

4.1 Architektura sprzętowa - najwyższy poziom 31
4.2 Diagram dystrybucji zasilania 34
4.3 Sekcja wejściowa 36
4.4 Symetryzacja szyn zasilających 37
4.5 Zasilanie domeny cyfrowej 38
4.6 Referencja domeny analogowo-cyfrowej 39
4.7 Układ napięcia odniesienia 40
4.8 Schemat blokowy wejściowego toru analogowego 40
4.9 Wejście zbalansowane 41
4.10 Filtr EMI wraz z ogranicznikiem diodowym 42
4.11 Wzmacniacz różnicowy 42
4.12 Wzmocnienie różnicowe 43
4.13 Topologia Sellena-Keya z jednostkowym wzmocnieniem 44

4.14 Charakterystyka aplitudowa i fazowa układu filtra 45
4.15 Filtr antyaliasingowy 46
4.16 Układ pozycjonujący 47
4.17 Filtr dolnoprzepustowy i limiter 47
4.18 Pełen schemat wejściowej sekcji analogowej 48
4.19 Schemat blokowy wyjściowego toru analogowego 48
4.20 Wejściowy filtr pasmowoprzepustowy 49
4.21 Wzmocnienie sekcji wyjściowej 49
4.22 Układy sterujące wyjściem analogowym 50
4.23 Filtracja końcowa toru analogowego 51
4.24 Pełen schemat wyjściowej sekcji analogowej 51
4.25 Diagram czasowy komunikacji I^2S 53
4.26 Diagram czasowy komunikacji *Left justified* I^2S 54
4.27 Ideowy diagram transmisji SPI w trybie 3 55
4.28 Diagram połączeń sekcji przetworników 56
4.29 Zapewnienie powrotów przez kondensatory odsprzęgające 57
4.30 Zmiany warst przez kondensatory odsprzęgające 58
4.31 Problematyczne powroty - nieciągłość plaszczyzny odniesienia 58
4.32 Poprawne ułożenie sekcji 59
4.33 Rozłożenie warstw 59

5.1 Przebieg wywłaszczenia zadania 62
5.2 Przebieg wywłaszczenia zadania w sytuacji wywołania systemowego 62
5.3 Przebieg problemu inwersji priorytetów 63
5.4 Wywłaszczenie zadania z przestrzeni jądra 63
5.5 Dziedziczenie priorytetu 64
5.6 Porównanie obsługi przerwań sprzętowych 64
5.7 Proces uruchomienia systemu operacyjnego 65
5.8 Diagram aplikacji 71
5.9 Podział bufora sterownika 72
5.10 Porównanie operacji wektorowych 74
5.11 Przykładowy LFSR 76
5.12 Funkcja autokorelacji i GWM sygnału z generatora *xoroshiro16++* 76
5.13 Przebieg operacji - pętla ANC 77

6.1 Sporządzona płytka drukowana 79
6.2 Pomiary wzmacniacza wyjściowej sekcji analogowej 80
6.3 Pomiary filtra wyjściowej sekcji analogowej 81
6.4 Wyjście różnicowe wyjściowej sekcji analogowej 82
6.5 Pomiary wejściowego toru analogowego dla $f_s = 200$ Hz 82

6.6 Gęstość widmowa mocy pomiaru dla $f_s = 200$ Hz 83
6.7 Pomiary wejściowego toru analogowego dla $f_s = 500$ Hz 83
6.8 Gęstość widmowa mocy pomiaru dla $f_s = 500$ Hz 84
6.9 Stanowisko pomiarowe 84
6.10 Indentyfikacja toru wtórnego — wpływ n_f 85
6.11 Przebiegi referencyjne 85
6.12 Suma sinusoid o $f_0 = 120$ Hz i $f_1 = 160$ Hz 86
6.13 Przebiegi $e(n)$ dla sygnałów testowych 87
6.14 Badania wpływu rozmiaru kroku α — sygnał 1 88
6.15 Badania wpływu rozmiaru kroku α — sygnał 2 88
6.16 Badania wpływu rozmiaru kroku α - sygnał 3 89
6.17 Badania wpływu długości L filtra $W(z^{-1})$ — sygnał 1 90
6.18 Badania wpływu długości L filtra $W(z^{-1})$ — sygnał 2 90
6.19 Badania wpływu długości L filtra $W(z^{-1})$ — sygnał 3 91
6.20 Badanie wpływu częstotliwości próbkowania — sygnał 1 92
6.21 Badanie wpływu częstotliwości próbkowania — sygnał 2 92
6.22 Badanie wpływu częstotliwości próbkowania — sygnał 3 92
6.23 Badanie wpływu rozmiaru fragmentu — sygnał 1 93
6.24 Badanie wpływu rozmiaru fragmentu — sygnał 2 94
6.25 Badanie wpływu rozmiaru fragmentu — sygnał 3 94
6.26 Badania wpływu poziomu priorytetu P — sygnał 1 95
6.27 Badania wpływu poziomu priorytetu P — sygnał 2 95
6.28 Badania wpływu poziomu priorytetu P — sygnał 3 96
6.29 Badania rozszerzeń czasu rzeczywistego — zmiana wytycznej planisty . . . 96
6.30 Eksperyment — pseudolosowe pobudzenia sinusoidalne 97
6.31 Sygnał testowy — szum losowy (po przybliżeniu i normalizacji) 98
6.32 Sygnał losowy — pierwszy eksperyment 98
6.33 Eksperyment — hałas generowany przez pracujące młyny kulowe 99
6.34 Eksperyment — hałas generowany przez pracujące pompy wody 99
6.35 Eksperyment - jądro bez łatek PREEMPT_RT 100

Spis tablic

4.1 Wpływ wartości CPOL na charakterystyke transmisji 54
4.2 Wpływ wartości CPHA na charakterystyke transmisji 55
4.3 Realizacja połączeń SPI do I^2S 55

5.1 Konfiguracja wyprowadzeń McASP w drzewie urządzeń 69
5.2 Test wykorzystania SIMD 74

6.1 Pomiary sekcji zasilania 80
6.2 Wybrany zestaw parametrów referencyjnych 86
6.3 Wybrany zestaw parametrów referencyjnych 89